

From Limit-Order-Book Information to Executable Decisions:

Signals, Queues, Deterministic Simulation, and LLM-Assisted Model Discovery

Vincent Maciejewski

M2 Technologies

mayeski@gmail.com

Draft — October 2, 2026

Abstract

The limit order book (LOB) is the principal observable representation of liquidity in electronic markets. A large literature extracts predictive information from it: order-flow imbalance, queue imbalance and the micro-price; stochastic queue and first-passage models; Hawkes and neural point processes; and convolutional, recurrent, attention-based and spatial-temporal Transformer models, pretrained message-level encoders, generative models and hybrids that combine statistical structure with learned representations. A substantial part of that literature evaluates models primarily as statistical forecasters, and large independent re-tests find that many published deep models lose much of their accuracy on new data. This survey asks a different question: *how does information contained in the limit order book become an executable trading decision, and how should prediction, queue dynamics, simulation and execution be evaluated together?*

We organise the literature along four axes—representation, model class, prediction objective and economic role—and follow a single chain throughout: LOB state $\rightarrow$ representation $\rightarrow$ prediction $\rightarrow$ decision $\rightarrow$ queue $\rightarrow$ execution $\rightarrow$ P&L. Each link can destroy information or add uncertainty. We distinguish models that predict prices or events from models that estimate execution outcomes—fill probability, queue depletion, adverse selection—and argue that residual queue position and conditional execution value form the missing translation layer between LOB prediction research and trading.

Two further parts are specific to this survey. First, we describe a deterministic, queue-exact replay architecture (the Kaspar system for CME futures) in which simulated orders enter the reconstructed historical first-in-first-out queue and are filled only by recorded executions that reach an order behind them; the same strategy and order-management code runs in replay and live modes. Second, we describe LLM-Assisted Model Discovery (LAMD), in which a language model proposes candidate models while fitting, replay on observed market data, validation and statistical certification remain external to the language model and reproducible under fixed inputs and procedures. We take the position that the market is observed, not generated: within the methodology proposed here, language models generate hypotheses and candidate models, not the market data used as evidence.

Two criteria organise the evaluation: *tradeability*, under which predictive performance is judged after latency, spread, fees, queue position and adverse selection; and *Occam's razor*, under which each additional layer of complexity—model capacity, hardware, and the breadth of the search that produced the model—must earn incremental out-of-sample economic value. This is a theoretical and methodological paper: all numerical examples are stylised, and no new experiments are reported.

Keywords: limit order book, market microstructure, order-flow imbalance, micro-price, Hawkes process, queue position, first-passage time, adverse selection, deterministic simulation, deep learning, spatial-temporal Transformer, foundation models, uncertainty quantification, multiple testing, deflated Sharpe ratio, Occam’s razor, LLM-assisted model discovery.

Contents

I	The Problem	7
1	Introduction	7
1.1	The limit order book as a dynamic market state	7
1.2	Notation	10
1.3	A short probability primer	12
1.4	Prediction versus tradeability	12
1.5	Prediction, execution and simulation are different problems	14
1.6	Three fixed positions	15
1.7	How the field developed	15
1.8	Positioning and taxonomy	16
2	Survey methodology	19
2.1	Scope: which markets and which questions	19
2.2	How the sources were chosen	20
2.3	How strongly to believe a claim	20
2.4	What makes two LOB studies comparable?	21
2.5	A reader’s checklist	21
II	Models of the Limit Order Book	22
3	Stylised facts a model should reproduce	23
4	Order-flow imbalance and queue imbalance	24
4.1	Order-flow imbalance	24
4.2	Evidence and extensions	25
4.3	Queue imbalance and richer book features	27
4.4	The micro-price	28
4.5	The shape of the book	29
5	Events rather than snapshots	30
6	Queue depletion and first-passage models	31
6.1	The Cont–Stoikov–Talreja model	32
6.1.1	The model	32
6.1.2	Estimating the model	33
6.1.3	The key simplification: two independent birth–death queues	33
6.1.4	Three probabilities a trader needs	34
6.1.5	What the model reproduces	34
6.1.6	A trading test, and what it shows	35

6.1.7	Limitations and descendants	35
6.2	The Cont-de Larrard model	36
6.3	The queue-reactive model	37
6.4	Decision rules from first-passage times	39
7	Hawkes processes	39
7.1	Why not a Poisson process?	40
7.2	The multivariate model	41
7.3	Reading the branching matrix	42
7.4	What Hawkes models are for	42
7.5	From events to prices	43
7.6	What fitted Hawkes models find	44
7.7	Near-criticality	44
7.8	State-dependent Hawkes models	44
7.9	Fitting Hawkes models in practice	45
7.10	From intensities to decisions	47
7.11	Variants of the Hawkes model	48
7.12	Neural point processes	49
8	Deep learning for limit order books	52
8.1	Problem formulations	52
8.2	Convolutional and recurrent models	53
8.3	Attention and spatial-temporal models	54
8.4	State-space models	55
8.5	What the large benchmark studies show	56
8.6	Representation versus architecture	57
8.7	Prediction targets	57
8.8	Fill-probability models	58
8.9	Generative and foundation models	58
8.10	Foundation encoders and uncertainty	59
9	Hybrid models	60
9.1	What the statistical layer supplies	61
9.2	Ways to combine the two	61
9.3	Published hybrids	62
10	Cross-asset information	63
III	From Prediction to Execution	64
11	Passive and aggressive execution	65
11.1	Signal value and execution value	65
12	Queue position and fill probability	66
12.1	Residual queue position	66

13 Adverse selection	66
13.1 Conditioning on the fill	67
13.2 Queue position as an information variable	67
14 Latency and signal decay	67
15 Replay and deterministic simulation	68
15.1 What a naive backtest assumes	68
15.2 What a queue-exact replay requires	68
IV Case Study: The Kaspar HFT Architecture	69
16 Architecture overview	69
17 Queue-exact simulated fills	70
17.1 The book holds every order	70
17.2 Placement and latency	70
17.3 The fill rule	70
18 The replay pipeline	71
19 Determinism	72
20 Replay, paper and live equivalence	72
21 A model-free execution benchmark	73
22 Measurement	73
V LLM-Assisted Model Discovery	74
23 Three uses of generative models, kept apart	74
24 The lamd loop	75
24.1 A worked implementation	76
25 Hypothesis generation is not validation	77
26 Search versus abduction	77
27 Search complexity and multiple testing	77
VI Evaluation	78
28 Statistical metrics	79
29 Economic metrics	79

30 Temporal validation	79
31 Capacity, generalisation and multiple testing	80
31.1 Why simpler models often generalise better	80
31.2 Multiple testing and the deflated Sharpe ratio	81
31.3 Backtestability	81
32 Occam’s razor as a model-selection principle	82
33 Benchmarking standard and datasets	82
VII Hardware, Failure Modes, and Outlook	83
34 Hardware acceleration and the infrastructure constraint	83
34.1 Determinism as a risk-control property	83
34.2 Book-building latency versus inference latency	84
34.3 The commercial FPGA landscape	84
34.4 Between FPGA and GPU	84
34.5 Transformers on FPGAs	84
34.6 GPUs: throughput for research	85
34.7 Networks and distance	85
34.8 Workload placement	85
34.9 Hardware cannot extend signal lifetime	85
34.10 Open questions in hardware	86
35 Failure modes	86
36 Research agenda	87
37 Conclusion	89
A Symbols and acronyms	97
A.1 Symbols	97
A.2 Acronyms	99
B Glossary	99
B.1 Markets and trading	99
B.2 Exchanges and market data	101
B.3 Statistics and evaluation	102
B.4 Machine learning	104
B.5 Probability and stochastic processes	106
B.6 Systems and hardware	107
B.7 Economics	107
C Point-process background	108
C.1 The Poisson process	108
C.2 Points, processes and marks	110
C.3 The probability space	110
C.4 History: information accumulated over time	110

C.5	The counting process	110
C.6	Conditional intensity	111
C.7	Three canonical models: Poisson, renewal and Hawkes	111
C.8	Branching ratio	112
C.9	What Hawkes processes look like	112
C.10	Three signatures that separate Poisson, renewal and Hawkes processes	113
C.11	Fano factor	113
C.12	Hurst exponent from the scaling of the Fano factor	113
C.13	Fitting an exponential-kernel Hawkes process	114
C.14	Symbols used in this appendix	114

Part I

The Problem

About this part. Part I sets up the problem before any model appears. It explains what a limit order book is and how it is built from exchange messages, fixes the notation and the probability used throughout, and makes the survey’s central distinction: predicting the next price move is not the same as making money from it. Section 1 covers the market, the notation and the three principles that organise the survey; Section 2 explains how the literature was chosen and how strongly each kind of claim should be believed.

1 Introduction

This section introduces the object of study and the language used to describe it. It explains how an electronic market matches buyers and sellers through a limit order book, what the book looks like as data, and how exchanges publish it; then fixes the notation and the few ideas from probability that the rest of the survey needs. It ends with the survey’s main argument—prediction, execution and simulation are different problems—and the three principles and four axes used to organise the literature.

1.1 The limit order book as a dynamic market state

Modern electronic markets operate as *continuous double auctions*: buyers and sellers may submit orders at any moment, and a trade happens whenever a buy price meets a sell price. The study of how these trading mechanisms turn orders and information into prices is called *market microstructure* (O’Hara, 1995; Harris, 2002; Hasbrouck, 2007). *Liquidity*—how easily one can trade without moving the price—is supplied by resting limit orders and consumed by aggressive orders, and the visible limit order book records the outcome at a resolution that ranges from individual messages to aggregated price levels (Gould et al., 2013; Bouchaud et al., 2018).

This survey is written for readers with a computer-science or electrical-engineering background and no prior finance. Market, statistical and machine-learning terms are explained where they first appear, and Appendix B collects them with references. Most examples use *futures*: standardised exchange-traded contracts to buy or sell an asset at a set future date (Hull, 2022). The running examples are the E-mini S&P 500 future (ES, on an index of 500 large US stocks), the E-mini Nasdaq-100 future (NQ) and the 10-year US Treasury note future (ZN), all traded on CME Group’s electronic platform. Each such contract is an *instrument*.

Orders and order types. An *order* is an instruction to buy or sell a quantity of a contract. The quantity is counted in *lots* (one futures contract, or one share), and prices are restricted to multiples of the *tick*, the smallest allowed price step. The main order types are the following (Harris, 2002).

- A *limit order* says “buy (or sell) up to this many lots, at this price or better”. Whatever cannot trade immediately waits in the book. Waiting buy orders are *bids*; waiting sell orders are *asks* (or *offers*).
- A *market order* says “buy (or sell) this many lots now, at whatever price is available”. It trades immediately against waiting orders and never waits itself. A limit order priced to trade immediately (a buy at or above the best ask) behaves the same way; both are called *aggressive*, and the trader is said to *cross the spread*. A trader whose limit order waits in the book is *passive*.

- A *cancel* removes a waiting order, completely or partly; a *modify* (or *replace*) changes its price or quantity.
- *Time-in-force* qualifiers say how long an order may wait: a *day* order waits until the session ends; an *immediate-or-cancel* (IOC) order trades what it can at once and cancels the rest; a *fill-or-kill* (FOK) order trades in full at once or not at all.
- An *iceberg* order shows only part of its quantity; when the visible part trades, more appears. A fully *hidden* order shows nothing. Both trade but are partly or wholly invisible in the book.

The book is a data structure. The *limit order book* (LOB) is the set of all waiting orders for one instrument. Each side is naturally implemented as an ordered map from price to a queue of orders: bids sorted from highest price down, asks from lowest price up, with a hash map from order identifier to order so that a cancel can find its order in constant time. The *best bid* is the highest bid price, the *best ask* the lowest ask price. When an aggressive buy arrives, the exchange’s *matching engine* trades it against the best ask, then the next price up, and so on until the buy is filled or its limit price is reached; trading through several price levels is called *walking the book*. Figure 1 shows the structure.

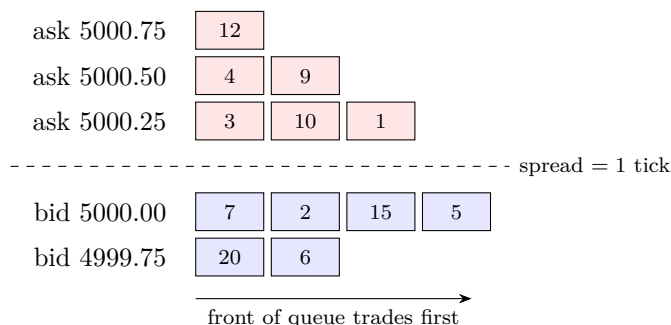


Figure 1: A small book. Each box is one waiting order and the number is its size in lots. Orders at the same price form a first-in-first-out queue, front on the left. The best bid is 5000.00 with $7 + 2 + 15 + 5 = 29$ lots; the best ask is 5000.25 with 14 lots; the spread is one tick of 0.25.

Queues and priority. Which waiting order trades first is decided by the venue’s *priority rule*. Most venues use *price-time priority*: better prices trade first, and among orders at the same price the one that arrived first trades first. Each price level is then exactly a FIFO queue. Some products use *pro-rata* allocation instead, in which an incoming order is split among the waiting orders in proportion to their sizes; at CME, for example, most futures match first-in-first-out while SOFR interest-rate futures use a pro-rata or allocation algorithm (CME Group, n.d.b). Under price-time priority an order’s place in the queue matters, and the rules for keeping it matter too: on CME Globex, decreasing an order’s quantity keeps its place, while increasing the quantity or changing the price sends it to the back of the queue (CME Group, n.d.b). The volume waiting ahead of an order is its *queue position*; much of this survey is about why that number matters.

What a market-data feed shows. Exchanges publish the book through market-data feeds at three levels of detail. *Level 1* gives only the best bid and ask and their sizes. *Level 2*, or *market by price* (MBP), gives the total size at each of the best few price levels. *Level 3*, or *market by order* (MBO), reports every individual order: each addition, execution, cancellation and modification, with an order identifier. MBO data provide the information needed to reconstruct the displayed

order-level queues, and therefore to estimate queue positions, subject to the venue’s matching rules and feed semantics; hidden liquidity and feed gaps remain outside what any feed shows.

Example: building the book from Nasdaq ITCH. Nasdaq’s TotalView-ITCH 5.0 feed is a widely used MBO protocol and a clear example of how a book is built from messages ([Nasdaq, 2015](#)). Every message is a small fixed-layout binary record with a one-character type, a timestamp in nanoseconds since midnight, and, for order messages, an 8-byte *order reference number* that identifies the order for the rest of the day. The order messages are:

Type	Name	Effect on the book
A, F	Add Order	new order (side, shares, price) joins the back of its queue
E	Order Executed	order traded; reduce its shares by the executed amount
C	Order Executed With Price	as E, at a price different from the order’s own
X	Order Cancel	partial cancel; reduce its shares
D	Order Delete	order cancelled; remove it
U	Order Replace	remove the old order; add a new one with a new reference number
P	Trade (non-cross)	an execution against a non-displayed order; no book change

Building the book is a short event loop over these messages:

```
book = {'B': SortedDict(), 'S': SortedDict()} # price -> deque of orders
orders = {} # reference number -> order

def on_message(m):
    if m.type in 'AF': # add: join the back of the queue
        o = Order(m.ref, m.side, m.price, m.shares)
        book[o.side].setdefault(o.price, deque()).append(o)
        orders[o.ref] = o
    elif m.type in 'ECX': # execution or partial cancel
        o = orders[m.ref]
        o.shares -= m.shares
        if o.shares == 0: remove(o)
    elif m.type == 'D': # full cancel
        remove(orders[m.ref])
    elif m.type == 'U': # replace: remove, re-add at back
        old = orders[m.old_ref]
        remove(old)
        on_message(Add(m.new_ref, old.side, m.price, m.shares))

def remove(o):
    q = book[o.side][o.price]
    q.remove(o) # O(1) with a linked list
    if not q: del book[o.side][o.price]
    del orders[o.ref]
```

Three details matter later in this survey. First, an execution message names the waiting order that traded, so the feed reveals *which* order in the queue was hit—the information needed to reconstruct queue positions (Part IV). Second, a replace carries a new reference number, so in the reconstructed book the order leaves its place and rejoins at the back. Third, hidden orders produce only P messages and never enter the book: the book a feed lets you build is the *displayed* book. ITCH is delivered over a sequenced UDP multicast protocol (MoldUDP64), so a receiver can detect lost packets by gaps in the sequence numbers. CME’s MDP 3.0 market-by-order feed, used by the system in Part IV, follows the same pattern with its own binary encoding ([CME Group, n.d.c](#)).

At time t , an L -level description of the book is

$$\mathcal{L}_t = \left\{ p_{\ell,t}^b, v_{\ell,t}^b, p_{\ell,t}^a, v_{\ell,t}^a \right\}_{\ell=1}^L, \quad (1)$$

where $p_{\ell,t}^b$ and $v_{\ell,t}^b$ are the price and displayed volume at the ℓ -th best bid level ($\ell = 1$ is the best bid), and $p_{\ell,t}^a$, $v_{\ell,t}^a$ the same on the ask side.

In words: this is the market-by-price view of the book: a table with L rows per side, each row a price and the total number of lots waiting there. It can be computed from the order-level book by summing each queue. It is the state of the market, and it changes every time an order arrives, trades or is cancelled.

The mid-price and spread,

$$m_t = \frac{1}{2} \left(p_{1,t}^a + p_{1,t}^b \right), \quad s_t = p_{1,t}^a - p_{1,t}^b, \quad (2)$$

In words: the mid m_t is halfway between the best price anyone will buy at and the best price anyone will sell at; it is the usual single-number “price” of the market. The spread s_t is the gap between them: buying and immediately selling back costs one spread.

These are convenient summaries that discard most of the book: the distribution of liquidity across levels, the sequence of events that produced the current state, and the queue position of any individual order.

Throughout, numerical examples use stylised values chosen to make the arithmetic visible. They are not empirical estimates, and this paper reports no new experiments.

Example 1.1 (Same mid, different market). Two E-mini S&P 500 (ES) books have the same mid (5000.125) and the same one-tick spread (tick = 0.25). In the first, 12 lots rest at the best ask; in the second, 900. Buying 100 lots aggressively costs several ticks more in the first book than in the second. Mid and spread do not distinguish them.

Three properties make LOB modelling unusually hard. The data are *event-driven*: orders arrive, execute and cancel at irregular times. The state is *high-dimensional* even when only a few levels are observed. And the statistical properties are *non-stationary* across time of day, assets, volatility regimes and market-structure changes. A model that performs well for one representation, horizon or market is therefore not, by that fact, a model of order-book dynamics.

1.2 Notation

This subsection fixes the symbols used throughout, and explains what each one means for a trader watching the book. Symbols used in only one section are defined where they first appear. To make the symbols concrete, each group is followed by its value in one running example: the book of Figure 1, where at time t we decide to place a passive buy order for one lot at 5000.00.

Time and events

t is time. Book events (additions, cancellations, executions) are numbered $n = 1, 2, \dots$ and happen at times $t_1 \leq t_2 \leq \dots$. A quantity written with subscript t is its value just after everything that happened up to and including t ; $t-$ means “just before t ”. $H > 0$ is a horizon—how far ahead we look—and δ is the tick size.

What it means: the market is a stream of events, and we look at it as a sequence of snapshots taken whenever we like. H is the question “how long am I willing to wait?”

Example: $\delta = 0.25$ (one ES tick), and we look $H = 1$ s ahead.

Book

$p_{\ell,t}^b, v_{\ell,t}^b, p_{\ell,t}^a, v_{\ell,t}^a$, the mid m_t and the spread s_t are as defined in Section 1. The change in the mid over the next H seconds is $\Delta m_{t,H} = m_{t+H} - m_t$.

What it means: $\Delta m_{t,H}$ is “how much did the price go up (or down, if negative) while I waited”. Almost every prediction model in the literature targets some function of it.

Example: $p_{1,t}^b = 5000.00, v_{1,t}^b = 29, p_{1,t}^a = 5000.25, v_{1,t}^a = 14$, so $m_t = 5000.125$ and $s_t = 0.25$. If one second later the best bid and ask are 4999.75 and 5000.00, then $m_{t+H} = 4999.875$ and $\Delta m_{t,H} = -0.25$: the price fell one tick.

Information and prediction

X_t is everything observable at t that a model may use: the book, recent events, and anything derived from them. Y_{t+H} is the quantity we want to predict, which becomes known only at $t + H$; $\hat{Y}_t$ is a model’s prediction of it, made at t .

What it means: X_t is the model’s input vector and $\hat{Y}_t$ its output. The rule is that X_t may contain only what was really known at t —no peeking at the future.

Example: X_t could be the sizes of the five best levels on each side; Y_{t+H} could be “did the mid rise within one second?”, and $\hat{Y}_t = 0.6$ a model’s estimate that it will.

Actions and orders

a_t is what we do at t : place an order, keep it, cancel it, or cross the spread. d is the side of an order, $d = +1$ for a buy and $d = -1$ for a sell. For a waiting order, Q_t^{ahead} is the displayed volume ahead of it in its queue, and q stands for a particular value of Q_t^{ahead} .

What it means: d lets one formula serve both buyers and sellers, since a rising price is good for a buyer and bad for a seller. Q_t^{ahead} is “how many people are in line in front of me”.

Example: $a_t =$ place a bid at 5000.00, $d = +1$. The new order joins the back of the 5000.00 queue, so $Q_t^{\text{ahead}} = 29$.

Fills

τ_F is how long after t the order is filled ($\tau_F = \infty$ if it never is), $t_F = t + \tau_F$ is the clock time of the fill, and p^{fill} is the price received. Fill is the event “the order was filled within the horizon”, $\{\tau_F \leq H\}$, and $\mathbb{1}_{\text{Fill}}$ is 1 if that happened and 0 otherwise.

What it means: a passive order is a bet that someone will come and trade with it; these symbols record whether, when and at what price they did.

Example: sellers trade through the whole 5000.00 queue 0.4 s later and reach our order: $\tau_F = 0.4$ s, $p^{\text{fill}} = 5000.00$, and since $0.4 \leq 1$, $\mathbb{1}_{\text{Fill}} = 1$.

Profit

Π is the profit per lot of an order decided at t , in price points unless stated otherwise (some examples use ticks); it is defined precisely in Section 11. A passive order that is never filled makes $\Pi = 0$. $h > 0$ is the *mark-out horizon*: how long after a fill we wait before scoring it.

What it means: Π is the number we actually care about; every other symbol is a step on the way to it.

Example: with $h = 1$ s, if the mid one second after our fill is 4999.875, our buy at 5000.00 is worth $4999.875 - 5000.00 = -0.125$: we were filled, and then the price moved against us. This is *adverse selection* (Section 13), and it is why a fill is not automatically good news.

Models

M denotes a model, M_0 a baseline (usually the simplest reasonable one) and M_1 a candidate. $V(M)$ is how much money-relevant value M delivers *out of sample* (OOS)—on data that were not used to build it, as opposed to *in sample*—for example net profit after costs, or

a Sharpe ratio (average return divided by the standard deviation of return (Sharpe, 1994)).
 $\Delta V = V(M_1) - V(M_0)$.

What it means: ΔV answers “is the new model worth it, compared with the simple one, on data it has never seen?”

Example: if a linear model earns 0.010 points per trade after costs and a neural network earns 0.012, then $\Delta V = 0.002$ points per trade; Principle 3 below asks whether that is real and whether it pays for the extra complexity.

Operators

$\mathbb{P}$ is “the probability of”, $\mathbb{E}$ is “the average of”, and a vertical bar means “given”. Section 1.3 explains these in plain terms.

1.3 A short probability primer

Most equations in this paper are statements about probabilities and averages. The few ideas needed are the following; a first course in probability covers all of them.

$\mathbb{P}(A)$, “the probability of A ”

A number between 0 and 1: the fraction of times event A would happen if the situation were repeated many times. Example: $\mathbb{P}(\Delta m_{t,H} > 0)$ is the fraction of moments after which the mid is higher H seconds later.

$\mathbb{P}(A \mid B)$, “the probability of A given B ”

The same fraction, but counted only over the cases where B is true. The vertical bar is read “given”. Example: $\mathbb{P}(\Delta m_{t,H} > 0 \mid \text{bid queue much larger than ask queue})$ asks: of all the moments when buyers were queued much more heavily than sellers, what fraction were followed by a rise? Writing $\mid X_t$ means “given everything we can observe at time t ”—it is what a prediction model computes: a function from the current inputs X_t to a probability.

$\mathbb{E}[Z]$, “the expected value of Z ”

The long-run average of a random number Z . If a trade makes +2 with probability 0.6 and -1 with probability 0.4, then $\mathbb{E}[Z] = 0.6 \times 2 + 0.4 \times (-1) = 0.8$.

$\mathbb{E}[Z \mid B]$

The average of Z over only the cases where B is true.

$\mathbb{I}_A$, the indicator of A

Equal to 1 if A happens and 0 otherwise, like a Boolean cast to an integer. Its average is the probability: $\mathbb{E}[\mathbb{I}_A] = \mathbb{P}(A)$. A useful consequence, used in Section 11, is $\mathbb{E}[Z \mathbb{I}_A] = \mathbb{P}(A) \mathbb{E}[Z \mid A]$: the average of “ Z if A happened, else 0” equals how often A happens times the average of Z when it does.

$\inf\{u > 0 : \text{condition}\}$

“The earliest time u after now at which the condition holds”—the time a **while** loop that polls the condition would exit.

A hat, as in $\hat{Y}_t$

An estimate produced by a model, as opposed to the true value.

1.4 Prediction versus tradeability

Suppose a model estimates $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$, the probability that the mid rises over the next H . That number describes directional predictability. It does not say whether a trader can monetise it.

An aggressive buy pays the offer and must overcome the half spread and fees. A passive buy avoids the spread but may never execute, and the event that makes it execute may itself carry information about the next price move. The economically relevant sequence is

$$\begin{aligned} X_t \rightarrow \hat{Y}_t \rightarrow a_t \rightarrow \text{placement} \rightarrow \text{queue position} \\ \rightarrow \text{execution} \rightarrow \text{realised profit and loss (P\&L)}. \end{aligned} \tag{3}$$

In words: data go into a model; the model makes a forecast; the forecast is turned into a decision (buy, sell, wait, cancel); the decision becomes an order; the order takes a place in a queue; whether and when it trades depends on that place; and only then is money made or lost. A forecast can be right and still be lost at any later arrow.

Example 1.2 (Accurate but untradeable). In ES (tick 0.25 points, one-tick spread), a classifier is right on the sign of the next mid move 55% of the time, and the typical move is half a tick (0.125 points). The expected move per trade in the predicted direction is

$$(0.55 - 0.45) \times 0.125 = 0.0125 \text{ points},$$

while an aggressive entry costs the half spread, 0.125 points, before fees. The expected directional edge is one tenth of the half-spread cost, before fees, so it does not cover even the cost of entry. The accuracy is real, and the strategy loses money. The same signal used to decide *which resting passive order to keep and which to cancel* pays no spread; whether it helps is an execution question (Part III), not an accuracy question.

A related line of work asks not only for a forecast but for a measure of *when the forecast can be trusted*. UQ-LOB (Edward Manoharan et al., 2026) attaches an uncertainty module to a pretrained LOB encoder and outputs a *calibrated* distribution—one whose stated probabilities match observed frequencies, so that events given 70% happen about 70% of the time (Guo et al., 2017)—with a scalar confidence. Restricting attention to the most confident tenth (*decile*) of predictions raises directional macro-F1 by 0.11–0.15 in its regression variant (F1 is the harmonic mean of precision and recall; *macro*-F1 averages it over the classes “up”, “down” and “stationary”), across seven cryptocurrency assets at 5, 10 and 15 second horizons. Selective prediction changes the tradeability arithmetic directly: if a confident tier moved accuracy from 55% to 70% on a tenth of the signals, the expected edge per trade in Example 1.2 would rise from 0.10 to 0.40 of the typical move, with one tenth as many trades. Whether that clears the half spread still depends on the size of the moves in the confident tier. UQ-LOB itself stops at directional reliability: its authors state that they evaluate reliability, not profitability, and that a trading simulation with realistic execution and fees is required before any claim about returns (Edward Manoharan et al., 2026). That is the step this survey is about.

This is the central question of the survey:

Which information extracted from the limit order book survives the complete translation from statistical prediction to executable trading?

It changes how models should be compared. A model that improves classification accuracy by a fraction of a percentage point is not more useful than a linear model if the improvement vanishes after latency and costs. A model with modest directional accuracy may still be valuable if it improves the timing or selection of passive executions.

1.5 Prediction, execution and simulation are different problems

LOB research frequently mixes up three different questions. Each is written below as a probability or an expected value (Section 1.3). The big $\mathbb{P}$ is “the probability of”, $\mathbb{E}$ is “the average of”, and the vertical bar is read “given”: everything to the right of the bar is what we already know, everything to the left is what we want to know.

$$\text{Prediction: } \mathbb{P}(Y_{t+H} \mid X_t), \quad (4)$$

$$\text{Execution: } \mathbb{P}(\text{Fill} \mid X_t, Q_t^{\text{ahead}}, a_t) \text{ or } \mathbb{E}[\Pi \mid X_t, Q_t^{\text{ahead}}, a_t], \quad (5)$$

$$\text{Simulation: } \mathbb{P}(\{X_u\}_{u \in (t, t+H]} \mid X_t). \quad (6)$$

In words:

- *Prediction* (4): given what we can see now, how likely is each possible value of some future quantity Y —for example, “will the mid be higher in one second?” It is a question about the market, and our own orders play no part in it.
- *Execution* (5): given what we can see now, how much volume is ahead of our order, and what we chose to do, how likely is our order to be filled within H seconds—or, in the second form, how much do we expect to earn? It is a question about *our* order, and its answer changes with our action a_t .
- *Simulation* (6): given what we can see now, how likely is each possible *whole future path* of the market over the next H seconds? A simulator draws sample paths from this distribution. The difference from (4) is the difference between a weather simulator, which produces whole plausible tomorrows, and a forecast of tomorrow’s maximum temperature.

Two different activities are both called “simulation” in this literature, and the survey keeps them apart. *Generative market simulation* draws new market paths from a model of (6): Hawkes simulators, agent-based markets and the generative networks of Section 8.9. *Historical replay* does not draw paths at all: it reconstructs the one path that was recorded, event by event, and evaluates the counterfactual outcome of the researcher’s own orders against it—an estimate of (5) on observed data. The Kaspar system of Part IV is a replay system, not a generative simulator. They are related but not interchangeable. A Hawkes process can generate realistic clustered order flow without producing directional alpha. A survival model can give accurate fill probabilities while playing no role in price prediction. A Transformer can predict short-horizon returns without modelling queue dynamics. Each model must be evaluated against the problem it solves.

Table 1: One event stream (a day of ES market-by-order data), three models, three evaluation criteria. Using the wrong column is a common source of misleading comparisons.

Model	Target	Appropriate metric	Inappropriate metric
Hawkes, 4 event types	event intensity	point-process likelihood; time-rescaling residual test	directional accuracy
Survival model	$\mathbb{P}(\tau_F \leq 1 \text{ s} \mid Q_t^{\text{ahead}} = q)$	survival likelihood; calibration by queue bucket	mid-move AUC
Classifier	$\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$	log loss, AUC, then net P&L	fill rate

1.6 Three fixed positions

Three positions recur throughout the paper.

Principle 1 (The market is observed, not generated). Recorded exchange data are the ground truth. A replay simulator reconstructs the historical book from those data; it does not invent market behaviour. The only counterfactual it produces is the execution outcome of the researcher’s own orders.

Principle 2 (Language models generate hypotheses, not data). In LLM-assisted model discovery the language model proposes candidate models. Fitting, replay, validation and certification are carried out outside the language model, by fixed procedures that are reproducible given the same data, configuration and random seeds. The language model does not generate the market data used as evidence.

Principle 3 (Occam’s razor: complexity is an empirical hypothesis). Among models that deliver the same out-of-sample economic value, prefer the simplest. Each added layer—model capacity, specialised hardware, and the breadth of the search that produced the model—is a hypothesis that it adds value, and must earn incremental out-of-sample economic value that survives realistic execution assumptions and the multiple-testing burden of its own selection.

Occam’s razor is applied here to economic value, not to statistical fit alone: a more complex model is not rejected for being complex, but it carries the burden of showing that its extra value is real. Section 32 makes this operational.

1.7 How the field developed

The modelling of order books has passed through five overlapping stages.

1. **Zero-intelligence models.** Orders arrive at random, with no strategy at all, and the matching rules do the rest (Smith et al., 2003). The surprise was how much this explains: on eleven London stocks, a model with one free parameter accounts for 96% of the variance of the bid-ask spread (Farmer et al., 2005). Market structure, not trader intelligence, explains much of what the book looks like.
2. **Stochastic queue models.** The best queues are treated as random walks driven by order arrivals, cancellations and executions, giving closed-form results for the time to the next price change and the probability of its direction (Cont et al., 2010; Cont and de Larrard, 2013), later with rates that depend on queue sizes (Huang et al., 2015).
3. **Self-exciting point processes.** Hawkes models capture the clustering of events and the fact that many events are triggered by earlier ones; estimated branching ratios on E-mini S&P data range from about 0.7–0.8 to about one (Bacry et al., 2015; Filimonov and Sornette, 2012; Hardiman et al., 2013).
4. **Hand-engineered signals.** Order-flow imbalance (Cont et al., 2014), queue imbalance (Gould and Bonart, 2016) and the micro-price (Stoikov, 2018) compress the book into a few numbers with clear economic meaning.
5. **Learned models.** From 2017, convolutional and recurrent networks (Tsantekidis et al., 2017a; Zhang et al., 2019), then attention and spatial-temporal Transformers (Wallbridge, 2020; Berti and Kasneci, 2025), then pretrained message-level encoders and large generative models (Linna et al., 2025; Zhang et al., 2026), together with hybrids that put point-process structure inside networks (Shi and Cartlidge, 2022; Bodor and Carlier, 2025).

Each stage added capacity. Whether each also added *economic* value is the question this survey keeps asking.

1.8 Positioning and taxonomy

The literature has three historical traditions. The first is econometric and reduced-form modelling of prices, order flow and impact (Kyle, 1985; Glosten and Milgrom, 1985; Cont et al., 2014). The second is continuous-time point-process and queueing models: Hawkes processes preserve the event-driven structure and model excitation between event types (Hawkes, 1971; Bacry et al., 2015), while queue-reactive and first-passage models describe displayed liquidity and its depletion (Cont et al., 2010; Huang et al., 2015). The third is machine learning, from hand-engineered features through CNNs, recurrent networks, Transformers, state-space models and, most recently, pretrained message-level encoders (Tsantekidis et al., 2017a; Zhang et al., 2019; Wallbridge, 2020; Linna et al., 2025).

Position relative to existing surveys. Existing surveys and books cover parts of this ground well (Table 2): the LOB as a trading mechanism and its empirical literature (Gould et al., 2013; Tripathi et al., 2020), machine-learning prediction from the book (Zaznov et al., 2022), generative simulation of order books (Jain et al., 2024), and, more recently, large language models and autonomous agents in financial trading (Ding et al., 2024; Dong et al., 2025; Hua et al., 2026). These works provide taxonomies of models and applications, but generally treat LOB modelling, trading prediction, execution and queue position, market simulation, and automated hypothesis search as separate topics. We are not aware of a survey that brings these strands together around the question of how information extracted from the LOB survives the complete translation from statistical prediction to executable trading. That question is also this survey’s criterion for selecting literature: a model or method is included when it bears on one link of the chain—representation, prediction, calibration, decision, queue position, execution, replay, economic value, model search and multiple testing, and LLM-assisted hypothesis generation. The individual subjects are not new; the integration, and the criterion that connects them, are. Two distinctions follow from it and recur throughout: historical replay is distinguished from generative market simulation, and language models are treated as generators of hypotheses, not as sources of market data or of validation.

Generative LOB simulation versus historical replay. The closest neighbouring review, on LOB simulation (Jain et al., 2024), classifies, by methodology, models that *generate* order-book dynamics, and evaluates them by their ability to reproduce stylised facts and price impact. That is one of two legitimate ways of using a simulator, and they answer different questions:

- *generative simulation* asks whether a model can produce plausible market trajectories, including ones never observed, such as stress scenarios;
- *historical replay* asks what would have happened to a particular strategy’s orders against one observed trajectory.

To decide whether an order would have executed against the historical market, the market does not need to be generated, only reconstructed. This survey is concerned primarily with the second question; generative models are covered in Section 8.9 as tools for simulation and stress testing.

Language models: hypothesis generator, not trading agent. The surveys of LLMs in trading review systems in which the language model reads market information, reasons about it and

produces trading decisions that are then backtested (Ding et al., 2024; Hua et al., 2026). This survey does not review that architecture. In the loop of Part V the language model proposes a candidate model from a fixed vocabulary of primitives; in the worked implementation it never receives market prices (Mayeski, 2026c). Data enter only when an external, reproducible research engine compiles the candidate, fits it, evaluates it out of sample and corrects for the number of candidates tried. The implementation described there works on daily data; for order-book strategies the design adds replay of each candidate’s orders against the recorded book, which is the role of Part IV.

The purpose here is therefore not another catalogue of architectures, but an organisation of the literature around the translation from information to decision. Three questions recur: what representation contains the useful information; what model extracts it; and does the result produce incremental economic value after execution constraints?

Table 2: Existing surveys, books and benchmark studies, and what this survey adds.

Work	Focus	Not covered there
Gould et al. (2013)	empirical facts and models of LOBs	deep learning; execution value; queue position in practice
Abergel et al. (2016), Bouchaud et al. (2018)	books on empirical properties, models and simulation of order books and impact	recent deep and generative models; model-search statistics
Tripathi et al. (2020)	systematic review of 103 articles (1981–2018) on the LOB as a trading mechanism and its implications for market efficiency, classified by sub-area, market, method and LOB measure	deep-learning prediction (named there only as a future direction); execution value; queue position
Jain et al. (2024)	review of LOB simulation models and the stylised facts used to validate them	prediction and trading value
Ding et al. (2024), Dong et al. (2025), Hua et al. (2026)	surveys of LLMs as trading and financial agents: architectures, inputs, workflows, benchmarks, backtests, deployment	LOB microstructure; separation of hypothesis generation from evidence
Zaznov et al. (2022)	survey of price-change prediction from the LOB	classical point-process and queue models; execution
Prata et al. (2024), Briola et al. (2025a)	benchmark re-tests of deep models; transaction-level evaluation	classical models; queue position; search statistics
This survey	information-to-decision chain: classical and deep models, queue position, deterministic replay, automated model discovery, evaluation	–

Contributions. The survey offers:

1. a single chain—representation, prediction, decision, queue, execution, profit—that places classical and learned models in one framework;
2. a treatment of queue position, fill probability and adverse selection as modelling targets in their own right, with the signal-value/execution-value distinction (Section 11.1);
3. a description of queue-exact deterministic replay as the evaluation instrument for execution claims (Part IV);
4. an account of LLM-assisted model discovery as a statistical search process (Part V);
5. evaluation standards that combine proper scoring, economic metrics, temporal validation and search-aware significance (Part VI);

6. a reader-oriented apparatus: notation, a probability primer, plain-language explanations of every equation, a glossary and a point-process appendix.

How to read this survey. Readers new to markets should read Part I in full and use the glossary. Readers interested mainly in models should read Part II, then Section 8.5. Practitioners concerned with execution should read Parts III and IV and Section 32. Readers interested in automated research should read Parts V and VI. The appendix on point processes supports Section 7.

Four axes for locating a piece of work

Every model or paper in this survey answers four separate questions, and we describe it by its answer to each. Think of the four answers as the four fields of a record; a work is a point in a four-dimensional space.

Representation: what does the model see?

The input format: raw book snapshots (the table of Equation (1) at sampled times); event or message streams (every add, cancel and trade, in order); order-flow aggregates such as OFI (Section 4); queue state (sizes and positions in the queues); or the state of several instruments at once.

Model class: what kind of function maps input to output?

Linear and econometric models; queue and first-passage models; point processes such as Hawkes; state-space models; convolutional and recurrent networks; Transformers; pretrained encoders; or a hybrid of these.

Objective: what quantity does it output?

Price direction; return; event intensity; volatility; queue depletion time; fill probability; execution cost; or a simulated future.

Economic role: what decision is the output for?

Aggressive trading (crossing the spread); passive trading (resting orders); market making (quoting both sides); execution of a given order; risk measurement; or simulation for research. A paper that evaluates only statistical accuracy and names no decision has an *unspecified* economic role.

The axes are used in three ways.

First, to decide what can be compared. Two results are comparable only if they share an objective and an economic role, and in addition the data fields of Section 2.4. Representation and model class are the things a study varies; objective and role are the things it must hold fixed. A Hawkes model scored on event-intensity likelihood and a Transformer scored on direction accuracy are not competitors, however often they appear in the same table.

Second, to organise the paper. Part II moves along the model-class axis, from linear models to pretrained encoders, and Section 5 along the representation axis. Part III moves along the objective axis from direction to fill probability and adverse selection. Parts IV and V fix the economic role—passive execution—and ask what each objective is worth for it.

Third, to expose a gap. Table 3 places the main works discussed in this survey on the four axes. The economic-role column is where the literature is thinnest: many of the learned models are evaluated on accuracy alone, and the decision their output would drive is left unspecified. The rest of the paper is about filling that column.

Table 3: Selected works placed on the four axes. “Unspecified” means the work evaluates statistical accuracy without stating or evaluating a trading decision. SHADOW-PPOV (Shadow Passive Percentage-of-Volume) is the model-free execution algorithm of Section 21.

Work	Representation	Model class	Objective	Economic role
OFI (Cont et al., 2014)	order-flow aggregate	linear	contemporaneous price change	price impact (execution)
Book dynamics (Cont et al., 2010)	queue state	queueing / first passage	direction; queue depletion	market making, execution
Queue-reactive (Huang et al., 2015)	queue state	queueing (Markov)	event intensity	simulation
Hawkes models (Bacry et al., 2015)	event stream	point process	event intensity	simulation, risk
Queue value (Moallemi and Yuan, 2017)	queue state	queueing	value of a queue position	passive trading, market making
DeepLOB (Zhang et al., 2019)	raw snapshots	CNN + recurrent	direction	aggressive trading (simple trading simulation)
TransLOB (Wallbridge, 2020)	raw snapshots	Transformer	direction	unspecified
Deep OFI (Kolm et al., 2023)	order-flow aggregate	recurrent / feed-forward	return, several horizons	unspecified (scored by out-of-sample variance explained)
LOBERT (Linna et al., 2025)	event stream	pretrained encoder	direction; next message	unspecified
UQ-LOB (Edward Manoharan et al., 2026)	event stream	encoder + uncertainty module	direction with confidence	unspecified (stated by the authors)
SHADOW-PPOV (Maciejewski, 2026d)	event stream	none (rule)	execution cost	passive execution

2 Survey methodology

A survey makes many claims, drawn from many papers that were written for different purposes. This section explains what the survey covers, how its sources were chosen, how strongly each kind of claim should be believed, and what has to be known about two studies before their numbers can be compared. The last part is a practical checklist for reading any paper in this field.

2.1 Scope: which markets and which questions

The survey covers limit order books in four kinds of market. They share the mechanism of Section 1 but differ in ways that change what a model can learn and whether a result carries over from one to another.

Within these markets the survey includes work whose main object is one of four things: *prediction* (forecasting prices or events), *signal extraction* (building features from the book), *execution* (deciding how to place, keep and cancel orders), or *simulation* (reproducing the market for research). Two adjacent areas are deliberately left out. Equilibrium models, which derive prices from the optimal behaviour of rational traders, and agent-based models, which simulate many rule-following traders, appear only where they help explain an LOB model; they are not fitted to the data the rest of the paper is about. Synthetic market data produced by learned generative models are discussed in Section 23, but, following Principle 1, they play no part in the methodology the survey recommends.

Table 4: The markets covered, and how they differ in ways that matter for LOB models.

Market	How it trades	Typical data	What it means for models
Futures (e.g. ES, NQ, ZN)	Each contract trades on one exchange, in nearly continuous sessions	Exchange MBO feeds such as CME MDP 3.0	One book holds all the liquidity, so the book a model sees is the whole market.
US equities	The same stock trades on many exchanges and off-exchange venues at once	Exchange feeds such as ITCH, one venue at a time	One venue’s book is only part of the market; flow elsewhere moves prices too.
Cryptocurrencies	Many independent exchanges, open 24 hours a day, every day	Public exchange APIs, often snapshots plus updates	Data are easy to obtain, but fees, participants and trading hours differ from traditional markets.
Fixed income	Government bond futures on futures exchanges; cash bonds on electronic inter-dealer platforms	Varies by venue	Treasury futures are large-tick instruments: the price rarely moves by more than a tick, queues are long, and queue position matters.

2.2 How the sources were chosen

This is a *narrative* survey, not a *systematic review*. A systematic review fixes a search query and a list of databases in advance, screens every result against written criteria, and reports how many papers were kept at each step, so another team could repeat the search exactly. A narrative survey instead selects work that illustrates and tests an argument. That is appropriate here, because the aim is to organise the field around one question—what survives the translation from prediction to execution—rather than to count papers. It also has a known weakness: the selection may favour work that fits the argument. To limit that, the survey includes work that cuts against its own position where such work exists, for example evidence that deep models given suitable inputs reach state-of-the-art predictive accuracy (Section 8).

A work was included if it made at least one of the following contributions: a representation of the order book; a predictive model; a queue or execution model; an evaluation method; a public data set or benchmark; or a design for simulation or research infrastructure. Preference was given to peer-reviewed work and to widely cited preprints; recent preprints are included when they are the only source for an idea, and are identified as preprints in the bibliography. Every reference was checked against the publisher, the preprint server or the official documentation.

2.3 How strongly to believe a claim

Not all statements in a survey deserve the same confidence. We distinguish four kinds, and the reader should ask of every claim which kind it is.

Established result

Supported directly by a cited study, or true by mathematics. *Example:* a linear Hawkes process is stationary if and only if the cascade of events it triggers dies out (Section 7) (Section 7). This is a theorem; no data can overturn it.

Empirical synthesis

A conclusion drawn by comparing several studies, none of which tested it directly. *Example:* “the input representation can matter as much as the architecture” (Section 8). It rests on a pattern across papers, so it can be wrong for a market those papers did not study.

Methodological recommendation

A proposed standard, argued for rather than proved. *Example:* every result should report how many model configurations were tried (Section 27). Its justification is a statistical argument, not a measurement.

Open hypothesis

Plausible, but the evidence is incomplete or conflicting. *Example:* adding Hawkes intensities to a large network that already sees the same events adds little value (Section 9). The survey argues for it; nobody has shown it.

The distinction matters because LOB studies differ in assets, horizons, targets and data, and a result established under one set of conditions is at best a hypothesis under another. Table 24 in Section 37 classifies the main claims of this paper in these terms.

2.4 What makes two LOB studies comparable?

Papers in this field routinely report a single headline number—an accuracy, an F1 score, a Sharpe ratio—and readers routinely compare those numbers across papers. Such comparisons are meaningful only when the studies agree on the conditions that produced the numbers. Table 5 lists the conditions that matter most and explains how each one can change the headline number on its own, with no change to the model.

Example 2.1 (Accuracy without knowledge). A three-class target labels each sample “up”, “down” or “stationary” according to whether the mid moves by more than a threshold within the horizon. Suppose that, with a short horizon on a large-tick future, 80% of samples are “stationary”, because the mid rarely moves at all. A model that ignores its input and always answers “stationary” is then 80% accurate. A genuinely useful model might be 75% accurate and still be better, because it is right on some of the up and down moves that are the only cases a trader cares about. Accuracy alone cannot show this; balanced accuracy, macro-F1 or the confusion matrix (the table of predicted against actual classes) can.

Example 2.2 (Two “65% accuracy” results). One study reports 65% on FI-2010 (Ntakaris et al., 2018): five Nasdaq Nordic stocks, ten trading days, a horizon of 10 in the data set’s own sampling units, three classes. Another reports 65% on a cryptocurrency pair sampled on a one-second calendar clock with two classes. With two balanced classes, 50% is what guessing achieves; with three classes, guessing achieves anywhere from 33% to the share of the largest class. The two numbers sit on different scales and share almost none of the fields of Table 5. Their equality carries no comparative information.

2.5 A reader’s checklist

The following questions can be asked of any paper in this field, including this one. A study that cannot answer them has not yet shown that its result is useful for trading.

1. What exactly is predicted, over what horizon, on what clock? (Section 8)

Table 5: Fields a study must report before its results can be compared with another’s.

Field	Why it changes the result
Asset, exchange, period, trading hours	Markets differ (Table 4); the same market differs between calm and volatile years and between the open, midday and the close.
Tick size, typical spread, depth	A large tick makes mid changes rarer and queues longer, which changes both what is predictable and what is tradeable.
Message rate	A busy instrument produces many events per second; a horizon of ten events is a few milliseconds in one market and minutes in another.
Data level (L1, L2, L3)	Only L3 data reveal the causes of book changes and queue positions (Section 5).
Sampling clock	An <i>event clock</i> samples after a fixed number of events, a <i>calendar clock</i> after a fixed number of seconds. The event clock over-represents busy periods, which behave differently.
Horizon and target	Predicting the sign of the next mid change is a different problem from predicting a move larger than a threshold, or a return over ten seconds (Section 8).
Class balance	If most samples belong to one class, a model that always predicts that class scores high accuracy while knowing nothing (Example 2.1).
Train/test protocol	A random split lets the model train on data from after its test period; a chronological split does not (Section 30).
Costs, latency and execution model	The same forecast is profitable or unprofitable depending on whether the spread, fees and delays are counted (Section 1.4).
Number of configurations tried	The best of many tries looks good by chance (Section 27).

2. **What does a trivial model score on the same data?** Always guessing the most common class, or a linear model on order-flow imbalance (Section 4).
3. **Was the test period strictly after the training period?** (Section 30)
4. **How many models or settings were tried before this one was reported?** (Section 27)
5. **What decision would the prediction drive, and was that decision evaluated after spread, fees and latency?** (Sections 1.4 and 11)
6. **If the decision uses passive orders, how were fills decided?** A fill rule that ignores the queue overstates results (Section 15).

Part II

Models of the Limit Order Book

About this part. Part II surveys the models of the order book, from the simplest to the most complex. It starts with the empirical facts any model must respect (Section 3); moves through hand-built signals computed from the book (order-flow imbalance, queue imbalance, the micro-price; Section 4); then to models of the queues themselves and the time until they empty (Section 6); to Hawkes processes, which model how events trigger further events (Section 7); to deep networks (Section 8); to hybrids of the two (Section 9); and finally to information from related instruments (Section 10). Every model ends with a model card answering the same four questions.

3 Stylised facts a model should reproduce

Before turning to models, it helps to list what any adequate model of the book must be consistent with. These regularities have been documented repeatedly across markets and periods.

Events cluster

Order-book events arrive in bursts separated by quiet periods, and a large share of events are triggered by earlier ones (Section 7).

Long waits are common

Times between price changes have heavy tails; in a simple model of exactly balanced queues the expected waiting time is even infinite (Cont and de Larrard, 2013).

Order flow has long memory

The signs of successive market orders are positively correlated over very long lags, with a Hurst exponent of about 0.7, mainly because large orders are split into many small ones (Lillo and Farmer, 2004; Lillo et al., 2005).

Prices nevertheless diffuse

Despite persistent order flow, prices are close to unpredictable beyond short horizons, because limit orders absorb the pressure: the market is held near a critical balance between the two (Bouchaud et al., 2004).

Short-horizon mean reversion

Measured volatility is higher over very short windows than over longer ones (bid–ask bounce), and correlations between assets fall when measured over very short windows (the Epps effect) (Bacry et al., 2013).

The average book is humped

Average depth is largest a few ticks away from the best price, not at it (Bouchaud et al., 2002).

Impact is concave

The price impact of a large order executed in pieces grows less than linearly with its size, close to a square-root law (Bouchaud et al., 2018); Hawkes models reproduce its concave, relaxing shape (Bacry and Muzy, 2014).

Order flow explains price changes

Over short windows, price changes are approximately linear in order-flow imbalance, with a slope inversely related to depth (Cont et al., 2014).

Not only trades move prices

Roughly half of price-changing events are limit orders placed inside the spread (Eisler et al., 2012), and the book beyond the best quotes carries part of the price information (Cao et al., 2009).

Activity is seasonal

Intensities and volatility follow intraday patterns, so models fitted with a constant baseline over a non-stationary period overstate how self-exciting the market is (Filimonov and Sornette, 2015).

Generative models are judged by how many of these they reproduce (Section 8.9); predictive models are judged by whether they exploit them after costs.

How each model is presented. Every model in this part ends with a *model card* that answers the same four questions: what has to be calibrated (the numbers estimated from data), how it is calibrated, what the fitted model is then used for, and a picture that makes it easy to remember and to teach. The cards summarise the section they close; the detail and the citations are in the text above each one.

4 Order-flow imbalance and queue imbalance

The simplest useful models of the book are single numbers computed directly from it. This section covers the two most important: *order-flow imbalance*, which measures the net buying pressure from the stream of book events, and *queue imbalance*, which measures how lopsided the two best queues are at an instant. It then refines queue imbalance into better estimates of the “true” price (the weighted mid and the micro-price) and describes features of the book’s shape beyond the best level. These signals are cheap, interpretable and hard to beat, which makes them the baselines for everything that follows.

4.1 Order-flow imbalance

The idea. Why does the price move? Look again at Figure 1. The best ask is 5000.25 with 14 lots waiting. If buyers take all 14 lots, or the sellers waiting there cancel, the best ask becomes 5000.50 and the mid rises. If instead more sellers join at 5000.25, the queue grows and a rise becomes harder. The same holds in mirror image on the bid side. So anything that *grows the bid queue or shrinks the ask queue* pushes the price up, and anything that *shrinks the bid queue or grows the ask queue* pushes it down. Order-flow imbalance (OFI) simply keeps a running score of these pushes (Cont et al., 2014).

Measuring one event. Visible changes at the best prices come from three event types: limit orders arriving, aggressive orders executing against waiting ones, and cancellations. For event number n , define

$$\Delta v_n^b = v_{1,t_n}^b - v_{1,t_n-}^b, \quad \Delta v_n^a = v_{1,t_n}^a - v_{1,t_n-}^a.$$

Recall that $v_{1,t}^b$ is the number of lots waiting at the best bid at time t , and that t_n- means “just before event n ”. So Δv_n^b is *the size of the best-bid queue just after event n minus its size just before*: how many lots event n added to the best-bid queue (positive) or removed from it (negative). Δv_n^a is the same for the best-ask queue. In the book of Figure 1:

- a new 20-lot buy order at 5000.00 gives $\Delta v_n^b = +20$ (29 lots become 49), and $\Delta v_n^a = 0$;
- a sell market order for 10 lots trades against the front of the bid queue and gives $\Delta v_n^b = -10$;
- a seller cancelling 3 lots at 5000.25 gives $\Delta v_n^a = -3$ (14 lots become 11).

Scoring and adding up. While the best prices do not change, the score of event n and the order-flow imbalance over the last w seconds are

$$e_n = \Delta v_n^b - \Delta v_n^a, \quad \text{OFI}_t^{(w)} = \sum_{n: t_n \in (t-w, t]} e_n. \quad (7)$$

In words: e_n is positive when event n grew the bid queue or shrank the ask queue—a push up—and negative for a push down; subtracting Δv_n^a is what turns a shrinking ask queue into a positive score.

The sum runs over every event n whose time t_n falls in the last w seconds, so $\text{OFI}_t^{(w)}$ is the net buying pressure over that window, like the net flow into a tank: lots in on the buy side and out on the sell side count as inflow, the reverse as outflow.

When a best price *does* change, the full definition of Cont et al. (2014) counts whole queues. If the best bid rises to a new, higher price, the entire new queue counts as added buying interest; if the best-bid level is emptied and the best bid falls, the entire old queue counts as removed. The ask side is treated symmetrically. This keeps the score meaningful across price changes.

From pressure to price. The associated short-horizon relation is a straight line,

$$m_t - m_{t-w} = \beta \text{OFI}_t^{(w)} + \varepsilon_t, \quad (8)$$

where β is the price-impact coefficient and ε_t a residual.

In words: over a short window, the mid moves roughly in proportion to the net buying pressure. β converts lots of pressure into price: if β is 0.01 points per lot, a net 100 lots of pressure moves the mid by about one point. ε_t is whatever the line does not explain. Fitting β is an ordinary linear regression of price change on OFI.

Cont et al. (2014) find that β is inversely related to market depth: in a thin book a few lots empty a queue and move the price, while in a deep book the same lots barely dent it. OFI is low-dimensional, cheap to compute and easy to interpret, which makes it the natural baseline against which more complex models should be judged.

Example 4.1 (OFI by hand). Five events at unchanged best prices, all inside one window of length w :

n	event	Δv_n^b	Δv_n^a	e_n
1	bid limit order, 20 lots	+20	0	+20
2	ask cancel, 15 lots	0	-15	+15
3	buy market order hits ask, 10 lots	0	-10	+10
4	ask limit order, 30 lots	0	+30	-30
5	bid cancel, 5 lots	-5	0	-5
$\text{OFI}_t^{(w)}$				+10

Events 2 and 3 enter through identical arithmetic, yet one is a cancellation and the other an aggressive trade. OFI does not distinguish them; Section 5 returns to this.

4.2 Evidence and extensions

How well OFI explains price changes. On 50 US stocks, Cont et al. (2014) find that the linear relation explains mid-price changes over short intervals with an average R^2 of about 65%— R^2 , the coefficient of determination, being the fraction of the variance of price changes that the fitted line explains—against about 32% for the imbalance of *trades* alone; adding trade imbalance to OFI raises the average only to about 67%. The slope is close to inversely proportional to average depth, $\beta \propto 1/\text{depth}$. The fit varies a great deal across stocks—from above 90% for some to below 15% for others—which is itself a warning against quoting a single number for “the” predictability of order flow. The lesson that limit orders and cancellations carry as much price information as trades is one of the most robust in the field.

Looking deeper than the best level. Information also sits beyond the best quotes. On the Australian Stock Exchange, [Cao et al. \(2009\)](#) attribute about 22% of price discovery to the book beyond the best bid and offer, and find that imbalances deeper in the book predict short-term returns even after controlling for the autocorrelation of returns, the inside spread and the imbalance of trades. *Multi-level OFI* computes the imbalance separately at each of the first L levels, giving a vector rather than a number; for six Nasdaq stocks, [Xu et al. \(2018\)](#) find that the out-of-sample fit improves with every additional level included. [Cont et al. \(2023\)](#) combine the level-wise imbalances into a single *integrated OFI*, the first principal component of the multi-level vector, which captures over 89% of its variance. On the 100 largest S&P 500 stocks over 2017–2019 the integrated OFI raises the average out-of-sample R^2 of the contemporaneous price-impact regression from about 65% (best level only) to about 84%.

Order flow in other instruments. The same study asks whether the order flow of *other* stocks helps (cross-impact, Section 10). Two answers are needed. For the *contemporaneous* price change, once integrated OFI is used, adding other stocks’ order flow does not improve the out-of-sample fit. For *forecasting* the next minute’s return, lagged order flow of other stocks does help significantly, although the out-of-sample R^2 of all the forecasting models remains slightly negative and the effect decays within minutes. Explaining the present and predicting the future are, again, different problems.

Horizon and reversal. The predictive content of order imbalance depends on the horizon. In Chinese equities, [Zhang et al. \(2025\)](#) find that order imbalance predicts returns positively from 5 to 30 minutes ahead and negatively from 60 to 120 minutes ahead, with no reversal for large or heavily traded stocks. The usual interpretation is that part of the price pressure from imbalance is temporary and is undone as liquidity is replenished. A signal must be used over the horizon on which it has the sign one assumes.

Estimating the slope in real time. Because β moves with depth, it is re-estimated continuously in practice: by a rolling-window regression, by normalising OFI by recent average depth (a *depth-normalised* OFI), or by treating β_t as a hidden state that drifts over time and tracking it with a Kalman filter ([Kalman, 1960](#)), the standard recursive estimator for linear systems observed with noise.

What OFI is used for. The same quantity serves several decisions. As a directional signal, it predicts the sign of the next short-horizon move. For a market maker, it sets the *skew* of quotes—leaning bids and offers in the direction of pressure—and informs inventory control ([Cartea et al., 2015](#)). For execution, it indicates when resting passively is likely to be rewarded and when the order book is about to move away.

Model card: Order-flow imbalance

What you calibrate. The price-impact slope β for a chosen window w (and, for multi-level OFI, one slope per level or the weights of the integrated OFI).

How. An ordinary least-squares regression of mid changes on $\text{OFI}_t^{(w)}$. Because β falls as depth rises, it is re-estimated on a rolling window, absorbed by dividing OFI by recent depth, or tracked as a drifting hidden state with a Kalman filter. Integrated OFI takes the first principal component of the level-wise OFIs.

What you do with it. A baseline directional signal; the skew of a market maker’s quotes; a warning, for a passive order, that the book is about to move away from it.

How to picture it. Net flow into a tank: lots joining the bid or leaving the ask are inflow, lots joining the ask or leaving the bid are outflow; the level of the tank leads the price.

4.3 Queue imbalance and richer book features

Queue imbalance compresses the top of book into one number,

$$QI_t = \frac{v_{1,t}^b - v_{1,t}^a}{v_{1,t}^b + v_{1,t}^a} \in [-1, 1]. \quad (9)$$

In words: the difference between the best-bid and best-ask queue sizes, divided by their total so the result does not depend on how busy the market is. It is +1 when only buyers are queued, −1 when only sellers are, and 0 when the two queues are equal.

Why imbalance predicts the next move: the thin side breaks first. The mid moves when one of the two best queues is used up. If the book is skewed—many lots on one side, few on the other—the thin queue needs far less trading and cancelling to disappear than the thick one, so it is likely to be emptied first. The next price move is therefore more likely to be *towards the thin side*: a thin ask is likely to be taken out, and the price ticks up; a thin bid is likely to be hit, and the price ticks down. Think of two piles of sand being removed at roughly the same rate: the smaller pile is gone first. Stochastic models of the book make this precise, computing the probability of an up move as a function of the two queue sizes (Cont et al., 2010).

Example 4.2 (Reading the imbalance in Figure 1). The best bid has 29 lots and the best ask 14, so $QI_t = (29 - 14)/(29 + 14) \approx 0.35$: the book leans towards the buyers. The ask queue is the thinner one and is likely to break first, so the next change in the mid is more likely to be up than down. For a trader the skew has two consequences. A new passive buy order would join the back of the long 29-lot queue and probably wait while the price moves away from it. A new passive sell order would join the short ask queue and probably be filled—just before the up move that the imbalance predicted, which is adverse selection (Section 13).

A price that leans with the book. The same intuition gives a better single-number price than the mid. The *weighted mid* weights each side’s price by the size on the *other* side,

$$m_t^{\text{wmid}} = \frac{v_{1,t}^b p_{1,t}^a + v_{1,t}^a p_{1,t}^b}{v_{1,t}^b + v_{1,t}^a}, \quad (10)$$

In words: when the bid queue is much larger than the ask queue, the price is pulled towards the ask, the side expected to break, and vice versa. With equal queues it equals the mid. In Figure 1 it is $5000.00 + 0.25 \times 29/43 \approx 5000.169$, above the mid of 5000.125.

Model card: Queue imbalance and weighted mid

What you calibrate. Nothing for QI_t or m_t^{wmid} themselves. To turn QI_t into a probability, the coefficients of a logistic regression of the next mid-move direction on QI_t (Gould and Bonart, 2016).

How. Count, over historical data, which way the mid moved next after each observed value of the imbalance, and fit the logistic curve by maximum likelihood; report it separately for large-tick and

small-tick instruments, where its value differs.

What you do with it. Predict the direction of the next mid change; decide which side a passive order should rest on and when it is likely to be adversely selected.

How to picture it. Two piles of sand removed at about the same rate: the smaller pile is gone first, and the price moves towards the side whose pile disappeared.

4.4 The micro-price

The weighted mid has three defects (Stoikov, 2018). It is not a *martingale*—its expected future value is not its current value, so it is a biased estimate of where the price is going. It is noisy, especially when the spread widens. And it can move the wrong way: if the last lot on a thin ask is cancelled, the ask moves up a tick, yet the weighted mid can fall. The *micro-price* of Stoikov (2018) replaces the ad hoc weighting by an estimate of the expected future mid itself.

Definition. Let $\eta_1 < \eta_2 < \dots$ be the successive times after t at which the mid changes, let $\iota_t = v_{1,t}^b / (v_{1,t}^b + v_{1,t}^a) = (1 + \text{QI}_t)/2 \in [0, 1]$ be the imbalance rescaled to $[0, 1]$, and let s_t be the spread. The micro-price is the limit of the expected mid after more and more price changes,

$$m_t^{\text{micro}} = \lim_{k \rightarrow \infty} \mathbb{E}[m_{\eta_k} \mid \iota_t, s_t]. \quad (11)$$

In words: start from the current imbalance and spread, and ask where the mid will be after the next price change, after the one after that, and so on. The answers settle down to a value; that value is the micro-price. By construction it is the best estimate, given the top of the book, of where the price is heading.

Two assumptions make it computable: the pair (ι_t, s_t) evolves as a Markov chain, and the dynamics do not depend on the price level itself. Then

$$m_t^{\text{micro}} = m_t + \sum_{k=1}^{\infty} g^{(k)}(\iota_t, s_t), \quad (12)$$

$$g^{(1)}(\iota, s) = \mathbb{E}[m_{\eta_1} - m_t \mid \iota, s], \quad g^{(k+1)}(\iota, s) = \mathbb{E}[g^{(k)}(\iota_{\eta_1}, s_{\eta_1}) \mid \iota, s].$$

In words: the first term is the expected size of the next price change; each further term is the expected next change, averaged over the states the book can be in after the previous one. The micro-price is the mid plus the sum of all these expected adjustments.

Estimation. In practice the imbalance is split into n_ι buckets (for example by quantiles) and the spread into n_s values, so the bucketed state (ι, s) takes $n_\iota n_s$ values. From data one counts three kinds of transition from each state: to another state with the mid unchanged, collected in a matrix $\mathbf{U}$; to each possible size of mid change, in a matrix $\mathbf{J}^{(1)}$; and to another state with the mid changed, in a matrix $\mathbf{J}^{(2)}$. With Δ the vector of possible mid changes and $\mathbf{B} = (I - \mathbf{U})^{-1} \mathbf{J}^{(2)}$,

$$g^{(1)} = (I - \mathbf{U})^{-1} \mathbf{J}^{(1)} \Delta, \quad g^{(k+1)} = \mathbf{B} g^{(k)}.$$

The factor $(I - \mathbf{U})^{-1}$ sums over all the steps in which the book changes without the mid moving—the same geometric-series device used for the Hawkes model in Section 7. The data are symmetrised before estimation (each observation is added again with bid and ask swapped and the price change negated), which guarantees that the series converges. Stoikov reports that a handful of terms suffices in practice, and that the micro-price predicts short-term price changes better than either the mid or the weighted mid, for both a large-tick and a small-tick stock.

Model card: Micro-price

What you calibrate. The transition matrices $\mathbf{U}$, $\mathbf{J}^{(1)}$, $\mathbf{J}^{(2)}$ between imbalance–spread buckets, after choosing the numbers of buckets n_i and n_s .

How. Bucket the imbalance and spread, symmetrise the data, and count transitions; the adjustments $g^{(k)}$ then follow by matrix algebra, and a handful of terms suffices.

What you do with it. A fair-value estimate to quote around in place of the mid, and a better reference than the mid or weighted mid for predicting short-term price changes.

How to picture it. A weather forecast that starts from today’s state and averages over every path the next few changes can take: the micro-price is where those paths land on average.

4.5 The shape of the book

Beyond the best level, several features summarise how liquidity is distributed:

Multi-level imbalance

the queue imbalance computed with sizes summed over the best ℓ levels on each side, for several ℓ .

Book slope

how fast cumulative depth grows with distance from the best price, usually the fitted slope of cumulative size against price distance, separately for each side. A steep slope means a deep book that absorbs large orders.

Curvature

the change of that slope with distance—whether depth is concentrated near the touch or further out. Its empirical relevance is tied to the average shape of the book, which in equities is humped: depth is usually largest a few ticks away from the best price, not at it (Bouchaud et al., 2002).

Depth-weighted and volume-weighted prices

averages of the prices at the first ℓ levels, weighted by their sizes; the weighted mid is the one-level case.

Resilience

how quickly the book refills after a large trade. Large (2007) measures it with a ten-variate Hawkes model on an LSE stock and finds that when the book does refill, it does so the impulse responses have half-lives under 20 seconds, but a resilient response follows a large trade in under 40% of cases.

Depth-normalised quantities

order-flow measures divided by recent average depth, motivated by the inverse relation between price impact and depth.

Limits of the intuition. Displayed size is not committed size. A thin queue may be refilled immediately by new orders; hidden or iceberg orders may make the thin side thicker than it looks; and orders placed only to be cancelled (spoofing, Section 35) can create a skew that is not real. Imbalance is therefore a strong but noisy signal.

Evidence. Gould and Bonart (2016) fit a logistic regression of the direction of the next mid move on queue imbalance for ten Nasdaq stocks. The relationship is strongly significant for every stock,

but its value depends on the tick size: out of sample, queue imbalance improves binary classification over a null model by about 50–60% for large-tick stocks and only about 10–30% for small-tick stocks, and probabilistic classification by about 20–30% against about 2–6%. Queue imbalance is most informative exactly where queues are long and the price rarely moves—the large-tick regime that ES and ZN futures also belong to.

Example 4.3 (Same imbalance, different history). Bid 300, ask 100 gives $QI_t = (300 - 100)/(300 + 100) = 0.5$. In book A the ask fell from 400 to 100 because a 300-lot market buy lifted it. In book B it fell because 300 lots were cancelled. Both books have the same snapshot and the same QI_t . A trader who saw the messages knows that A contains an aggressive buyer and B does not.

Model card: Book shape

What you calibrate. The number of levels ℓ to aggregate over; the slope and curvature of cumulative depth against distance; for resilience, the impulse response of a Hawkes model.

How. Slope and curvature are least-squares fits of cumulative size against price distance, per side and per snapshot. Resilience is read off a fitted multivariate Hawkes model (Section 7), as in [Large \(2007\)](#).

What you do with it. Estimate how much a large order will move the price, normalise other signals by depth, and detect a book that is thinning out before the touch does.

How to picture it. The profile of a hillside on each side of a valley: a steep hillside holds a lot of liquidity close to the price, a shallow one makes large orders walk far.

The central empirical question is not whether richer representations improve in-sample fit, but whether the improvement survives out-of-sample evaluation and execution costs.

5 Events rather than snapshots

The same reduction in displayed volume has different information content depending on its cause. A snapshot representation can make two states identical even when their histories differ (Example 4.3). Market-by-order (L3) data carry the cause: every add, modify, cancel and execution, with an order identifier. On CME’s MDP 3.0 feed, for instance, book updates for individual orders and trade summaries arrive as separate message templates, so the two books in Example 4.3 produce visibly different message sequences.

Treating the three event types jointly is not optional. Price changes are often attributed to market orders, but in Nasdaq data roughly half of all price-changing events are limit orders placed *inside* the spread, which improve the best price rather than consume it ([Eisler et al., 2012](#)). A model that sees only trades misses half of what moves the price.

The case for message-level representation has recently been made directly in deep learning. LOBERT ([Linna et al., 2025](#)) adapts the BERT encoder to LOB data by treating each complete multi-dimensional message as a single token, while keeping continuous representations of price, volume and time. It is designed as a general-purpose, fine-tunable foundation model, and reports leading performance on mid-price movement and next-message prediction with a shorter required context than earlier methods; it is trained and evaluated on Nasdaq ITCH message data for four large US stocks in 2015, with millisecond timestamps. A message-level encoder sees the difference between a cancellation and an execution in Example 4.3; a snapshot encoder, by construction, does not.

6 Queue depletion and first-passage models

The idea: a queue is a bucket with a leak. Think of the best-ask queue as a bucket of lots. New sell limit orders pour lots in; buy market orders that trade against it, and sellers who cancel, drain lots out. Each event adds or removes a random amount at a random time, so the level of the bucket wanders up and down like a random walk. The question this section asks is: *how long until the bucket is empty?* That moment matters for three reasons.

1. When the best-ask queue empties, the best ask moves up a tick, and so does the mid. Price changes *are* queue depletions.
2. The direction of the next price move is a race between the two best queues: whichever empties first decides it. This is the formal version of “the thin side breaks first” (Section 4).
3. For a resting order, the wait until it is filled is the time until the volume *ahead of it* is used up—the same question, asked about part of a queue (Section 12).

The definition. The depletion time of the best ask is the first-passage time

$$\tau^a = \inf\{u > 0 : v_{1,t+u}^a \leq 0\}, \quad (13)$$

where u runs over time after t ; τ^b is defined in the same way for the bid.

In words: τ^a is how long from now until the best-ask queue is empty for the first time—the time at which a loop that polls the queue size would first see zero. “First passage” is the general name for the first time a wandering quantity reaches a level.

The race between the two queues is then

$$\mathbb{P}(\text{next mid move is up} \mid X_t) = \mathbb{P}(\tau^a < \tau^b \mid X_t), \quad (14)$$

In words: the price goes up next exactly when the ask queue empties before the bid queue does.

A first estimate. If lots drain out at an average rate of r_{out} lots per second and pour in at r_{in} , the bucket loses $r_{\text{out}} - r_{\text{in}}$ lots per second on average, and a best-ask queue of $v_{1,t}^a$ lots takes on average

$$\mathbb{E}[\tau^a] \approx \frac{v}{r_{\text{out}} - r_{\text{in}}} \quad (r_{\text{out}} > r_{\text{in}})$$

seconds to empty: size divided by net drain rate, as for any bucket. Two warnings come with this formula. First, it is an average; the actual time is random and often far from it. Second, if inflow keeps up with outflow ($r_{\text{in}} \geq r_{\text{out}}$), the bucket has no net leak and the expected time can be very long or infinite.

Models. Stochastic models of the book (Cont et al., 2010) treat arrivals, cancellations and executions as random event streams (point processes) and derive the distribution of first-passage times and the probability of a mid move in either direction, including the race above. Queue-reactive models (Huang et al., 2015) go one step further and let the rates themselves depend on the current queue sizes—for example, orders may join or cancel at different rates when a queue is short than when it is long. That reproduces many observed regularities of large-tick assets.

6.1 The Cont–Stoikov–Talreja model

The model of Cont et al. (2010) is the foundation on which most later queue-based models of the order book are built, and it repays careful study. It was one of the first models of the whole book to satisfy three requirements at once: its parameters can be estimated directly from order-book data with simple counting; it reproduces important empirical features of the book without being tuned to do so; and it is simple enough that the probabilities a trader actually needs—will the price go up next, will my order be filled before the price moves—can be *computed*, quickly and exactly, rather than estimated by simulation. Its central idea is that a limit order book is a queueing system: limit orders wait in line to be served by market orders, or leave the line by cancelling.

6.1.1 The model

State. Prices live on a grid of ticks. The state of the book is the number of orders resting at every price on the grid, counted in units of the average limit-order size. Because most activity happens near the current price, the model measures position relative to the quotes: let $Q_{\bar{\ell}}^{\text{bid}}(t)$ be the number of buy orders $\bar{\ell}$ ticks below the best ask, and $Q_{\bar{\ell}}^{\text{ask}}(t)$ the number of sell orders $\bar{\ell}$ ticks above the best bid. Note the convention: distance is measured from the *opposite* best quote. With a one-tick spread, the best bid is at distance $\bar{\ell} = 1$ from the best ask.

Events and their rates. Six kinds of event change the book, and each is modelled as an independent Poisson stream (Section 7 explains what that assumption means and where it fails):

- a **limit buy** order arrives $\bar{\ell}$ ticks below the best ask, and a **limit sell** order $\bar{\ell}$ ticks above the best bid, each at rate $r_L(\bar{\ell})$;
- a **market buy** order removes one unit from the best ask, and a **market sell** order one unit from the best bid, each at rate r_M ;
- each resting order $\bar{\ell}$ ticks from the opposite quote is **cancelled** independently at rate $r_c(\bar{\ell})$, so a level holding Q orders loses orders to cancellation at total rate $r_c(\bar{\ell}) Q$.

Two modelling choices carry the economics. First, limit orders arrive more often near the current price; following earlier empirical work (Bouchaud et al., 2002), the arrival rate is a power law in distance,

$$r_L(\bar{\ell}) = \frac{r_0}{\bar{\ell}^\Xi}, \quad (15)$$

with a scale r_0 and exponent Ξ fitted to data.

In words: the farther a price is from where trading happens, the fewer orders are placed there, and the fall-off follows a power law: doubling the distance divides the rate by 2^Ξ .

Second, cancellations are proportional to queue size: each order has the same chance per second of being cancelled, so a long queue sheds orders faster than a short one. This single assumption is what keeps queues from growing without bound.

Under these assumptions the book is a continuous-time Markov chain: its future depends only on the current queue sizes, not on how they were reached. The authors show that a book that starts with the bid below the ask always keeps that property, and that if every cancellation rate is positive the chain is *ergodic*—it has a unique long-run distribution, so averages over a long simulated history converge to well-defined values, such as the average shape of the book.

6.1.2 Estimating the model

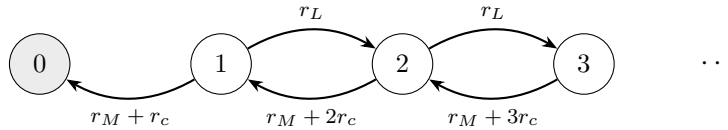
The estimation is deliberately elementary, which is part of the model’s appeal. With the unit of size set to the average limit order:

- the limit-order rate at each distance is the number of times the queue at that distance *increased*, divided by the trading time, $\hat{r}_L(\bar{\ell}) = N_L(\bar{\ell})/T_{\text{obs}}$, and a power law is fitted to the first five distances by least squares to extrapolate further out;
- the market-order rate is the number of market orders that changed the best quotes, scaled by the ratio of average market-order size to average limit-order size, divided by the trading time;
- the per-order cancellation rate at each distance is the number of decreases not caused by trades, scaled by size, divided by the trading time *and by the average queue size at that distance*—because the model makes total cancellations proportional to queue size.

Here $N_L(\bar{\ell})$ is the count of queue increases at distance $\bar{\ell}$ and T_{obs} the total observed trading time. The authors use five levels of quotes and trades for Tokyo Stock Exchange stocks over 125 trading days (August to December 2006) and report estimates for Sky Perfect Communications (Table 7).

6.1.3 The key simplification: two independent birth–death queues

The full book is a large Markov chain, but the questions a trader asks concern only the next price change, and until the price changes the quotes do not move. During that time the best-bid queue and the best-ask queue each evolve as a *birth–death process*: a counter that goes up by one at a constant “birth” rate and down by one at a “death” rate that depends on its current value. If the spread is s_t/δ ticks, the best queues sit at distance $\bar{\ell} = s_t/\delta$ from the opposite quote, and a queue holding x orders evolves as



with birth rate $r_L(s_t/\delta)$ and death rate $r_M + x r_c(s_t/\delta)$; reaching state 0 (shaded) means the queue has been used up and the price moves, so the count stops there. The authors prove that, up to the first price change, the two best queues can be replaced by *independent* birth–death processes with these rates. Every question about the next price move becomes a question about which of two independent counters hits zero first—the race of Section 6, now with exact rates.

Why Laplace transforms. The time for a birth–death process starting at x to reach zero is a sum of x independent pieces: the time to go from x to $x - 1$, then from $x - 1$ to $x - 2$, and so on. Distributions of sums are awkward, but the *Laplace transform* of a waiting time τ , $\mathbb{E}[e^{-\varpi\tau}]$ as a function of a variable ϖ , turns sums of independent waiting times into products of their transforms. For birth–death processes each one-step transform has a known closed form as a continued fraction (Cont et al., 2010), so the transform of the whole first-passage time is a product of known pieces. Probabilities such as “queue A empties before queue B” are then obtained by numerically inverting a transform built from these products—a computation that takes far less time than simulating thousands of paths of the book, and gives full distributions rather than only averages.

In words: instead of simulating the book many times and counting how often the price went up, the model writes the answer as a formula in transform space and evaluates it directly. The transform is a book-keeping device that makes adding independent waiting times as easy as multiplying numbers.

6.1.4 Three probabilities a trader needs

The paper computes three conditional probabilities, each given the current number of orders at the best bid, $v_{1,t}^b$, and at the best ask, $v_{1,t}^a$, and the spread.

1. Will the next price move be up? With a one-tick spread, the mid rises next if the ask queue empties before the bid queue. With a wider spread, a new order can also arrive inside the spread and move the mid first; the formula adds these arrivals as competing exponential clocks. Table 8 compares the model’s answer with the empirical frequency in the data. The model captures the main pattern well—the probability rises with the bid queue and falls with the ask queue—though it understates how strongly a large ask queue pushes the price down, and some cells differ noticeably (for example 0.548 against an empirical 0.714 with four orders at the ask and five at the bid).

2. Will my order at the bid be filled before the price moves? Suppose an order is placed at the back of the best bid, so that the bid queue holds $v_{1,t}^b$ orders including our own, and that it is never cancelled. It is filled when all orders ahead of it have been removed—by market sells, or by cancellation of the orders ahead—which is a pure-death process: with x orders in line including our own, the line shortens at rate $r_M + (x - 1)r_c$, since our own order is not cancelled. The order is filled before the price moves if this line empties before the ask queue does. Table 9 gives the model’s answers. The pattern is the execution problem of Part III in numbers: an order alone at the bid, facing five orders at the ask, is filled before the price moves about 78% of the time; the last of five orders at the bid, facing one at the ask, about 12%. The authors note that their data do not identify individual orders, so these probabilities could not be checked against data—exactly the gap that order-level data and queue-exact replay fill (Part IV).

3. Will I “make the spread”? A market maker places one order at the best bid and one at the best ask and earns the spread if both are filled before the price moves. The model computes this probability too. Its most interesting feature is that it is *not* monotone: for a fixed number of orders at the ask, the probability first rises and then falls as the bid queue grows. With one order at the ask, it is about 0.27 with one order at the bid, peaks near 0.31 with two or three, and falls back to about 0.29 with five. A longer bid queue protects the bid from being exhausted, which helps, but also makes the bid order wait longer, which hurts; the balance gives an interior optimum.

6.1.5 What the model reproduces

Without any tuning for these purposes, the fitted model reproduces several features of the real book:

- the **hump-shaped average book**: simulated average depth peaks two ticks from the opposite quote, as observed empirically (Bouchaud et al., 2002);
- the **order of magnitude of volatility**: over the first day of the sample the realised volatility was 0.0219 after 370 trades, against 0.0228 ± 0.0003 in simulations of the same number of trades—computed from order-flow rates alone;
- the **one-step behaviour of queues**: the probability that a queue grows at its next change, as a function of its size and distance, is close to the empirical frequencies at all five distances.

6.1.6 A trading test, and what it shows

The authors build a simple strategy on the probability of an up move. When the spread is one tick, there are at least three orders at the best bid, one at the best ask and at most one at the second ask, buy with a market order. If one order remained at the second ask, the book now has a two-tick spread, three or more orders at the bid and one at the ask, and the model gives a 0.62 probability that the next move is up. Exit with a market sell when the bid rises above the entry price, or when the bid queue falls to one order (a one-tick loss). In about 30 days of *simulated* trading from the fitted model, the strategy made 2,376 round trips with an average profit of 0.068 ticks per trade, although fewer than half of the trades were winners—and without transaction costs.

The test illustrates both the promise and the limits discussed throughout this survey. The decision is derived from a conditional probability of the book, which is exactly what an execution policy needs. But it is evaluated inside the model that generated the probability, not on recorded data, and without costs; a profit of 0.068 ticks per trade must also survive the half spread paid on entry and exit, the fees, and the latency between seeing the book and the market order arriving. These are the steps of the chain in Equation (3) that the model does not cover.

6.1.7 Limitations and descendants

The authors are explicit about what the model leaves out. Order arrivals are independent Poisson streams, so there is no clustering of events and none of the long memory of order flow (Section 3); orders all have the same size; rates do not change through the day or with market conditions; and there is no strategic behaviour. Each limitation defines a line of later work:

- Cont and de Larrard (2013, 2012) reduce the book to the two best queues and obtain closed forms and a diffusion limit (Section 6.2);
- queue-reactive models let all rates depend on the current queue sizes (Huang et al., 2015), and their neural extensions learn that dependence (Bodor and Carlier, 2025);
- Hawkes models replace the Poisson streams by self-exciting ones (Section 7);
- the micro-price (Stoikov, 2018) keeps the Markov-chain logic but estimates the transitions directly from data rather than from a parametric model;
- queue-position valuation (Moallemi and Yuan, 2017) and fill-probability models (Arroyo et al., 2024; Zhong et al., 2025) extend the second probability above to individual orders, with adverse selection.

Finally, because the Laplace-transform method yields full distributions rather than single probabilities, it can answer latency questions directly—for example, the probability that the ask queue empties more than a given delay after the bid queue, which is what matters when an order takes time to reach the exchange (Cont et al., 2010).

Model card: Cont–Stoikov–Talreja

What you calibrate. The limit-order rates $r_L(\bar{\ell})$, summarised by r_0 and Ξ ; the market-order rate r_M ; the per-order cancellation rates $r_c(\bar{\ell})$; the unit of size (the average limit order).

How. Counting: increases, decreases not caused by trades, and market orders at each distance, divided by trading time (and, for cancellations, by average queue size); a least-squares power-law fit for r_L .

What you do with it. Compute, from the current two best queues, the probability that the next move is up, that a resting order fills before the price moves, and that a market maker earns the spread—fast and exactly, via Laplace transforms.

How to picture it. Two counters racing to zero, each ticking up when an order joins and down when one is traded or cancelled; the first to hit zero decides the next price move.

6.2 The Cont–de Larrard model

Cont and de Larrard (2013) give the cleanest closed-form treatment of the race between the two best queues, and it is worth stating because every term has a direct trading meaning. The model assumes:

- the spread is always one tick (true over 98% of the time for the large-tick US stocks they study);
- at each best queue, limit orders arrive at rate r_L , market orders at rate r_M and cancellations at rate r_C , all as independent Poisson streams of unit-size orders, so each queue is a random walk that moves up at rate r_L and down at rate $r_M + r_C$ (in the notation of the bucket picture, $r_{\text{in}} = r_L$ and $r_{\text{out}} = r_M + r_C$);
- when a queue empties, the price moves one tick and both new best queues are drawn afresh from a distribution estimated from the data.

Empirically the rates nearly balance: r_L is slightly below $r_M + r_C$ (for example a ratio of about 0.95 for Citigroup in their data). Three results follow.

Time to the next price change. The waiting time has a closed-form distribution in terms of Bessel functions. Its tail is the important part: when inflow is slightly smaller than outflow, the probability that no price change has occurred after u seconds decays like $1/u^2$; when they balance exactly, it decays like $1/u$, so the *expected* waiting time is infinite. Long quiet periods punctuated by bursts are thus a direct consequence of (nearly) balanced queues, before any self-excitation is added.

Probability that the next move is up. In the balanced case the probability depends only on the two queue sizes, not on the rates—it is the probability that a symmetric two-dimensional random walk started at the queue sizes hits the ask axis first. In the companion heavy-traffic analysis (Cont and de Larrard, 2012), with uncorrelated queue changes of equal size and intensity on the two sides, it takes the simple form

$$\mathbb{P}(\text{next mid move is up} \mid v_{1,t}^b, v_{1,t}^a) = \frac{2}{\pi} \arctan\left(\frac{v_{1,t}^b}{v_{1,t}^a}\right). \quad (16)$$

In words: the probability rises from 0 to 1 as the bid queue grows relative to the ask queue, and equals 1/2 when they are equal. It is the “thin side breaks first” rule of Section 4, made exact for this model. In Figure 1, with 29 lots bid and 14 offered, it gives $(2/\pi) \arctan(29/14) \approx 0.71$.

(The general formula in the paper allows for correlated queue changes; its printed zero-correlation special case has the two queues interchanged, and the form above is the one implied by the paper’s general expression.)

From queues to volatility. Over long horizons the price behaves like a random walk whose volatility, in the balanced case, is proportional to $\delta\sqrt{r_L/\Theta}$, where Θ is the average product of the bid and ask queue sizes just after a price change. Volatility rises with order activity and falls with the depth that reappears after each move—a microstructural explanation of why thin, busy markets are volatile. Across Dow Jones stocks the authors find ten-minute volatility roughly proportional to this quantity.

Example 6.1 (Why the rates must depend on the state). A best-ask queue holds 200 lots. Averaged over the day, net depletion (executions plus cancellations minus additions) runs at 40 lots per second, so the first estimate gives $200/40 = 5$ seconds. Now condition the rate on OFI. When buying pressure is in its top decile, the net drain is 90 lots per second and $\mathbb{E}[\tau^a] \approx 200/90 \approx 2.2$ s; in the bottom decile it is 15 lots per second and $\mathbb{E}[\tau^a] \approx 13$ s. The same queue has expected lifetimes six times apart, depending on what the rest of the market is doing. A model that uses one daily average rate would be wrong in both directions.

Depletion and adverse selection. Queue depletion is also where adverse selection comes from. Consider a resting buy order at the back of the best-bid queue. It is filled only when everything ahead of it has been traded or cancelled. Because it is last in line, in practice it is filled mostly by a burst of selling large enough to empty the whole level. And an emptied bid level is, by point 1 above, a downward price move. The order is therefore filled at the very moment the price is moving against it: the buyer acquires the lots just as they become cheaper. The fill and the bad price move are not two unlucky events that happen to coincide; they are the same event.

An order near the *front* of the queue is in a different position. It can be filled by a small sell order that takes a few lots and leaves the rest of the queue, and the price, where they were. Such fills carry little information, so front-of-queue orders are less adversely selected. This is one source of the front-back value difference in Moallemi and Yuan (2017), whose model splits the value of an order into a static component—the spread earned minus adverse selection, which increases with queue position—and a dynamic component, the option value of holding the queue position.

Example 6.2 (Filled by the move). In Figure 1 we join the best bid at 5000.00 with one lot, behind 29 lots. Suppose a 30-lot sell market order arrives. It takes all 29 lots ahead of us and our lot, and the 5000.00 level is gone: the best bid is now 4999.75 and the mid has fallen from 5000.125 to 5000.00 or below. We bought at 5000.00 and the market immediately values the contract lower. Had we been first in the queue, a 2-lot sell would have filled us and left 28 lots and the price untouched.

Model card: Cont-de Larrard

What you calibrate. Three rates per best queue, r_L , r_M , r_C , and the distribution of the new best-queue sizes after a price change (which gives Θ).

How. Counting events at the best quotes over a sample, and the empirical distribution of queue sizes just after each price change.

What you do with it. The probability that the next move is up, the distribution of time to the next price change, and a volatility estimate $\propto \delta\sqrt{r_L/\Theta}$ from order-flow rates alone.

How to picture it. A drunkard's walk in two dimensions, starting at the point (bid size, ask size): whichever axis the walk hits first decides the price move.

6.3 The queue-reactive model

The idea. Cont–Stoikov–Talreja make cancellations proportional to queue size and keep the other rates fixed. Huang et al. (2015) ask the data instead: *how do traders actually behave when a queue is short, and when it is long?* Their answer is the queue-reactive model, in which the rate of every kind of event at a queue is an arbitrary function of the current size of that queue (and, in richer versions, of the neighbouring queues), estimated without assuming a functional form.

State and rates. The book is described by the queues at the first few price levels on each side of a reference price p_{ref} (the mid when the spread is an odd number of ticks; three levels in the paper,

because levels four and five behave like level three). Queue sizes are counted in units of the average size of events at that level, so a queue of size x holds about x typical orders. In the basic version each queue is a birth–death process like those of Section 6.1, but with size-dependent rates: limit orders arrive at rate $r_L(x)$, and cancellations and market orders remove units at rates $r_C(x)$ and $r_M(x)$, where x is the current queue size.

In words: the model writes down a separate arrival, cancellation and trading rate for each possible queue size, rather than one rate for all sizes. The data decide how the rates change with size.

Calibration by counting. For every event at a queue, record the time since the previous event at that queue, the event type, and the queue size just before the event. The estimator of the paper is

$$\hat{r}_L(x) = \frac{\#\{\text{limit orders that arrived while the queue held } x\}}{\text{total time the queue spent at size } x},$$

and likewise for cancellations and market orders. (The paper writes it as the inverse of the mean waiting time at size x , times the fraction of events at size x that were limit orders; the two forms are algebraically equal.) Bid and ask are pooled by symmetry, and recording restarts whenever p_{ref} moves.

In words: to learn how fast orders arrive when a queue holds x units, count the arrivals that happened while it held x units and divide by how long it held x units in total.

What the data show. On France Telecom and Alcatel-Lucent (Euronext Paris, 2010–2012, five levels), the rates are far from those of the simpler models. At the best level, the limit-order rate is roughly constant in queue size (lower when the queue is empty); the cancellation rate increases concavely up to about 25 units and then flattens, rather than growing linearly as Cont–Stoikov–Talreja assume; and the market-order rate *decreases* exponentially with the available volume—traders hit small queues. The long-run distribution of queue sizes implied by the fitted rates matches the empirical one closely, and better than a Poisson model with linear cancellation. Richer versions let the best-level rates depend on the opposite queue: insertion falls and market orders rise as the opposite queue grows.

Prices. The full queue-reactive model adds a price. When a best queue empties, the reference price moves one tick with probability π_{move} ; and with probability π_{reinit} the whole book is redrawn from its long-run distribution around the new price, which the authors interpret as the share of price changes driven by outside information. ($\pi_{\text{reinit}} = 1$ recovers the Cont–de Larrard assumption of Section 6.2.) The two probabilities are calibrated to the ten-minute volatility and to the ratio of price continuations to reversals; for France Telecom $\pi_{\text{move}} = 0.7$ and $\pi_{\text{reinit}} = 0.85$. Order-book dynamics alone, with no outside information, generate at most about 5 basis points of volatility against 14 observed, and far too much mean reversion; the authors use this gap to motivate the re-initialisation step, which stands for information arriving from outside the book.

Uses and descendants. The model is a simulator of large-tick books that reacts to the orders a trader adds: the paper uses it to compute execution probabilities and to compare placement tactics (“fire and forget” against pegging) for a large order. Its neural extension learns the dependence of the rates on the book with networks (Bodor and Carlier, 2025), and queue-reactive Hawkes models add self-excitation to the size-dependent rates (Section 7.11).

Model card: Queue-reactive model

What you calibrate. The rate functions $r_L(x)$, $r_C(x)$, $r_M(x)$ for each of the first few levels (and, in richer versions, their dependence on neighbouring queues); the price-move probabilities π_{move} and π_{reinit} .

How. Count events by queue size and divide by the time spent at that size; calibrate π_{move} and π_{reinit} to the observed volatility and to the continuation–reversal ratio.

What you do with it. Simulate a large-tick book that responds to one’s own orders; compute fill probabilities; compare placement tactics for a large order.

How to picture it. A toll booth whose customers and attendant react to the line: when it is long, more drivers give up and leave; when it is short, cars are served faster.

6.4 Decision rules from first-passage times

Depletion times translate directly into execution decisions.

Limit or market

If the expected time for a passive order to reach the front and be filled is longer than the horizon over which the trader’s signal is valid, waiting forfeits the signal; crossing the spread is then the better choice despite its cost.

When to cancel

If the estimated depletion time of the side on which an order rests collapses, or the race of Section 6.2 turns against it, the order is about to be filled just before an adverse move, and should be cancelled or repriced.

What a queue position is worth

Moallemi and Yuan (2017) split the value of a resting order into a *static* part—the spread earned at execution minus the adverse-selection cost, which grows with queue position—and a *dynamic* part—the option value of holding a place in the queue that advances over time. They find that the front–back difference can be comparable to the bid–ask spread for some large-tick US stocks (for example BAC and CSCO), and smaller for others.

Fill probability as survival

The time to fill is a waiting time that, for orders cancelled first, is never observed; survival models (Cox, 1972), which handle such *censored* observations, are the natural statistical tool (Section 8.8).

Queue-depletion models thus serve execution in two ways: they estimate *when* a resting order will be filled, and, because a depletion is a price move, they indicate *how informative* the fill is likely to be. Section 13 develops the second point. Depletion and price prediction are related but distinct: a depletion model answers *when* a level empties, and says which way the price goes next only when it is used for the race between the two sides.

7 Hawkes processes

Works in calm, fails in stress. The queue models of Section 6 assume that orders arrive as independent streams at rates fixed over the sample. In a calm book that is a reasonable approximation: the Cont–Stoikov–Talreja up-move probabilities of Table 8 track the empirical frequencies closely. It breaks down exactly when it matters most. After a large trade or a news release, rates jump to several times their average for a few seconds, and a model that uses the day’s

average rate underestimates how fast queues empty (Example 6.1). The missing ingredient is that events cause events. This section introduces the standard model of that mechanism.

7.1 Why not a Poisson process?

The simplest model of events arriving at random times is the *Poisson process* (Daley and Vere-Jones, 2003). Divide time into tiny slices and, independently in each slice, let an event happen with a small fixed probability; in the limit, events arrive at a constant average rate μ per second. The number $N(u)$ of events between time 0 and time u —and likewise the number in any other interval of length u —has the distribution

$$\mathbb{P}(N(u) = k) = e^{-\mu u} \frac{(\mu u)^k}{k!}, \quad k = 0, 1, 2, \dots \quad (17)$$

In words: the chance of seeing exactly k events in u seconds depends only on the average rate and the length of the interval—not on when the interval starts, and not on anything that happened before it. The process has no memory: an event now makes the next one neither more nor less likely. The waiting times between events are independent and exponentially distributed, and the count in an interval has variance equal to its mean.

For order-book events these properties fail, in three ways that can be seen in any day of data.

1. **Events cluster.** Activity comes in bursts. A large trade is followed by a flurry of new orders, cancellations and further trades as other participants react; a news release does the same on a larger scale. Between bursts the market can be quiet for long stretches. In a Poisson process an event says nothing about the next one, so bursts can only arise by chance, and large ones are vanishingly rare.
2. **Counts vary far more than Poisson allows.** Because of clustering, the number of events per second has a variance much larger than its mean, whereas a Poisson count has variance equal to its mean (Bacry et al., 2015).
3. **Event types trigger each other.** A buy market order makes further buy market orders more likely, because large orders are split into many small ones and because other traders follow; a trade that empties a level triggers cancellations and new limit orders around it. A set of independent Poisson processes, one per event type, cannot represent any of this.

A fourth, milder departure is that average activity changes predictably through the day, typically highest near the open and the close. A Poisson process with a time-varying rate (an *inhomogeneous* Poisson process) can absorb that pattern, but it still cannot make events cause other events, which is the defining feature of order flow.

Example 7.1 (How rare is a burst under Poisson?). Suppose events arrive at an average rate of $\mu = 10$ per second. Under a Poisson model the probability of 30 or more events in a single second is about 2.5×10^{-7} . Over a 6.5-hour trading day of 23 400 seconds, the expected number of such seconds is about 0.006—roughly one every 170 trading days. In real order flow, seconds with several times the average activity occur routinely around large trades and announcements. A model that calls them once-a-year events will badly underestimate how quickly queues can empty (Section 6).

A compact way to remember the difference: a Poisson process is *drizzle*—drops fall independently at a steady rate, and one drop says nothing about the next. Order flow is a *storm*—long dry spells broken by downpours, in which each burst of rain makes more rain likely for a while.

Appendix C gives a self-contained introduction to these ideas: the Poisson, renewal and Hawkes processes, the conditional intensity, the branching ratio, and the statistics that tell the three apart on real data, with simulated examples.

The Hawkes process keeps what is convenient about the Poisson process—events at random times, described by a rate—but lets the rate *rise after each event* and then decay back. It is the smallest change to the Poisson model that makes events cause events. When every kernel below is zero, it reduces to independent Poisson processes with rates μ_i .

7.2 The multivariate model

Let $N_i(t)$ count events of type $i \in \{1, \dots, K\}$. A linear multivariate Hawkes process (Hawkes, 1971) has conditional intensity

$$\lambda_i(t) = \mu_i + \sum_{j=1}^K \int_0^t \phi_{ij}(t-t') dN_j(t'), \quad (18)$$

with baseline rates μ_i and non-negative kernels ϕ_{ij} . Here $\lambda_i(t)$ is the *intensity*: the expected number of type- i events per second at time t . The term $dN_j(t')$ is nonzero only at the instants t' when a type- j event occurred, so the integral is simply a sum over past events, and the formula can be read as $\lambda_i(t) = \mu_i + \sum_j \sum_{\text{past type-}j \text{ events at } t'} \phi_{ij}(t-t')$.

In words: the current rate of type- i events is a constant background rate μ_i plus a “bump” for every past event; each bump fades as that event recedes into the past, with a shape set by the kernel ϕ_{ij} . Events make further events more likely for a while, the way an earthquake raises the rate of aftershocks or a viral post triggers reposts.

The branching matrix $\Gamma_{ij} = \int_0^\infty \phi_{ij}(u) du$, the total area under a kernel, is the expected number of type- i events directly triggered by one type- j event. The process is stationary when the spectral radius $\rho(\Gamma) < 1$, the largest absolute eigenvalue of Γ . Then, with $\mu = (\mu_1, \dots, \mu_K)^\top$ and I the $K \times K$ identity matrix, the stationary mean intensity vector $\bar{\lambda} = (\bar{\lambda}_1, \dots, \bar{\lambda}_K)^\top$ is

$$\bar{\lambda} = (I - \Gamma)^{-1} \mu. \quad (19)$$

In words: expand the inverse as a geometric series, $(I - \Gamma)^{-1} \mu = \mu + \Gamma \mu + \Gamma^2 \mu + \dots$. The long-run event rate is the background events, plus the events they trigger, plus the events *those* trigger, and so on through every generation. The series converges—the cascade dies out—exactly when $\rho(\Gamma) < 1$. Readers who know discrete-time linear systems will recognise the condition: it is the stability condition for $\psi_{k+1} = \Gamma \psi_k + \mu$, a feedback loop whose gain must be below one.

A note on notation. Much of the literature writes the exponential kernel as $\alpha e^{-\beta u}$ and the (scalar) branching ratio as n . In this survey β is reserved for price impact (Section 4) and n for event indices, so the decay rate is written ω and the branching ratio Γ (a 1×1 branching matrix). With $\phi(u) = \alpha e^{-\omega u}$, the branching ratio is $\Gamma = \alpha/\omega$. Hawkes models have been applied extensively to order flow and price formation (Bacry et al., 2015).

Example 7.2 (Endogeneity in two dimensions). Take buy and sell market orders with $\mu = (1, 1)$ events per second and

$$\Gamma = \begin{pmatrix} 0.6 & 0.2 \\ 0.2 & 0.6 \end{pmatrix}.$$

The eigenvalues are 0.8 and 0.4, so $\rho(\Gamma) = 0.8 < 1$. Then $I - \Gamma = \begin{pmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{pmatrix}$ with determinant 0.12, and

$$\bar{\lambda} = \frac{1}{0.12} \begin{pmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

Only one event in five is exogenous. The model says a great deal about clustering and nothing, by itself, about direction: the two sides are symmetric.

7.3 Reading the branching matrix

The branching matrix is the economically interesting output of a fitted Hawkes model. Take two event types, buy and sell market orders:

$$\Gamma = \begin{pmatrix} \Gamma_{BB} & \Gamma_{BS} \\ \Gamma_{SB} & \Gamma_{SS} \end{pmatrix}.$$

Γ_{BB} is the average number of further buy market orders directly triggered by one buy market order: buys beget buys, for example because a large order is being executed in pieces. Γ_{SB} is the number of sell orders triggered by a buy: a cross effect, for example traders leaning against a move. A large diagonal relative to the off-diagonal means trends in order flow; a large off-diagonal means reversal. Asymmetry ($\Gamma_{BS} \neq \Gamma_{SB}$) means one side reacts more strongly to the other than vice versa. Linear Hawkes models allow only excitation; inhibition (an event making others *less* likely) requires non-linear extensions. The shape of each kernel ϕ_{ij} says how long an influence lasts: an exponential kernel forgets quickly, a power-law kernel remembers for a long time.

An analogy: acoustic feedback. A microphone near a loudspeaker picks up each sound and plays it again, slightly quieter. If the loop gain is below one, every echo is fainter than the last and a clap dies away; the closer the gain is to one, the longer the ringing lasts. At a gain of one or more the echoes grow and the system howls. The branching ratio is that loop gain for order flow: the average number of events that one event triggers. At 0.8, a single exogenous event produces on average $1/(1 - 0.8) = 5$ events in total, as in Example 7.2; at 0.95 it produces 20.

Example 7.3 (A chain reaction in the book). A stylised sequence at the best ask, all within a few milliseconds:

time	event
0.0 ms	a buy market order takes 10 of the 14 lots at the best ask (exogenous)
0.3 ms	two sellers at the best ask cancel, fearing a move up (triggered)
0.6 ms	a buyer joins the best bid, leaning with the flow (triggered)
0.9 ms	a second buy market order takes the remaining lots; the best ask ticks up (triggered)
1.2 ms	new sellers post at the new best ask; buyers follow at the new bid (triggered)

One outside event produced four more and a price move. In a Hawkes model each row raises the intensity of the event types in later rows—entries Γ_{ij} for cancellation after trade, trade after trade, limit order after cancellation—and the fitted kernels say how quickly that influence fades.

7.4 What Hawkes models are for

Hawkes models represent clustering, self- and cross-excitation, short-term persistence and the split between exogenous and endogenous activity. They are valuable for intensity forecasting and event

simulation even when directional alpha is weak. A model can reproduce the temporal statistics of order flow very well while adding little information about the sign of the next price move. Hawkes models should therefore be evaluated separately for intensity forecasting, simulation, price prediction and execution prediction (Table 1).

An execution use deserves emphasis. The intensity of aggressive flow on one side is a natural input to a cancellation decision for a passive order on that side: a spike in sell-market-order intensity raises the probability that a resting bid is about to be filled at a bad time. This is an adverse-selection question (Section 13), not a direction question.

7.5 From events to prices

The simplest bridge from event counts to prices treats each mid move as an event. Let $N^+(t)$ and $N^-(t)$ count the upward and downward mid moves up to time t , each of size δ_m . Then

$$m_t = m_0 + \delta_m (N^+(t) - N^-(t)). \quad (20)$$

In words: the price is a running tally of up-ticks minus down-ticks; modelling the two counting processes models the price.

Write $\lambda^+(t)$ and $\lambda^-(t)$ for the intensities of N^+ and N^- : the expected numbers of up and down moves per second. Bacry et al. (2013) make the two counts *mutually* exciting: an up move raises the intensity of down moves and vice versa,

$$\lambda^+(t) = \mu_0 + \int_0^t \alpha e^{-\omega(t-t')} dN^-(t'), \quad \lambda^-(t) = \mu_0 + \int_0^t \alpha e^{-\omega(t-t')} dN^+(t'), \quad (21)$$

In words: each down move adds a bump of height α to the up-move rate, and vice versa; the bump decays at rate ω . The background rate on each side is μ_0 .

The cross-kernel has total weight $\Gamma_\times = \alpha/\omega < 1$. This one mechanism produces the familiar microstructure effects. Measured over short windows, prices look very volatile because every up-tick tends to be followed by a down-tick (bid–ask bounce); over long windows that back-and-forth cancels. Writing $\sigma^2(w)$ for the variance of price changes per unit time measured over windows of length w , in units of ticks squared,

$$\sigma^2(w) = \Lambda \left(\frac{1}{(1 + \Gamma_\times)^2} + \left(1 - \frac{1}{(1 + \Gamma_\times)^2}\right) \frac{1 - e^{-(\alpha + \omega)w}}{(\alpha + \omega)w} \right), \quad \Lambda = \frac{2\mu_0}{1 - \Gamma_\times}. \quad (22)$$

In words: Λ is the total rate of price moves. For very short windows the measured variance is Λ —every move counts. For long windows it falls to $\Lambda/(1 + \Gamma_\times)^2 = 2\mu_0/((1 - \Gamma_\times)(1 + \Gamma_\times)^2)$, because moves that are immediately reversed contribute nothing to the long-run change. The curve between the two is the “signature plot” that practitioners use to choose a sampling frequency for volatility estimates.

Here cross-excitation *damps* long-run variance. Self-excitation, by contrast, inflates the variance of event *counts* relative to their mean rate (Bacry and Muzy, 2014); which effect dominates for prices depends on the structure of the kernel matrix, not on a single number. The two-dimensional version of the model also reproduces the Epps effect, the drop in measured correlation between two assets when it is computed over very short windows (Bacry et al., 2013).

Bacry and Muzy (2014) extend the construction to four kernels linking trades and price changes—trades exciting trades, prices reverting, trades moving prices, and prices triggering trades—and recover the concave, relaxing impact of large orders executed in pieces. The same machinery was used early to study book resilience (Large, 2007) and to make a zero-intelligence order-book simulator produce realistic spreads (Muni Toke, 2011).

7.6 What fitted Hawkes models find

Branching ratios. Estimates of the branching ratio for financial event streams have been high since the mid-2000s, but the exact value is contested. On E-mini S&P 500 futures, [Filimonov and Sornette \(2012\)](#) find it rising from about 0.3 around 1998–2000 to 0.7–0.8 after 2007, with spikes to 0.9. [Hardiman et al. \(2013\)](#), using power-law kernels, find the kernel integrating to approximately one—a market at criticality. [Filimonov and Sornette \(2015\)](#) argue that estimates near one are biased upward by outliers, kernel regularisation, edge effects and regime shifts—fitting a constant background rate to a period in which the true background varies; their re-estimation gives values that later fluctuate between about 0.8 and 1.1. On CME NQ market data, an exponential-kernel fit to packet arrivals gives about 0.80 (Appendix C). The defensible summary is that a large majority of events are endogenous, and that the precise number depends on the kernel family and on how non-stationarity is handled.

Kernel shape and long memory. Non-parametric estimation ([Bacry et al., 2016](#)) finds kernels that decay roughly as power laws with exponent slightly above one (in simulation the method recovers power laws over six decades of time). In an eight-dimensional model of DAX and Bund futures (price moves, trades, limit orders and cancellations, each split by side), trades, limit orders and cancellations mainly excite events of the same type and side, while price moves on one side mainly excite price moves on the other—mean reversion. The persistence of order flow is a long-documented fact: the signs of market orders form a long-memory process with a Hurst exponent of about 0.7 ([Lillo and Farmer, 2004](#)), explained mainly by the splitting of large orders into many small ones ([Lillo et al., 2005](#); [Bouchaud et al., 2009](#)). Power-law Hawkes kernels are the point-process expression of the same fact.

7.7 Near-criticality

When the branching ratio approaches one, appropriately rescaled Hawkes processes converge to continuous limits. With light-tailed kernels the limit is an integrated Cox–Ingersoll–Ross process, and the price model of [Bacry et al. \(2013\)](#) converges to a Heston stochastic-volatility model ([Jaisson and Rosenbaum, 2015](#)). With heavy-tailed kernels—power laws with tail $\propto u^{-(1+\zeta)}$ with $\zeta \in (1/2, 1)$ —the limit is a *rough* fractional process with Hurst parameter $\zeta - 1/2$ ([Jaisson and Rosenbaum, 2016](#)), which the authors offer as a microstructural foundation for rough volatility. Empirically, log-volatility does behave like a fractional process with a Hurst exponent of order 0.1—for example 0.125 for DAX and 0.082 for Bund futures ([Gatheral et al., 2018](#)). The lesson for LOB modelling is qualitative: strong endogenous excitation, together with long memory, generates volatility that is itself random and persistent. It is not that markets literally sit at a singularity, nor that a single scalar multiplies volatility.

7.8 State-dependent Hawkes models

The standard specification lets intensity depend on past events but not on the current book. A state-dependent form ([Morariu-Patrichi and Pakkanen, 2022](#)) couples the two; one convenient parametrisation is

$$\tilde{\lambda}_i(t) = \lambda_i(t) \exp\left(\vartheta_i^\top X_{t-}\right), \quad (23)$$

where $\tilde{\lambda}_i(t)$ is the state-dependent intensity, $\lambda_i(t)$ the Hawkes intensity defined above, ϑ_i a vector of coefficients, and the state X_{t-} may contain the spread, queue imbalance, depth, volatility or cross-asset variables.

In words: take the Hawkes rate and multiply it by a factor that depends on the current book. The dot product $\vartheta_i^\top X_{t-}$ is a weighted score of the book’s features, as in logistic regression, and the exponential turns any score into a positive multiplier, so the rate can never go negative.

The gain is realism; the cost is more parameters and more work to fit them. The relevant question is whether state dependence provides incremental out-of-sample economic value sufficient to justify that cost.

Model card: State-dependent Hawkes

What you calibrate. Everything in the linear Hawkes model, plus the coefficient vector ϑ_i that scales each intensity by the current book state.

How. Maximum likelihood, as for the linear model, with the book state X_{t-} evaluated just before each event; more parameters, so more data and regularisation are needed.

What you do with it. Intensity forecasts that react to the current spread, imbalance and depth, for simulation and for timing passive orders.

How to picture it. The same aftershock model, with a dial set by the state of the book that turns every rate up or down.

7.9 Fitting Hawkes models in practice

Maximum likelihood. The standard method chooses the parameters that make the observed event times most probable (Ozaki, 1979). For a K -type process observed on $[0, t_{\text{end}}]$ the log-likelihood is a sum over types,

$$\ln \text{Lik} = \sum_{i=1}^K \left(\sum_{n: \text{event } n \text{ of type } i} \ln \lambda_i(t_n) - \int_0^{t_{\text{end}}} \lambda_i(t') dt' \right), \quad (24)$$

In words: for every type, reward the model for a high intensity at the moments type- i events happened, and penalise it for the total number of type- i events it expected.

Because λ_i depends only on μ_i and on the kernels $\phi_{i1}, \dots, \phi_{iK}$ in row i of the kernel matrix, the i -th term involves only those parameters. As long as no parameter is shared across rows, the fit splits into K independent problems, one per target type, each with $1 + K$ kernels’ worth of parameters (Wu et al., 2019; Bacry et al., 2015). For exponential kernels each term is computed in one pass with a running sum (Appendix C.13), so the cost grows only linearly with the number of events (Bacry et al., 2015).

Three practical rules follow. First, parameters have natural bounds— $\mu_i > 0$, jumps $\alpha_{ij} \geq 0$, decay $\omega > 0$ —and the optimiser should enforce them. Second, nothing in the likelihood forces the fitted Γ to satisfy $\rho(\Gamma) < 1$; it must be checked after fitting, and a fit at or above one read as a sign of non-stationarity or a wrong kernel rather than as a fact about the market (Appendix C.8). Third, the likelihood is not convex in the decay rates, but if the decays are fixed in advance—for example a few exponentials with chosen time scales—it becomes convex in the background rates and jumps (Wu et al., 2019), so the remaining fit has a single optimum.

Expectation–maximisation. Direct maximisation can be unstable because the likelihood is flat in some directions. The branching structure suggests a different route (Veen and Schoenberg, 2008): if we knew which earlier event triggered each event, or that it was a background event, the parameters would be simple averages. The EM algorithm alternates between guessing those parents probabilistically and re-estimating. For one type with kernel $\alpha e^{-\omega u}$ (Lewis and Mohler, 2011):

- *E step*: the probability that event n was triggered by earlier event n' is $\alpha e^{-\omega(t_n - t_{n'})} / \lambda(t_n)$, and the probability that it was a background event is $\mu / \lambda(t_n)$;
- *M step*: μ is the expected number of background events divided by t_{end} ; Γ is the expected number of triggered events divided by the number of events; and $1/\omega$ is the probability-weighted average of the parent–child delays $t_n - t_{n'}$; then $\alpha = \Gamma\omega$.

In words: give each event a fractional family tree, then count: how many events had no parent, how many children an event has on average, and how long children take to arrive.

Moments and cumulants. A second family avoids the likelihood altogether and matches statistics of the counts. The Wiener–Hopf method estimates kernel shapes from the empirical second-order statistics of the event streams (Bacry and Muzy, 2016) (it underlies the non-parametric results of Bacry et al. (2016)); the cumulant method NPHC estimates only the integrated kernels—the branching matrix itself—from second- and third-order integrated cumulants, with no assumption on kernel shape (Achab et al., 2018); and method-of-moments calibration with closed-form moments can be rolled forward through time cheaply (Da Fonseca and Zaatour, 2014).

The branching ratio from count dispersion. The simplest moment estimator needs only counts. Split the data into windows of length w , and compute the mean and variance of the number of events per window; their ratio is the Fano factor $D(w)$ of Appendix C.11. For a one-type Hawkes process whose correlations die out on time scales much shorter than w , Hardiman and Bouchaud (2014) show that

$$\Gamma \approx 1 - \sqrt{1/D(w)}. \quad (25)$$

In words: self-excitation makes counts burstier than Poisson; the more bursty they are, the closer the branching ratio is to one. A Poisson stream ($D = 1$) gives $\Gamma = 0$.

The estimate is biased low for finite windows and exact only as w grows without bound, and because the sample variance can be zero the authors report medians. Applied to the Fano factors of CME NQ packet data reported in Appendix C.11—about 52 at $w = 0.1$ s and about a thousand at $w = 60$ s—the formula gives $\Gamma \approx 0.86$ and $\Gamma \approx 0.97$, against 0.80 from the exponential-kernel likelihood fit on the same kind of data. Two cautions make this a teaching example rather than a measurement. The estimate keeps rising with w , the same symptom that a slowly varying background rate produces (Filimonov and Sornette, 2015). And the same stream with its gaps shuffled—which destroys all self-excitation but keeps the gap distribution—has $D \approx 17$, for which the formula would still report $\Gamma \approx 0.76$: dispersion alone does not prove excitation, because a renewal process with very irregular gaps is also over-dispersed.

Non-stationarity. A single fit over a long window assumes a constant background rate. Because activity follows intraday patterns and changes across regimes, such fits overstate endogeneity (Filimonov and Sornette, 2015). In practice models are fitted over short windows, with a time-varying background, or with the background modelled explicitly.

Online use. With exponential kernels the intensity is updated in constant time per event, which makes Hawkes intensities usable inside a low-latency trading loop. The parameters change more slowly. One workable scheme, which we describe rather than cite, holds the decay rates fixed during a session and re-fits the background rates and jumps on a rolling window, starting each fit from the previous solution; with fixed decays each re-fit is a convex problem. Fully online estimators also

exist: for example, [Yang et al. \(2017\)](#) update non-parametric kernels with projected gradient steps on a discretised, regularised likelihood, with regret growing only logarithmically in time. Whatever the scheme, the gradient of the log-likelihood contains the compensator term $-\int \lambda_i$ as well as the event term, so an update that looks only at the events (“the rate was high when events happened”) and ignores how many events the model expected will push the intensities up without limit.

Goodness of fit: time rescaling. A fitted point process can be checked with a classical transformation. Replace each event time t_n by the expected number of events up to it under the fitted model, $\int_0^{t_n} \hat{\lambda}(t') dt'$. If the model is right, the transformed times form a Poisson process with rate one, so the gaps between them are independent and exponential with mean one ([Ogata, 1988](#); [Brown et al., 2002](#)). A quantile–quantile plot of those gaps against the exponential distribution, or a Kolmogorov–Smirnov test, shows where the model fails—for example, too few very short gaps means the model underestimates clustering.

Simulation. Hawkes paths are generated by *thinning* ([Ogata, 1981](#)): propose a candidate event using an upper bound on the current intensity, which is valid because exponential intensities only decay between events, and accept it with probability equal to the true intensity divided by the bound. Simulated paths are how a fitted model answers questions that have no closed form, such as the probability that a queue empties within a given latency.

Software. The open-source library *tick* ([Bacry et al., 2018](#)) implements maximum-likelihood fits with exponential and sum-of-exponential kernels, EM, Wiener–Hopf and cumulant estimators, and simulation.

Model card: Linear (multivariate) Hawkes

What you calibrate. Background rates μ_i ; the kernels ϕ_{ij} —for exponential kernels, the jumps α_{ij} and decay ω —and hence the branching matrix Γ .

How. Maximum likelihood, split into one problem per target type, with bounded parameters and a one-pass running sum for exponential kernels; or EM, Wiener–Hopf, cumulant and dispersion estimators. Fit on short or rolling windows, check $\rho(\Gamma) < 1$ afterwards, and test the fit by time rescaling.

What you do with it. Forecast event intensities and, from them, drift, expected OFI and depletion times; simulate order flow by thinning; monitor how much activity is endogenous; raise a cancellation alarm when the aggressive intensity against a resting order spikes.

How to picture it. Aftershocks after an earthquake, or acoustic feedback: each event raises the rate of further events, which fades; the branching ratio is the loop gain.

7.10 From intensities to decisions

A fitted Hawkes model gives, at every instant, the expected rate of each kind of event. Four quantities a trader can use follow directly; each is valid only over horizons short enough that the intensities do not change much, because they decay between events.

Drift. In the price model of the previous subsection, up and down moves of size δ_m arrive at rates $\lambda^+(t)$ and $\lambda^-(t)$, so over a short horizon H the expected mid change is approximately

$$\mathbb{E}[m_{t+H} - m_t] \approx \delta_m (\lambda^+(t) - \lambda^-(t)) H.$$

In words: if up moves are currently arriving faster than down moves, the price is expected to drift up, by one tick per surplus up move.

Expected order-flow imbalance. Model six event types at the best quotes: limit orders, cancellations and market orders on each side, with intensities $\lambda^{L,b}$, $\lambda^{C,b}$, $\lambda^{M,b}$ (bid queue added to; bid queue cancelled; market *buy*, which takes from the ask) and $\lambda^{L,a}$, $\lambda^{C,a}$, $\lambda^{M,a}$ (ask queue added to; ask queue cancelled; market *sell*, which takes from the bid). With $\bar{v}$ the average event size and the best prices unchanged, the expected OFI per second is

$$\bar{v} \left(\underbrace{\lambda^{L,b} - \lambda^{C,b} - \lambda^{M,a}}_{\text{net change of the bid queue}} - \underbrace{(\lambda^{L,a} - \lambda^{C,a} - \lambda^{M,b})}_{\text{net change of the ask queue}} \right).$$

In words: the same score as Section 4, but computed from expected rates instead of observed events: additions to the bid and removals from the ask count as buying pressure.

It is a forecast of OFI, which by the regression of Section 4 is a forecast of the price change.

Depletion time. The bucket picture of Section 6 needs an inflow and an outflow. For the best ask, $r_{\text{in}} = \bar{v} \lambda^{L,a}$ and $r_{\text{out}} = \bar{v} (\lambda^{M,b} + \lambda^{C,a})$, so with the current intensities

$$\mathbb{E}[\tau^a] \approx \frac{v_{1,t}^a}{r_{\text{out}} - r_{\text{in}}}, \quad r_{\text{out}} > r_{\text{in}}.$$

When this falls below the trader’s own latency T_{lat} , a sell order resting in that ask queue is in danger: if the trader decides to cancel, the level may be swept—filling the order just before the price moves up—before the cancellation arrives. The full probability $\mathbb{P}(\tau^a \leq T_{\text{lat}})$ has no closed form under a Hawkes model, but can be estimated by simulating from the current state by thinning.

A branching-ratio monitor. Re-fitting Γ on a rolling window gives a running measure of how much of current activity is self-generated. A rising value signals a market in which events are feeding on each other. Two cautions apply. A rise can be produced by a changing background rate rather than by more excitation (Filimonov and Sornette, 2015); and published estimates differ widely by instrument, period and method (Section 7), so a fixed universal alarm level is not justified. A threshold should be set from the instrument’s own history, and the monitor is a regime indicator, not a forecast of direction.

7.11 Variants of the Hawkes model

The linear model with exponential kernels is a starting point. Each variant below relaxes one of its assumptions; Table 10 gives the model card for each.

Sums of exponentials. Power-law kernels capture long memory but lose the constant-time update. A power law can be approximated over many decades by a small number of exponentials with geometrically spaced decay rates—for an exponent of two, about 1.7 exponentials per decade (Bochud and Challet, 2007)—and each exponential keeps its own running sum, so the intensity is still updated in constant time per event. With the decay rates fixed, the likelihood is convex in the remaining parameters (Wu et al., 2019).

Nonlinear and inhibitory kernels. Linear Hawkes kernels can only excite. A nonlinear Hawkes process passes the sum through a function that keeps the intensity non-negative, $\lambda_i(t) = h(\mu_i + \sum_j \int \phi_{ij}(t - t') dN_j(t'))$, which allows negative kernels, that is, events that make others *less* likely; general conditions for a stationary version were given by Brémaud and Massoulié (1996). In limit-order-book data, Lu and Abergel (2018) observe inhibition effects that lead them to nonlinear Hawkes models, and Rambaldi et al. (2017) find that limit orders (and cancellations) on one side inhibit the same type on the other side, which they read as a fair-price effect.

Marks: the size of the event. A 100-lot trade should excite more than a 1-lot trade, but the obvious fix—multiplying the kernel by a function of size—is rejected by the data: on Bund and DAX futures the kernels for different trade sizes have different *shapes* (Rambaldi et al., 2017). The authors instead split events into volume classes and treat each class as its own event type. They find that large trades excite limit orders on the opposite side, and that large orders are more exogenous than small ones.

Regimes. In a Markov-modulated Hawkes process a hidden state switches among a few regimes, each with its own background rate and decay, and the parameters are fitted by EM (Wang et al., 2012). It was developed for earthquake sequences (main shocks, aftershocks, quiescence); the analogue for markets is a model whose quiet and busy regimes have different parameters, rather than one set of parameters averaged over both.

Quadratic Hawkes and the Zumbach effect. Volatility has a time asymmetry: past volatility measured over long horizons predicts future short-horizon volatility better than the reverse—the *Zumbach effect* (Zumbach, 2009). A Hawkes intensity that depends only on past event counts cannot produce it. The quadratic Hawkes model of Blanc et al. (2017) adds a term in the *square* of a moving average of past signed price changes, so that a sequence of moves in the same direction raises activity more than the same number of moves that cancel out. Calibrated on five-minute bins of 133 NYSE stocks over 2000–2009, it reproduces the time asymmetry, and it generates long memory in volatility without requiring the process to be critical.

Queue-reactive Hawkes. Wu et al. (2019) combine the two ideas of this part: the background rates depend on the current queue size, as in Section 6.3, and events excite each other through sum-of-exponential kernels. On Bund and DAX futures the Hawkes term greatly improves the pure queue-reactive model, both in the distribution of times between events and in the distribution of queue sizes; market-order and price-change rates are mainly functions of the volume imbalance, while limit and cancellation rates are not.

Cross-asset Hawkes. Events in one instrument can excite events in another; this is the point-process form of lead–lag, discussed in Section 10.

7.12 Neural point processes

Where we are. Everything so far in this section has the same structure: the rate of type- i events is a background rate plus a sum of bumps, one per past event, each with a fixed shape ϕ_{ij} . The variants of Section 7.11 change one ingredient at a time—the shape of the bump, its sign, its dependence on size or on the queue. A neural point process changes all of them at once: it keeps the idea of an intensity, but lets a neural network compute it from the history. This subsection explains what that means, step by step.

What the Hawkes formula cannot say. Three limitations of the linear Hawkes model motivate the step.

1. **Effects only add.** Each past event contributes its own bump, independently of the others. In the chain reaction of Example 7.3, a trade followed by two cancellations at the same level is far more alarming than either alone: together they say the level is about to go. A sum of separate bumps cannot express “A *and then* B matters more than A plus B”.
2. **The shape is fixed in advance.** The analyst chooses exponential, power law, or a sum of exponentials, and the data only set the parameters.
3. **The rate can only jump up and then decay.** Between events, a linear Hawkes intensity with positive kernels can only fall. Real flow can do the opposite: after a sweep, new limit orders may arrive slowly at first and then faster as participants re-quote.

The recipe. Every neural point process has three parts.

1. Describe each event as a vector

Event n is turned into a vector of numbers (an embedding) built from its type, the time since the previous event $t_n - t_{n-1}$, and any marks such as size or side.

2. Summarise the history

A sequence model reads the embeddings of events $1, \dots, n$ in order and produces a *history vector* $\mathbf{z}^{\text{hist}}(t)$ that summarises everything that happened before time t . This is the role played in a Hawkes model by the running sums of decaying bumps.

3. Turn the summary into intensities

For each event type i , the intensity is

$$\lambda_i(t) = \text{softplus}(\langle \mathbf{o}_i, \mathbf{z}^{\text{hist}}(t) \rangle), \quad \text{softplus}(y) = \ln(1 + e^y), \quad (26)$$

where $\mathbf{o}_i$ is a vector of learned output weights for type i and y stands for any real number.

In words: score the current history with a weighted sum, one set of weights per event type, and pass the score through the softplus, a smooth function that is always positive and close to the score itself when the score is large. Rates must be positive; softplus guarantees it, as the exponential did in the state-dependent model.

A linear Hawkes model is the special case in which the “summary” is the vector of running sums $\sum \phi_{ij}(t - t_{n'})$ and the output step simply adds them to μ_i . A neural model replaces the hand-written summary with a learned one.

Training: the same likelihood as Hawkes. The parameters—the network weights and the output weights $\mathbf{o}_i$ —are fitted by maximising exactly the log-likelihood of Section 7.9: reward high intensity at the times events happened, penalise the total expected number of events $\int \lambda_i$. Gradient descent replaces the specialised optimisers, because the network has many parameters. One new difficulty appears. For an exponential kernel the integral $\int \lambda_i$ has a closed form; for a neural intensity it usually does not, and is estimated numerically—commonly by evaluating the intensity at randomly sampled times between events and averaging (Monte Carlo integration). Two lines of work remove this approximation: Omi et al. (2019) model the *integral* of the intensity with a network constrained to be increasing, and obtain the intensity as its derivative, so the likelihood is exact; Shchur et al. (2020) skip the intensity altogether and model the probability distribution of the time to the next event directly, for example as a mixture of simple distributions, which also makes sampling the next event exact.

Three ways to summarise the history. The models in the literature differ mainly in step 2.

Recurrent networks

An early and widely cited model, RMTTP (Du et al., 2016), updates a recurrent network’s state at each event and lets the intensity change between events through a fixed function of the elapsed time. The *neural Hawkes process* (Mei and Eisner, 2017) makes the recurrent state itself evolve in continuous time: each memory cell of an LSTM decays exponentially between events towards a target value set at the last event. Because the target can be above or below the current value, the intensity can rise or fall between events, and one event can make another *less* likely—the inhibition that linear Hawkes cannot represent.

Attention

The Transformer Hawkes process (Zuo et al., 2020) and the self-attentive Hawkes process (Zhang et al., 2020) let each event attend to all earlier events, as in the queries–keys–values box of Section 8, with the time gaps encoded into the embeddings. The attention weights play the part of the kernel: they say how much each past event matters now, but are computed from the content of the events rather than fixed in advance. Zhang et al. argue that the attention weights also give an interpretable measure of influence between event types.

State-space models

The Mamba Hawkes process (Gao et al., 2024) summarises the history with a selective state-space model (Section 8), which processes long sequences in time linear in their length.

Shchur et al. (2021) survey the family and its design choices.

Example 7.4 (What a learned summary can add). Take the chain reaction of Example 7.3. A two-type Hawkes model of trades and ask cancellations gives the intensity of a further buy market order as background plus one bump for the trade and one for each cancellation, whatever their order. A neural model reads the sequence “buy trade, cancel, cancel” as a whole. If, in the training data, that particular pattern was usually followed within a millisecond by a sweep of the level, the model can raise the intensity of buy market orders far more than the sum of the separate bumps would, and leave it unchanged after the same three events in a different order. Whether it learns this depends entirely on whether the pattern is frequent enough in the data; the flexibility is useless without the examples.

Applications to order books. The Transformer Hawkes process and the Mamba Hawkes process both report results on a data set of financial transactions (buy and sell events of a stock over a day), among other benchmarks (Zuo et al., 2020; Gao et al., 2024). Models built specifically for the book go further. Shi and Cartlidge (2022) compute a separate intensity for each order-book event type with parallel continuous-time LSTMs and add an interaction between events and the state of the book; on five days of LOBSTER data for three US stocks it outperforms both classical and neural point-process baselines in likelihood, timing and event-type accuracy, and the authors note that their simulator settings remain simple. Lalor and Swishchuk (2025) drive a twelve-event-type order-book simulator with an LSTM-based neural Hawkes process and use it as the training environment for a reinforcement-learning market maker.

What is lost. The flexibility has three costs, and each matters in this survey’s setting.

- **Interpretability.** A linear Hawkes fit returns a branching matrix whose entries have a meaning (Section 7) and a branching ratio that can be monitored. A neural model returns weights; quantities such as the fraction of endogenous events have no direct equivalent.

- **Speed.** An exponential Hawkes intensity is updated with a few multiplications per event. A neural intensity needs a forward pass of a network per event, and more if the intensity is needed between events—a cost that matters inside a trading loop (Section 34).
- **Guarantees.** The stability condition $\rho(\Gamma) < 1$ has no simple analogue, so a simulated neural model can drift into regimes never seen in training.

The evaluation follows Principle 3. The baseline is a multivariate Hawkes model, ideally with sum-of-exponential kernels, fitted to the same events; the comparison is the held-out log-likelihood per event, the time-rescaling test of Section 7.9 (which applies to any model that outputs an intensity), and, for a trading use, the economic value of the forecasts. A neural point process earns its place when it improves these by more than its cost in speed and interpretability.

Model card: Neural point processes

What you calibrate. The embedding of events, the weights of the sequence model that summarises the history (recurrent, continuous-time recurrent, attention or state-space), and the output weights $\mathbf{o}_i$ that turn the summary into an intensity for each event type.

How. Gradient descent on the point-process log-likelihood, with the integral of the intensity estimated by Monte Carlo, or made exact by modelling the integral or the next-gap distribution directly. Compare on held-out likelihood and by time rescaling against a multivariate Hawkes baseline.

What you do with it. Predict the type and time of the next event; simulate order flow, including as a training environment for execution or market-making agents; capture interactions between events that additive kernels miss.

How to picture it. A Hawkes model whose bumps are drawn by a network that has read the whole history in order, instead of by a fixed formula: it can notice that “trade, cancel, cancel” means more than the three events separately.

8 Deep learning for limit order books

Deep learning entered LOB research around 2017 and now dominates the volume of published work. This section describes what the main families of models actually do, what the large benchmark studies have found when the models are re-tested, and which design choices matter. Readers new to the architectures will find short definitions of convolutional, recurrent and attention-based networks in the glossary (Appendix B) and textbook treatments in Goodfellow et al. (2016).

8.1 Problem formulations

Most deep LOB papers solve one of six problems:

Direction classification

predict whether the mid will be up, down or stationary after a horizon H measured in events or seconds. Labels are usually built by comparing a smoothed future mid with the current one against a threshold; the choice of smoothing and threshold changes the problem, and a recent paper shows that coupling the smoothing window to the horizon biases the labels (Berti and Kasneci, 2025).

Multi-horizon forecasting

predict returns, or their direction, over several horizons at once—an “alpha term structure” (Kolm et al., 2023; Zhang and Zohren, 2021).

Next-event prediction

predict the type, side, price and size of the next message—the natural task for message-level and point-process models (Linna et al., 2025).

Full-book forecasting

predict the whole future book, prices and volumes at several levels (Jung and Lee, 2025).

Fill-time estimation

predict the distribution of the time until a resting order is filled (Section 8.8).

Generation

produce synthetic message streams or book snapshots (Section 8.9).

The benchmark data set. A large share of the literature reports results on FI-2010 (Ntakaris et al., 2018): five stocks from the Helsinki exchange over ten trading days in June 2010, about 4.5 million messages, ten levels, already normalised and downsampled into blocks of ten events. It made comparison possible, and it also made overfitting easy. The DeepLOB authors themselves describe it as “far too short, downsampled and taken from a less liquid market” (Zhang et al., 2019). Results on it should be read with the comparability fields of Section 2.4 in mind.

8.2 Convolutional and recurrent models

First models. Tsantekidis et al. (2017a) applied a convolutional network to sequences of book snapshots, treating the book like an image whose two axes are time and price level. A companion paper used an LSTM with 40 hidden units on the last 100 snapshots, reporting F1 scores of about 61–66% on FI-2010 against 48–56% for a multilayer perceptron (Tsantekidis et al., 2017b). Dixon (2018) applied a recurrent network to E-mini S&P 500 Level-2 data to classify the next price flip.

DeepLOB. Zhang et al. (2019) combined the two ideas and became the most widely used reference architecture. Its input is the last 100 snapshots of ten levels (40 numbers each: price and size for bid and ask at each level). Small convolutions first combine price with size at each level, then bid with ask, then levels with each other; an Inception module applies parallel convolutions over several time scales; a 64-unit LSTM summarises the sequence; and a softmax outputs probabilities for down, stationary and up. On a year of London Stock Exchange data for five stocks it reports F1 of about 70%, 63% and 61% at horizons of 20, 50 and 100 events, with similar numbers on five stocks not seen in training—evidence that it learns features that transfer. Its trading simulation, as the authors state, uses mid-prices without transaction costs and is “not ... a fully developed, stand-alone trading strategy”.

Learning across many stocks. Sirignano (2019) introduced a “spatial” network that exploits the ordering of price levels to use information deep in the book with few parameters, trained on 489 US stocks over twenty months on a GPU cluster; it outperforms logistic regression and a standard network, especially in the tails of the distribution. Sirignano and Cont (2019) train a single model on many stocks and find that it outperforms stock-specific models, which they interpret as evidence of universal features of price formation.

Lighter models and normalisation. Not every gain needs more capacity. The temporal attention-augmented bilinear network (TABL) acts separately on the feature and time axes of the input matrix and reaches competitive FI-2010 results with about 10^4 parameters (Tran et al., 2019).

Deep adaptive input normalisation (DAIN) learns how to normalise each input feature (Passalis et al., 2020); on FI-2010 it raises the macro-F1 of a simple MLP from about 55% with standard z-scoring to about 68%. Because the statistics of LOB data drift, how inputs are normalised is a modelling decision in its own right.

Model card: Convolutional and recurrent models (DeepLOB family)

What you calibrate. The network weights (from about 10^4 to 10^5 parameters for TABL and DeepLOB), and before that the input normalisation, the label threshold and the horizon.

How. Supervised training with a cross-entropy loss on labelled windows of snapshots (down/stationary/up), with chronological train, validation and test splits.

What you do with it. Short-horizon direction probabilities, used as a signal or as one input to an execution decision.

How to picture it. The book as an image—time across, price levels down—scanned by small filters that find local patterns, followed by a memory that reads the patterns in order.

8.3 Attention and spatial-temporal models

Attention. An attention layer lets each element of a sequence form a weighted average of all the others, with weights computed from the data (Vaswani et al., 2017). Applied to LOBs, the sequence can run along time, along price levels, or along assets. A model that attends along more than one of these axes is called *spatial-temporal*: “space” is the price ladder (and, in multi-asset models, the set of instruments), “time” is the sequence of snapshots or events.

Queries, keys and values on an order book

Each event n is first turned into a vector of numbers (its embedding). Three learned matrices turn that vector into a *query* $\mathbf{z}_n^{\text{qry}}$ (“what am I looking for?”), a *key* $\mathbf{z}_n^{\text{key}}$ (“what do I contain?”) and a *value* $\mathbf{z}_n^{\text{val}}$ (“what do I pass on?”), each of dimension dim_{emb} . Event n attends to earlier event n' with weight

$$\text{att}_{nn'} = \frac{\exp(\langle \mathbf{z}_n^{\text{qry}}, \mathbf{z}_{n'}^{\text{key}} \rangle / \sqrt{\text{dim}_{\text{emb}}})}{\sum_{n'' \leq n} \exp(\langle \mathbf{z}_n^{\text{qry}}, \mathbf{z}_{n''}^{\text{key}} \rangle / \sqrt{\text{dim}_{\text{emb}}})}, \quad \text{output}_n = \sum_{n' \leq n} \text{att}_{nn'} \mathbf{z}_{n'}^{\text{val}}.$$

In words: each event scores every earlier event by how well its key matches the query, turns the scores into weights that sum to one (the softmax), and takes the weighted average of their values.

A stylised sweep. The current event is a buy market order that empties the best ask. If training has taught the model that sweeps matter when sellers have been leaving, the sweep’s query matches the keys of the ask cancellations in the last few milliseconds, which receive most of the weight; the bid limit orders and the quiet stretch before them receive little. The output is, in effect, a summary “sellers were pulling before this sweep”—the chain reaction of Example 7.3, read off by a learned look-up rather than a fixed kernel.

TransLOB. Wallbridge (2020) combined five dilated causal convolutions with two Transformer blocks using masked (causal) self-attention, and reported FI-2010 F1 of 81–92% across horizons. An independent re-implementation obtained 59% on the same data (Prata et al., 2024)—the largest gap between claimed and reproduced performance among the fifteen models tested there, and a reminder that results on one small data set are fragile.

Axial and dual attention. Axial-LOB (Kisiel and Gorse, 2022) applies gated attention separately along the time and feature axes, keeping the parameter count below 10^4 . TLOB (Berti and Kasneci,

2025) alternates temporal self-attention (across snapshots) with spatial self-attention (across the features of a snapshot); its companion MLPLOB replaces attention by multilayer perceptrons that mix along the two axes, and matches TLOB on several data sets. Three observations from TLOB are instructive. Performance on Nasdaq and Bitcoin data falls steeply with the horizon—for INTC, F1 drops from about 80% at 10 events to about 50% at 100—whereas on FI-2010 it *rises* with the horizon, a sign of the data set’s peculiarities. Predictability on the same stock declined between 2012 and 2015. And when the label threshold is set to the average spread, so that only moves large enough to pay for crossing count, F1 drops to 31–41% (TSLA, horizons of 50 to 200 events). The authors conclude that the methods “are not sufficiently mature for practical deployment”.

Patches, full books and many assets. LiT (Xiao et al., 2025) cuts the book into structured patches—one side of the book over a short time window—as vision Transformers do with images, and applies it to Binance cryptocurrency data. Jung and Lee (2025) forecast the entire five-level book with a sequence-to-sequence Transformer and a loss that penalises forecasts violating the ordering of prices; on one day of data a linear model is slightly better on the overall forecasting error and on volumes, but worse on the mid-price and much worse at respecting the ordering of prices. OF-MATNet (Bandalina et al., 2025) feeds multi-level order-flow imbalance from many Nasdaq assets into a Transformer that attends along time, assets and levels, choosing peer assets by rolling Granger-causality tests, and reports improvements in out-of-sample R^2 in over 90% of cases on 110 assets. HLOB (Briola et al., 2025b) builds a graph over volume levels and processes it with a homological convolutional network.

Multi-horizon models and training hardware. Zhang and Zohren (2021) attach a sequence-to-sequence decoder with attention to the DeepLOB encoder to produce forecasts at several horizons at once, and report that training on Graphcore intelligence processing units took about 15% of the wall-clock time of a GPU.

Model card: Attention and spatial-temporal Transformers

What you calibrate. The query, key and value matrices of every attention layer, the feed-forward weights, and the choice of axes (time, levels, assets) along which to attend.

How. Supervised training as for the convolutional models; reproduction studies show that the result depends heavily on the data set and labelling, so re-testing on new data is part of calibration.

What you do with it. Direction and multi-horizon forecasts, full-book forecasts, and multi-asset models that choose which peers to read.

How to picture it. Every event asks a question of every earlier event and listens most to the ones whose answers match.

8.4 State-space models

Structured state-space sequence models such as S4 (Gu et al., 2022) and Mamba (Gu and Dao, 2023) process long sequences with a learned linear recurrence—the same mathematical object as the classical state-space model of control theory (Kalman, 1960)—computed efficiently in parallel. They are attractive for event streams, which are long and irregular. Apart from a hardware-aware design for cryptocurrency tick data (Popov and Huber, 2026), we could not identify a widely cited state-space model for LOB forecasting at the time of writing; this is an open direction rather than an established result (Section 36).

Model card: State-space sequence models

What you calibrate. The matrices of a learned linear recurrence and, for selective models such as Mamba, the functions that set its step size from the input.

How. Supervised training by gradient descent, with the recurrence computed in parallel during training.

What you do with it. Long-context forecasts on long, irregular event streams; for LOB data this is an open direction rather than an established result.

How to picture it. A Kalman-style running state, updated at every event, whose rate of forgetting is learned.

8.5 What the large benchmark studies show

Individual model papers report results on their own terms. Two studies re-test many models under common conditions, and two more vary the input or the evaluation while holding the model fixed; their findings are more informative than any single paper.

LOBCAST. [Prata et al. \(2024\)](#) re-implemented fifteen published models and tested them on FI-2010 and on new Nasdaq data from 2021 and 2022. Their headline is that “all models exhibit a significant performance drop when exposed to new data”: F1 on the new data lies between 48% and 61% for every model. The best model on the new data, BiNCTABL, loses about a fifth of its FI-2010 score. Five of the six best models use attention. A simple MLP ranks near the bottom but degrades the least. Parameter counts range from about 10^4 to 10^7 , with no clear relation between size and degradation. A trading simulation shows that profitability “is far from guaranteed”.

Simple models on the same inputs. [Briola et al. \(2020\)](#) compare a random classifier, logistic regression, an MLP, LSTMs and a CNN–LSTM on identical inputs for one large-tick stock. The MLP matches or exceeds the CNN–LSTM at all three horizons, while logistic regression is clearly worse. Linear models are not generally competitive with deep ones on raw book inputs; small non-linear models often are.

Representation. [Lucchese et al. \(2024\)](#) hold the DeepLOB architecture fixed and change only the input, on ten Nasdaq stocks. Models fed order-flow inputs or a new volume-on-a-price-grid representation “considerably” outperform those fed the raw level-by-level book, and predictability is detected up to about 50 order-book events ahead at the 99% confidence level, using model confidence sets ([Hansen et al., 2011](#)) to compare models. The authors stress that mid-to-mid returns “are not tradable in practice”. [Kolm et al. \(2023\)](#), on 115 Nasdaq stocks, find that networks trained on order flow “significantly outperform most models trained directly on order books”, and that the effective horizon of stock-specific forecasts is about two average price changes. In a later practitioner-oriented account, as reported in coverage of the authors’ 2026 award, very deep models “don’t really help” for this task, and careful time-stamping and stationary transformations of inputs mattered more ([Kolm and Westray, 2025](#)). [Wu et al. \(2021\)](#) show that level-based inputs are fragile: adding orders in empty price ticks drops DeepLOB’s accuracy from about 77% to about 51%, and they propose representations on a fixed price grid as an alternative.

From forecasts to trades. [Briola et al. \(2025a\)](#) retrain DeepLOB on fifteen Nasdaq stocks grouped by tick size and introduce a transaction-level score,

$$\text{TO} = \frac{N_{\text{correct}}}{N_{\text{possible}} + N_{\text{predicted}} - N_{\text{correct}}}, \quad (27)$$

where N_{possible} counts the round-trip trades (open, then close) that the true price path would have allowed, $N_{\text{predicted}}$ those the forecasts would have triggered, and N_{correct} those in both sets.

In words: TO, which its authors call p_T , is the overlap between the trades the model would make and the trades that were actually there to be made, as a fraction of their union—a score of 1 means the model traded exactly the available opportunities, 0 means no overlap.

The values are sobering: between about 0.01 and 0.19 across stocks and horizons, and close to zero for small-tick stocks once a confidence threshold is applied, even where F1 is high. Predictability, by both F1-type and transaction-level measures, is higher for large-tick stocks—those whose average spread is below about 1.5 ticks—than for small-tick stocks, whose spread exceeds about 3 ticks. The authors conclude that “high forecasting power does not necessarily correspond to actionable trading signals”.

8.6 Representation versus architecture

The benchmark evidence suggests that the choice and treatment of inputs can matter as much as the network—a synthesis across studies, since none compares both systematically. Four decisions should be made, and reported, before capacity is increased.

Representation

raw levels, order flow, volume on a price grid, or individual messages (Lucchese et al., 2024; Kolm et al., 2023; Wu et al., 2021; Linna et al., 2025).

Normalisation

fixed statistics from the training set, rolling-window statistics, or a learned normalisation (Passalis et al., 2020); the statistics of LOB data drift over days.

Target

label construction, threshold relative to the spread, and horizon (Berti and Kasneci, 2025).

Clock

event time, a fixed number of book updates, or calendar time; and how events are time-stamped (Kolm and Westray, 2025).

Evaluation rule (strongest-baseline comparison). A complex architecture should be compared not only against another complex architecture, but against the strongest reasonable low-capacity model on the best available representation.

8.7 Prediction targets

Superficially similar targets define different statistical problems:

$$\begin{aligned} \mathbb{P}(\Delta m_{t,H} > 0 \mid X_t), \quad \mathbb{E}[\Delta m_{t,H} \mid X_t], \quad \mathbb{P}(\Delta m_{t,H} > \theta \mid X_t), \\ \mathbb{P}(T_{\text{move}} \leq H \mid X_t), \quad \mathbb{P}(\text{Fill} \mid X_t), \quad \mathbb{E}[\Pi \mid X_t], \end{aligned}$$

where $\theta > 0$ is a move threshold and $T_{\text{move}} = \inf\{u > 0 : |m_{t+u} - m_t| \geq \delta\}$ is the time until the mid first moves by one tick.

In words: in order: the probability that the mid goes up; the average size of the move; the probability that it goes up by more than θ ; the probability that it moves at all within H ; the probability that our resting order is filled within H ; and our expected profit. These are six different questions, and a model good at one need not be good at another.

They should not be compared as if interchangeable. Benchmark papers should report the target, horizon, sampling clock and intended execution interpretation.

8.8 Fill-probability models

Deep learning has also been applied to the execution problem directly. [Arroyo et al. \(2024\)](#) estimate the full distribution of the time until a limit order is filled, using a convolutional-Transformer encoder of recent market activity and a decoder that outputs a monotone survival curve. Cancellations are treated as censored observations, and the models are trained and compared by the right-censored log-likelihood, a proper scoring rule; the authors explain why the Brier score and concordance index are unsuitable here, because their assumptions fail for limit orders. In their study, queue position enters through the data design—hypothetical orders placed at the back of the queue—rather than as an input. KANFormer ([Zhong et al., 2025](#)) adds the order’s queue position and features of the agents placing orders as inputs, on CAC 40 index futures, and reports a large improvement in both the likelihood and discrimination measures over the earlier architecture, which it attributes to including agent-level information and queue position. The study covers one futures contract, but it is consistent with the claim of Section 12 that queue position is a first-class variable.

Model card: Fill-probability (survival) models

What you calibrate. A survival curve $\mathbb{P}(\tau_F > u \mid X_t)$ for a resting order: for deep models, the weights of an encoder and a decoder constrained to output a monotone curve.

How. Maximise the right-censored log-likelihood, treating cancelled orders as censored observations; queue position enters as an input or through the design of hypothetical orders.

What you do with it. Choose between a limit and a market order, set how long to wait before repricing, and weight expected profit by the probability of being filled.

How to picture it. A waiting-room clock for the order, which runs faster when the queue ahead is short and the other side is aggressive.

8.9 Generative and foundation models

Generators. A growing literature generates synthetic order flow. [Nagy et al. \(2023\)](#) tokenise LOB messages and train an autoregressive model that generates message streams, processed through a simulator; LOB-Bench ([Nagy et al., 2025](#)) standardises the evaluation of such generators. Conditional generative adversarial networks act as “world agents” that produce realistic, reactive order flow inside the ABIDES agent-based simulator ([Coletta et al., 2021, 2022](#); [Byrd et al., 2020](#)). Diffusion models—which learn to reverse a gradual noising process—generate future volume snapshots ([Wang and Ventre, 2025](#)), order flow conditioned on the book ([Berti et al., 2025](#)), and books conditioned on a chosen future regime for counterfactual analysis ([Wang and Ventre, 2026](#)). M3 ([Zhang et al., 2026](#)) trains an autoregressive model on a joint sequence of order events and book states for 800 Chinese stocks and about 32 billion events, reporting a scaling law across model sizes, reproduction of stylised facts, and a square-root impact law when synthetic execution programmes are injected. Neural extensions of queue-reactive models ([Bodor and Carlier, 2025](#)) reproduce square-root impact and cross-queue correlations on Bund futures.

How to judge a generator. A generator is useful for research only if it preserves the joint structure of orders, queues, executions and prices—not just marginal statistics such as the average spread. The evaluation criteria in this literature are: marginal distributions (spreads, depths, inter-event times); temporal dependence (clustering, long memory); cross-event dependence (how

cancellations follow trades); price dynamics (volatility, signature plots); liquidity dynamics (queue depletion and refill); responsiveness to injected orders (market impact); and downstream utility—whether a model trained on generated data performs well on real data (Nagy et al., 2025; Jain et al., 2024). Consistent with Principle 1, this survey treats generators as tools for simulation and stress testing and evaluates trading models only on observed data.

Model card: Generative models of order flow

What you calibrate. The weights of an autoregressive, adversarial or diffusion network that produces messages or book states.

How. Next-token likelihood for autoregressive models, an adversarial game against a discriminator for GANs, a denoising loss for diffusion models; then the generator is checked against the criteria above.

What you do with it. Stress tests, counterfactual scenarios and training environments; not as evidence that a trading model works.

How to picture it. A flight simulator for the order book: good for practice and rare scenarios, not a substitute for flying.

8.10 Foundation encoders and uncertainty

A *foundation model* is pretrained once on a large, generic corpus and then adapted to many specific tasks. Two recent contributions from the same group illustrate a useful layering for LOB data.

LOBERT. LOBERT is a pretrained, fine-tunable message-level encoder (Linna et al., 2025). Each message is one token (293 distinct message tokens), with its price, volume and time kept as continuous values alongside; the context is 512 messages. It is pretrained by *masked message modelling*—hiding messages and asking the model to fill them in, as BERT does with words—on Nasdaq ITCH data for AAPL, INTC, MSFT and FB from May to September 2015 (about 470 million messages, with millisecond time stamps). The next-message model has about 1.1 million parameters and predicts the complete next message correctly 27.8% of the time, against 6.1% for a state-space baseline of similar size. On mid-price direction the comparison with DeepLOB depends on how many forecasts are kept. At full coverage, 100 messages ahead, the two tie at a macro-F1 of 0.55. When each model forecasts only when its confidence exceeds 0.9, LOBERT reaches 0.88 on the 10% of cases it keeps, and DeepLOB 0.61 on the 28% it keeps—scores on different subsets, so not a like-for-like comparison. Inference is 53% slower than DeepLOB (about 282 predictions per second on one V100 GPU with batch size one, 47% of DeepLOB’s throughput). The authors list as limitations that realistic long generated sequences are not established, that the model may lose track of order size and location when price levels move, that it does not model individual order identifiers, and that the data’s millisecond resolution covers time only partly.

Uncertainty layers. UQ-LOB is an encoder-agnostic module that attaches to a pretrained LOB encoder and, in the spirit of attentive neural processes, conditions each forecast on a context set of recently completed windows whose outcomes are already known (Edward Manoharan et al., 2026); the paper evaluates it with one encoder, a deep temporal attention bilinear network, on Kraken Level-2 and Level-3 data for seven cryptocurrency pairs. Its regression variant outputs a Gaussian over the future tick displacement and its classification variant a distribution over down, up and stationary; both expose a scalar confidence. On 5.2 billion events across seven cryptocurrency assets it reports near-nominal 68% interval coverage.

For this survey the point is the change of target: from a direction to a direction *with a calibrated confidence*. That is precisely the input an execution policy needs, because the decision to place,

keep or cancel a passive order depends on how sure the model is, not only on which way it leans (Sections 1.4 and 11).

General time-series foundation models. A parallel line of work pretrains forecasters on very large collections of generic time series and applies them *zero-shot*, without training on the target series. Chronos (Ansari et al., 2024) scales each series by its mean absolute value, bins the values into 4096 tokens and trains a language-model architecture with a cross-entropy loss on about 84 billion observations. TimesFM (Das et al., 2024) is a 200-million-parameter decoder over patches of 32 time points, trained on about 100 billion points, most of them Wikipedia page views. Moirai (Woo et al., 2024) is a masked encoder that outputs a mixture distribution, trained on 27 billion observations of which 0.1% are economic or financial. Lag-Llama (Rasul et al., 2023) feeds a decoder with lagged values and outputs a Student- t distribution. None of these is trained or evaluated on order-book data.

Finance-specific versions change the corpus. Kronos (Shi et al., 2026) is pretrained on over 12 billion candlestick bars (open, high, low, close, volume) from 45 exchanges and reports large gains over general models in zero-shot price forecasting; its data are bars from five minutes to a day, not messages. TradeFM (Kawawa-Beaudan et al., 2026), a 524-million-parameter preprint from J.P. Morgan AI Research, tokenises individual trade-flow events (inter-arrival time, price depth, volume, side, add or cancel) from more than 9,000 US equities and generates order flow inside a deterministic matching simulator. Its rollouts reproduce heavy tails and volatility clustering, with a smaller distance to the observed 10-second return distribution than a compound Hawkes model (0.013 against 0.039), but the Hawkes model matches the distribution of spreads better. The evidence for general models on financial prediction is weak: on daily excess returns from 94 countries, Chronos and TimesFM used zero-shot have negative out-of-sample R^2 (−1.37% and −2.80%) and lose to gradient-boosted trees, and pretraining from scratch on financial data closes much of the gap (Rahimikia et al., 2025). We found no fully reported study of zero-shot general models on intraday or tick-level order-book data.

Model card: Foundation encoders and uncertainty layers

What you calibrate. An encoder pretrained on large volumes of message data; then a small task head—and, for UQ-LOB, an uncertainty module—fine-tuned on the target task.

How. Self-supervised pretraining (for example, predicting masked or next messages), followed by supervised fine-tuning; for the uncertainty layer, a context set of recently completed windows.

What you do with it. Start new tasks from a pretrained representation, forecast only when confident (as in LOBERT’s selective evaluation), and pass a calibrated confidence to an execution policy that can threshold it. General time-series foundation models serve as baselines rather than as LOB models.

How to picture it. A reader who has studied a library of order flow before being asked a specific question, and who also says how sure the answer is.

9 Hybrid models

Hybrid models combine statistical microstructure structure with learned representations. The motivation is that a purely learned model must rediscover from noisy data facts that are already known: that events excite further events, that queues deplete and prices move when they do, that price impact is roughly linear in order flow over short windows. Supplying this structure explicitly should make a model more sample-efficient, more stable outside its training regime, and easier to interpret. These are hypotheses; the published evidence for them in LOB data is still thin, and each

hybrid should be tested against its parts.

9.1 What the statistical layer supplies

The statistical models of the preceding sections provide low-dimensional, interpretable features that can be computed in real time:

- Hawkes intensities $\lambda_i(t)$ for each event type, updated in constant time per event;
- best-level, multi-level and integrated OFI, and cross-asset OFI (Section 4);
- queue imbalance, the micro-price adjustment $m_t^{\text{micro}} - m_t$, and book-shape features (Section 4.5);
- estimated depletion times τ^a, τ^b and the up-move probability of the queue race (Section 6);
- for execution models, the order’s own queue position Q_t^{ahead} .

9.2 Ways to combine the two

Concatenation. The simplest hybrid appends the statistical features to the network’s input, or projects them into the same embedding space as the learned features. It is easy to test by ablation (below), and is the form most hybrids in the literature take.

Hawkes-biased attention. Attention computes, for each event n , a weighted average over earlier events n' , with weights proportional to the exponential of a score. A Hawkes kernel can be built into that score:

$$\text{score}(n, n') = \frac{\langle \mathbf{z}_n^{\text{qry}}, \mathbf{z}_{n'}^{\text{key}} \rangle}{\sqrt{\text{dim}_{\text{emb}}}} + \ln \phi(t_n - t_{n'}), \quad t_{n'} \leq t_n, \quad (28)$$

where $\mathbf{z}_n^{\text{qry}}$ and $\mathbf{z}_{n'}^{\text{key}}$ are the query and key vectors of the two events and dim_{emb} their dimension, as in Section 8.

In words: because the softmax exponentiates the score, adding $\ln \phi$ *multiplies* each attention weight by the Hawkes kernel: recent events get more weight, and the weight decays with elapsed time exactly as excitation does in a Hawkes process. With an exponential kernel $\phi(u) = \alpha e^{-\omega u}$ the added term is $\ln \alpha - \omega(t_n - t_{n'})$, a penalty linear in elapsed time.

The network learns what to attend to; the kernel imposes how quickly influence fades. Transformer Hawkes processes (Zuo et al., 2020; Zhang et al., 2020) encode time differently, through learned temporal encodings; the explicit kernel bias above is a design we propose for testing rather than a published LOB result. A minimal implementation:

```
def hawkes_biased_attention(E, t, Wq, Wk, Wv, log_alpha, omega):
    # E: one embedding row per event; t: event times (sorted)
    Q, K, V = E @ Wq, E @ Wk, E @ Wv
    scores = Q @ K.T / sqrt(Q.shape[1])
    dt = t[:, None] - t[None, :]                # elapsed time from event n' to event n
    scores = scores + (log_alpha - omega * dt)    # log of the exponential kernel
    scores[dt < 0] = -inf                        # causal: no attention to later events
    return softmax(scores, axis=1) @ V
```

The added cost over standard attention is one subtraction per pair of events.

Intensity-conditioned state-space models. Selective state-space models such as Mamba (Gu and Dao, 2023) let the step size of their recurrence depend on the input, so that the model decides how quickly to forget. Making that step depend on the current event intensity or OFI would let the model’s memory shorten when the market is busy and lengthen when it is quiet—the same adaptation that event-time sampling achieves by construction. We know of no published LOB model of this kind.

Residual gating. A statistical model produces a baseline forecast $\hat{Y}_t^{\text{stat}}$, a network produces $\hat{Y}_t^{\text{deep}}$, and a gate $\nu_t \in [0, 1]$ —learned, or driven by volatility or intensity—mixes them:

$$\hat{Y}_t = \nu_t \hat{Y}_t^{\text{stat}} + (1 - \nu_t) \hat{Y}_t^{\text{deep}}. \quad (29)$$

In words: when the gate is near 1 the forecast falls back on the simple, well-understood model; the network contributes only where it has earned trust. Setting ν_t by volatility gives a natural safety layer: in unusual conditions, defer to the model with fewer parameters.

A related form trains the network on the residuals of the statistical model, so that it learns only what the simple model misses.

9.3 Published hybrids

Neural point processes for LOB events are the most developed hybrid. Shi and Cartlidge (2022) compute a separate intensity for each event type with parallel continuous-time LSTMs and add an interaction between events and book state; on three US stocks it outperforms classical and neural point-process baselines on likelihood and event prediction. Lalor and Swishchuk (2025) drive a twelve-event LOB simulator with an LSTM-based neural Hawkes process and use it as the environment for a reinforcement-learning market maker. The deep queue-reactive model of Bodor and Carlier (2025) keeps the queue-reactive structure of Huang et al. (2015) but parameterises the state-dependent event intensities with neural networks. The uncertainty module UQ-LOB (Edward Manoharan et al., 2026) is hybrid in a different sense: it attaches a probabilistic, attentive neural-process layer to a pretrained encoder.

Model card: Hybrid models

What you calibrate. The statistical component (for example Hawkes parameters or an OFI slope), the network, and the way they are joined: a kernel bias in attention, or a gate ν_t .

How. Fit the statistical part first, then train the network on its features or residuals; or train end to end with the statistical part as a constrained layer. Always alongside the ablation below.

What you do with it. Forecasts that need less data, behave more predictably outside the training regime, and fall back on the simple model when the gate says so.

How to picture it. A trainee working under a checklist: the network adds judgement, the statistical model supplies rules it should not have to rediscover.

Example 9.1 (The right ablation). Let S_t be a Hawkes intensity computed from the same events already supplied to a neural model $M_0(X_t)$. The experiment is $M_0(X_t)$ against $M_1(X_t, S_t)$ with identical splits, seeds and economic evaluation, reporting ΔV with a confidence interval. We argue, but do not test here, that $\Delta V \approx 0$ when S_t is a deterministic function of X_t and the network sees enough context to compute it. A hybrid earns its place when S_t brings information the network cannot reconstruct, or when it lets a much smaller network match a larger one.

10 Cross-asset information

The book of one instrument can carry information about related ones. A simple cross-impact form is

$$m_t^{(\kappa)} - m_{t-w}^{(\kappa)} = \sum_{\kappa'} \beta_{\kappa\kappa'} \text{OFI}_t^{(w,\kappa')} + \varepsilon_t^{(\kappa)}, \quad (30)$$

where the superscripts (κ) and (κ') index instruments, $m_t^{(\kappa)}$ is the mid of instrument κ , $\text{OFI}_t^{(w,\kappa')}$ is the order-flow imbalance of instrument κ' , $\beta_{\kappa\kappa'}$ is the cross-impact of instrument κ' 's flow on instrument κ 's mid, and $\varepsilon_t^{(\kappa)}$ is a residual.

In words: the same straight-line fit as before, except that each instrument's price change may respond to the buying pressure in *every* instrument. The coefficients form a matrix; the diagonal is each instrument's own impact and the off-diagonal entries are the cross effects.

The form covers index constituents, ETF and constituents, futures and cash, correlated equities, related fixed-income contracts and cross-venue cryptocurrency markets.

Example 10.1 (Coefficients multiply). For NQ,

$$m_t^{(\text{NQ})} - m_{t-w}^{(\text{NQ})} = \beta_{\text{NQ,NQ}} \text{OFI}_t^{(w,\text{NQ})} + \beta_{\text{NQ,ES}} \text{OFI}_t^{(w,\text{ES})} + \varepsilon_t^{(\text{NQ})}$$

has two coefficients. Ten related instruments, each predicted from all ten OFIs at five lags, gives $10 \times 10 \times 5 = 500$ coefficients. At a 5% level about 25 are “significant” under the null alone.

What the evidence says. On the 100 largest S&P 500 stocks, cross-asset order flow adds nothing to the *contemporaneous* fit once multi-level order flow is integrated, but lagged cross-asset order flow does improve one-minute-ahead return forecasts, with an effect that decays within minutes (Cont et al., 2023). Multi-asset attention models such as OF-MATNet attend along the asset axis directly and choose which peers to use by Granger-causality tests (Bandealineaeni et al., 2025). For futures, the natural cross-asset structure is economic rather than statistical: index futures and their constituents, related equity index futures (ES and NQ), and the Treasury futures curve.

Lead-lag between futures and cash. The clearest cross-asset structure in futures is that one market moves first. For the S&P 500 and Nasdaq-100, most price discovery happens in the E-mini futures rather than in the cash index or ETF (Hasbrouck, 2003). At high frequency the lead is measured in milliseconds and is bounded by the physical distance between data centres. Between Chicago, where CME matches, and the New Jersey equity venues, the response of SPY to E-mini price changes took 7.25–7.95 ms in April 2010, fell after a straighter fibre route was built, and was estimated at 4.2–5.2 ms over microwave links, against a speed-of-light minimum of 3.93 ms; about 20% of SPY trading could be identified as a direct response to E-mini moves (Laughlin et al., 2014). Measuring how often trades in two markets occur at a given time offset, Dobrev and Schaumburg (2017) find activity peaks at ± 5 ms between 10-year Treasury futures and cash Treasuries, with leadership running both ways, a clear lead of the E-mini over SPY at +5 ms, and a lead of the E-mini over cash Treasuries but not the reverse. The order-flow link between SPY and the E-mini is “positive, but short-lived”: beyond one second the correlations have little economic significance (Garrison et al., 2019).

Estimating the lag. Two markets trade at different, irregular times, so the usual correlation of returns sampled on a common grid is biased towards zero (the Epps effect). The Hayashi–Yoshida

estimator avoids the grid by summing products of returns over every pair of overlapping intervals; shifting one series by a candidate lag and choosing the lag that maximises the estimated correlation gives a consistent lead-lag estimator (Hoffmann et al., 2013). On European data, Huth and Abergel (2014) find with this method that the CAC 40 future leads a large constituent stock by about 0.6 seconds on average, and that the leader’s past alone predicts the direction of the lagger’s next mid-quote move about 60% of the time—yet a naive strategy that crosses the spread on that forecast does not make money.

Cross-asset Hawkes models. Point-process models express lead-lag as cross-excitation: events in one market raise the intensity of events in another. The two-asset model of Bacry et al. (2013) reproduces the Epps effect on Euro-Bund and Euro-Bobl futures, and lead-lag can be read off its second-order properties; Da Fonseca and Zaatour (2017) fit a multi-asset Hawkes model whose diffusive limit links high- and low-frequency correlation and which captures lead-lag on Eurex assets.

The Treasury curve. For interest-rate futures the related instruments are the points of the yield curve. On daily inter-dealer data from 1992 to 1999, order-flow imbalances explain up to 26% of day-to-day yield changes on days without major announcements, and most of that effect comes from a factor common to all maturities: net buying across the whole curve says more about the level of yields than buying in one maturity (Brandt and Kavajecz, 2004). At one-minute resolution, Tomas et al. (2022) compare cross-impact models on 10-year Treasury note futures and E-mini futures jointly and find that models built on the covariance of returns and order flow, such as a multivariate Kyle model, fit out of sample far better than a model with only direct (own-instrument) impact. We found no published intraday cross-impact study covering the full Treasury futures curve.

Cross-asset models should therefore report in-sample and out-of-sample improvement, incremental economic value, added latency, parameter count and stability across regimes.

Model card: Cross-asset models

What you calibrate. The cross-impact matrix $\beta_{\kappa\kappa'}$ and the lags at which it acts; for a cross-asset Hawkes model, the cross-excitation kernels between instruments.

How. Regression of each instrument’s price change on the order flow of all (with shrinkage or a factor structure to control the number of coefficients); lead-lag by maximising a Hayashi–Yoshida correlation over candidate lags; Hawkes cross-kernels by maximum likelihood.

What you do with it. Use the leader’s move to anticipate the lagger—mainly to protect resting orders in the lagging market from being picked off—and hedge across related contracts.

How to picture it. Thunder after lightning: the futures move is the flash and the cash market the sound, a few milliseconds later; the lag is set by distance.

Part III

From Prediction to Execution

About this part. Part II asked what the book predicts. Part III asks what a trader can do with a prediction once the mechanics of trading are included. It compares crossing the spread with waiting in the queue (Section 11); shows that an order’s place in the queue determines whether, and when, it is filled (Section 12); explains why the fills a passive order does get are biased towards bad ones (Section 13); measures how latency eats into a signal (Section 14); and states what a simulation must do to answer these questions honestly (Section 15).

11 Passive and aggressive execution

Consider one unit traded on side d at time t . For an aggressive order, which executes immediately, a simplified realised profit is

$$\Pi^{\text{agg}} = d \Delta m_{t,H} - C_{\text{spread}} - C_{\text{fees}} - C_{\text{impact}}, \quad (31)$$

where $C_{\text{spread}} = s_t/2$ is the half spread paid to cross, C_{fees} the exchange and clearing fees, and C_{impact} the additional price concession from walking the book.

In words: profit from trading immediately is the price move in our favour—multiplying by d flips the sign for a sell, so a falling price is good for a seller—minus what it cost to get in: half the spread, the fees, and any extra paid for consuming more than the best level.

A passive order placed at t trades only if it is filled, at time t_F and price p^{fill} . Define its post-fill mark-out over horizon h as

$$R_h^{\text{fill}} = d \left(m_{t_F+h} - p^{\text{fill}} \right). \quad (32)$$

In words: once our order has traded, look at the mid h seconds later and ask how far it is from the price we got, counted positive when it is in our favour. A large positive mark-out means we bought cheaply; a negative one means the market moved against us right after we traded.

For a passive buy filled at the bid, the mark-out is the half spread at the fill time, $s_{t_F}/2$, plus the mid move after the fill. The passive profit is

$$\Pi^{\text{pass}} = \mathbb{1}_{\text{Fill}} \left(R_h^{\text{fill}} + R_{\text{rebate}} \right), \quad (33)$$

where R_{rebate} is any liquidity-provision rebate net of fees.

In words: if the order is filled, the profit is its mark-out plus any rebate the exchange pays for providing liquidity; if it is never filled, the indicator is zero and so is the profit.

The passive profit exists only conditional on execution, which gives the decomposition

$$\mathbb{E}[\Pi^{\text{pass}}] = \mathbb{P}(\text{Fill}) \mathbb{E}[\Pi^{\text{pass}} \mid \text{Fill}]. \quad (34)$$

In words: the average profit of a passive order equals the chance it is filled times the average profit when it is. This is the identity $\mathbb{E}[Z \mathbb{1}_A] = \mathbb{P}(A) \mathbb{E}[Z \mid A]$ of Section 1.3, with A the fill.

Equation (34) separates two quantities that a directional forecast conflates: how likely the order is to trade, and how good the trade is when it does. The rest of this part is about the two factors. Market-making models (Avellaneda and Stoikov, 2008; Cartea et al., 2015) and optimal-execution models (Almgren and Chriss, 2001) give the surrounding control problems; here we focus on the inputs those problems need from the book.

11.1 Signal value and execution value

The decomposition gives tradeability a precise meaning. A directional model estimates the *signal value*

$$V^{\text{sig}}(X_t) = \mathbb{E}[\Delta m_{t,H} \mid X_t], \quad (35)$$

the expected price move given what is known. A passive execution decision is worth its *execution value*

$$V^{\text{exec}}(X_t, a_t) = \mathbb{P}(\text{Fill} \mid X_t, a_t) \mathbb{E}[\Pi^{\text{pass}} \mid \text{Fill}, X_t, a_t], \quad (36)$$

which depends on the action through where the order is placed and therefore on its queue position. *In words:* signal value asks “which way will the price go?”; execution value asks “if I act on that view with this order, how likely am I to trade, and how good will the trade be when I do?” A model can be excellent at the first and useless at the second.

The two can even point in opposite directions. A forecast of a falling price is most confident when the book is already one-sided—a thin bid, heavy selling. That is exactly when a resting buy order is most likely to be filled and least likely to be worth having: the fill and the adverse move arrive together (Section 6). Conditioning on the fill reverses the apparent value of the signal. This is why the survey treats $\mathbb{P}(\text{Fill} \mid \cdot)$ and $\mathbb{E}[\cdot \mid \text{Fill}]$ as separate modelling targets, and why evaluation must use the conditional quantities rather than unconditional accuracy.

Who earns the execution value in practice is documented. A large high-frequency market maker studied by [Menkveld \(2013\)](#) traded passively in about four of every five trades, earning the spread while losing on its inventory; across high-frequency firms, differences in relative speed explain large differences in trading performance ([Baron et al., 2019](#)).

12 Queue position and fill probability

A passive order is filled only when everything ahead of it in its queue has been traded or cancelled. This section shows why that makes queue position a variable in its own right: two orders at the same price, with the same forecast behind them, can have very different chances of being filled.

12.1 Residual queue position

For a resting order, the displayed size at its price does not determine its fill probability. Let Q_t^{ahead} be the volume ahead of the order at the same price. Under price–time priority the order cannot execute until that volume has been removed by executions or cancellations. Two orders at the same price can therefore have very different values, and queue position has been modelled explicitly as a source of value ([Moallemi and Yuan, 2017](#)).

For a passive order with q ahead of it, the probability of a fill within horizon H is $\mathbb{P}(\text{Fill} \mid Q_t^{\text{ahead}} = q, X_t)$. It must be distinguished from $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$. A model can predict direction excellently and still have poor trading value if the predicted moves happen mainly when the passive order is unlikely to execute.

Example 12.1 (Same forecast, different orders). Two resting bids at the same price have 20 and 480 lots ahead of them. Net depletion runs at 40 lots per second. The naive time to reach the front is 0.5 s for the first and 12 s for the second. Both carry the same $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$. Over a five-second horizon the first almost surely fills and the second almost surely does not.

13 Adverse selection

Being filled is not always good news. A passive order tends to be filled exactly when someone better informed wants to trade against it, so the fills it gets are, on average, followed by price moves against it. This section defines that cost precisely and shows how it depends on queue position.

13.1 Conditioning on the fill

Using the mark-out R_h^{fill} of (32), define the conditional adverse-selection component for an order that had q ahead of it when placed as

$$\text{AS}(q, h) = -\mathbb{E} \left[R_h^{\text{fill}} - \frac{s_{t_F}}{2} \mid \text{Fill}, Q_t^{\text{ahead}} = q \right], \quad (37)$$

the expected mid move against the order after it is filled, excluding the half spread it earns.

In words: among orders that started with q lots ahead of them *and were filled*, how far, on average, did the price then move against us? It is a form of the winner’s curse: in an auction, the times your low bid wins are disproportionately the times when others knew something you did not. A passive order is most likely to be filled just when an informed trader wants to trade the other way.

The conditioning on the fill is essential. Without it, the quantity measures the unconditional drift of the market rather than the information in the event that the order actually traded. Classical models make the mechanism explicit: a liquidity provider who trades with a better-informed counterparty loses on average (Glosten and Milgrom, 1985; Kyle, 1985).

13.2 Queue position as an information variable

Queue position affects both the probability of execution and the information content of execution. Moallemi and Yuan (2017) model both effects and find that a position at the front of the queue is always worth more than one at the back: in their model adverse selection increases with queue position, because an order at the end of a long queue tends to be filled only by a large aggressive order, which is more likely to be informed. For some large-tick US stocks they estimate the front–back value difference to be comparable to the bid–ask spread. The direction of the effect is therefore supported by a model; its size is an empirical object that can depend on asset, tick regime, spread, order-flow state, horizon and cancellation behaviour, and execution-aware benchmarks should report it directly (Section 36).

Example 13.1 (The decomposition at two positions). All values in ticks; fill window $H = 10$ s, mark-out horizon $h = 10$ s, one-tick spread, no rebate. Here “ $| q$ ” abbreviates $| Q_t^{\text{ahead}} = q$, and $\mathbb{E}[\Pi^{\text{pass}} | \text{Fill}, q] = s_{t_F}/2 - \text{AS}(q, h)$:

Position	$\mathbb{P}(\text{Fill} q)$	$s_{t_F}/2$	$-\text{AS}(q, h)$	$\mathbb{E}[\Pi^{\text{pass}} \text{Fill}, q]$	$\mathbb{E}[\Pi^{\text{pass}} q]$
Front	0.80	+0.5	−0.20	+0.30	+0.24
Back	0.25	+0.5	−0.45	+0.05	+0.0125

The table shows the arithmetic of (34); the ordering of the two rows matches the direction found by Moallemi and Yuan (2017), with stylised magnitudes. A metric that looked only at direction or only at fill rate would rank the two positions without seeing the product.

14 Latency and signal decay

A prediction at horizon H is useful only if its information survives until the decision is executed. Write the total latency as

$$T_{\text{lat}} = T_{\text{signal}} + T_{\text{decision}} + T_{\text{network}} + T_{\text{exchange}}, \quad (38)$$

the times taken to compute the signal, to decide on an action, to transmit the order, and for the exchange to process it.

In words: total delay from seeing the data to the order being live at the exchange is the sum of the stage delays, just as the latency of a processor pipeline is the sum of its stages.

A signal whose value decays materially before T_{lat} is not economically equivalent to one of the same accuracy with longer persistence. The relevant object is not $\mathbb{E}[\Delta m_{t,H} \mid X_t]$ but

$$\mathbb{E}[\Pi \mid X_t, T_{\text{lat}}, \text{execution policy}], \quad (39)$$

the expected profit given the information, the latency, and the rule that maps forecasts to orders.

In words: the quantity that matters is not “will the price go up?” but “how much do we expect to make, given what we knew, how slow we are, and what we do with the forecast?”

Competition on latency is itself an economic phenomenon (Budish et al., 2015; Hasbrouck and Saar, 2013).

Example 14.1 (Half-life against latency). Let a signal’s value decay exponentially with a half-life of 2 ms, so a fraction $2^{-T_{\text{lat}}/2\text{ms}}$ of it remains after a delay T_{lat} . At $100\text{ }\mu\text{s}$ total latency $2^{-0.05} \approx 97\%$ of its value remains; at 5 ms only $2^{-2.5} \approx 18\%$ does. The same forecast accuracy yields very different economics.

15 Replay and deterministic simulation

The questions of this part—will the order fill, and is the fill a good one—cannot be answered by a backtest that assumes orders fill when the price touches them. This section shows how such a backtest misleads and lists what a simulation must model to give trustworthy answers; Part IV describes a system built to those requirements.

15.1 What a naive backtest assumes

A common backtest of a passive signal assumes that a limit order placed at the bid is filled when the signal fires, or when the price touches the order’s level:

naive: signal at $t \rightarrow$ assume filled at bid $\rightarrow$ P&L = $m(t+H) - \text{bid}$

This fills every order, including those the market would never have reached. The orders it wrongly fills are exactly the ones where the price moved away before the queue was consumed—the fills a real passive order does not get, and which would have been profitable. The naive backtest is therefore biased toward adverse-selection-free fills that do not exist.

15.2 What a queue-exact replay requires

A replay that can answer the execution questions of this part needs:

1. market-by-order data, so that queue position is defined at all;
2. an order-level reconstructed book in which the simulated order occupies a place in the first-in-first-out queue;
3. explicit, independent latencies for outbound orders, outbound cancels and inbound market data;
4. an explicit fill rule stated in terms of recorded events;
5. a stated assumption about market impact.

```
replay:  signal at t -> order leaves at t
         -> arrives at t + feed delay + order delay
         -> appended at the tail of the real FIFO queue at its price
         -> filled only when recorded executions reach an order behind it
         -> P&L from the actual (simulated) fill time and price
```

The fill rule rests on price–time priority: if a recorded aggressor reached a resting order that arrived later than the simulated order at the same price, it must have passed through the simulated order first. The standard simplifying assumption is *no market impact*: the recorded market is replayed as it happened, as though the simulated order had not been there. Every cost measured this way is a cost before the market’s reaction to the order.

Replay and generative simulation answer different questions. Generative simulators produce new market trajectories and are judged on how realistic they are; a replay simulator keeps the recorded market fixed and produces only the counterfactual outcome of the researcher’s own orders. GPU simulators such as JAX-LOB (Frey et al., 2023) move the matching engine onto accelerators to speed up reinforcement-learning training; the concern here is not throughput but the fidelity of the fill inference. Part IV describes one implementation of a queue-exact replay.

Part IV

Case Study: The Kaspar HFT Architecture

About this part. Part III listed what a faithful simulation of execution requires. Part IV shows one system built to meet those requirements, as a worked case study rather than an empirical result. It describes the overall architecture (Section 16); how simulated orders are placed into, and filled from, a reconstructed queue of real orders (Section 17); how recorded exchange data are turned into replayable input (Section 18); why a replay gives the same answer every time (Section 19); how the same code runs in replay, paper and live trading (Section 20); a simple execution algorithm that serves as a model-free baseline (Section 21); and how latency is measured (Section 22).

16 Architecture overview

This part describes one system that implements the chain (3) from market data to simulated or live execution: Kaspar, a trading and simulation system for CME futures (ES, NQ and treasury futures) built on a C++20 actor framework (Maciejewski, 2026b). It is included as a case study of the methodology argued for in Part III, not as an empirical contribution; every statement below is a description of the design and code, and the few numbers quoted come from previously published measurements, attributed where they appear.

The data path is

CME MDP 3.0 multicast → socket reader → message buffer → decoder → feed handler → order book → strategy → simulated order manager → [exchange gateway | simulated fill].

Every component is an actor: it owns its state and communicates only by messages. Actors can be placed in a *group*, which runs several actors on one thread, processing one message at a time in arrival order. The simulated order manager (SOM) sits between the strategy and the venue. In live mode it forwards orders to the exchange gateway (CME iLink 3); in simulation mode it sends them into the reconstructed book, where they are matched against recorded order flow.

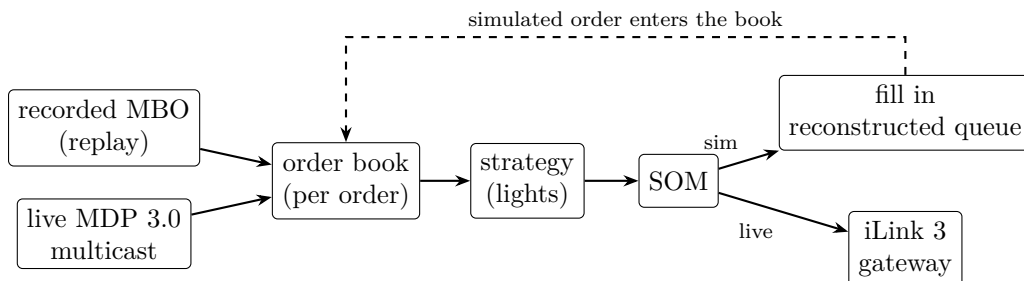


Figure 2: Kaspar data and order paths. Everything downstream of the market-data source is the same code in replay and live operation; the branch between simulated and live execution is a single decision in the simulated order manager.

17 Queue-exact simulated fills

The central requirement of Section 15 is that a simulated order should be filled only if a real order in its place would have been. This section explains how Kaspar meets it: simulated orders sit in the same per-order queue as real ones, reach the book after a modelled delay, and are filled only by real executions that reach them. Here and throughout, *queue-exact* means exact with respect to the displayed order sequence carried by the input MBO feed and the documented matching rules; it does not imply knowledge of hidden liquidity or unobserved participants, and it does not model the market impact of the simulated orders.

17.1 The book holds every order

The simulator reconstructs a full market-by-order book from recorded exchange data. Each price level is a queue that holds every resting order individually, in arrival order, and simulated orders are held in the same queue alongside real ones. There is no estimated “volume ahead” variable: a simulated order’s queue position is its position in the list. This is the property that makes a fill inference definable at all (Maciejewski, 2026d).

17.2 Placement and latency

In simulation mode the SOM turns a new order into a book *add* message tagged with a simulated venue identifier and sends it to the order-book actor. The book does not insert it immediately. It holds the order on a delay queue until its modelled arrival time at the matching engine:

```

auto order_engine_arrive_time =
    order_leave_time
    + uint64_t(feed_delay) * 1000
    + uint64_t(std::max(40, eff_delay)) * 1000; // 40 us floor

```

The outbound order delay, the outbound cancel delay and the inbound feed delay are independent parameters. The feed delay is added because the strategy saw the market late by that amount, so a round trip is feed plus order, not order alone. When the order arrives it is appended at the tail of the queue at its price, behind all depth already resting there.

17.3 The fill rule

A recorded cancellation of a real order simply unlinks it, and every order behind it moves up. A recorded execution triggers simulated fills only when the executing real order is the first real order

in the queue and a simulated order sits ahead of it:

```
// fill sim orders only if the executing order is at the front
if (o == q->get_head_no_sim())
{
    fill_prev_sim_order();
    ...
}
```

Any remaining executed quantity is carried forward to earlier simulated orders. This is the price-time-priority argument of Section 15 in code: an aggressor that reached a real order which arrived after the simulated order must have passed through the simulated order. Simulated orders are excluded from the best bid and offer that the strategy sees, so a strategy cannot react to its own quote. Market impact is not modelled (Maciejewski, 2026d).

Example 17.1 (A fill, step by step). Bid queue at 5000.00 over seven successive steps #0–#6, sizes in lots, real orders R1–R4 and one simulated order S:

#0	resting:	R1(10) R2(5) R3(20)	
#1	S(3) arrives:	R1(10) R2(5) R3(20) S(3)	appended at tail
#2	R2 cancels:	R1(10) R3(20) S(3)	S moves up
#3	exec 10 vs R1:	R3(20) S(3)	no sim order ahead of R1: no fill
#4	exec 20 vs R3:	S(3)	S is behind R3: no fill
#5	R4(8) joins:	S(3) R4(8)	
#6	exec 5 vs R4:	R4 is head real, S ahead	-> S filled 3 @ 5000.00

At step #6 the aggressor reached R4, which arrived after S, so it must have passed through S. A “touch” backtest would have filled S at step #3, three recorded events earlier and before the queue ahead of it was consumed.

18 The replay pipeline

A traditional backtester reads bars or trades from a file and applies a fill rule to a strategy’s orders. A queue-exact replay must instead reproduce the market data exactly as the trading system would have received it, so that the same code path builds the same book. Kaspar supports three market-data sources, each feeding the same downstream code:

Source	Reader	Clock
Live CME MDP 3.0 multicast	socket reader	real time
Packet capture (.pcap) of the same multicast groups	PCAP reader	capture timestamps
Decoded order-by-order record file	binary file reader, directly into the books	file timestamps

The capture reader parses each packet down to the exchange payload, optionally strips a hardware-timestamp trailer added by capture equipment, and hands the payload to the same message buffer the live socket reader uses, with both the software and hardware timestamps. It reads one packet per message to itself, so replay is an ordinary actor loop rather than a blocking file read. Two limits are stated in the project documentation: capture replay performs no packet recovery or A/B feed arbitration, so a gap in the capture propagates to the book; and the decoded-record path bypasses decoding entirely and is the fast option for repeated backtests. In the distributed version,

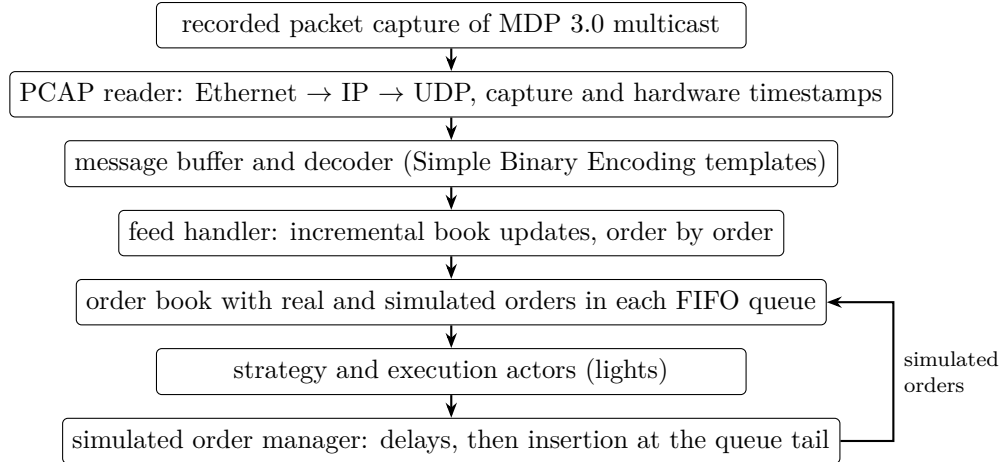


Figure 3: The packet-level replay path. Every stage below the reader is the same code that runs on live data; only the reader and the final branch of the order manager differ.

the capture and record readers are complete as libraries but not yet selectable from the trading binary’s command line; the separate simulator program drives replay from decoded record files.

The order book then holds every order individually (Section 17), so a simulated order’s queue position is exact, and fills are decided by the recorded executions that follow.

19 Determinism

In the replay configuration the file reader, the order book, the timer, the SOM, the strategy actors and the position manager all run in one actor group on one thread. The reader is added to the group last, so all consumers exist before the first market-data message is delivered. Three design choices then make a replay repeatable:

1. **One message sequence.** A single thread processes every message—market data, orders, acknowledgements, fills, timer events—in one total order.
2. **Market time.** The timer actor is driven by market-data timestamps, not the wall clock, so timeouts fire at the same point in the data on every run.
3. **Seeded randomness.** Any randomised strategy decision uses a per-actor seeded generator set in configuration.

The property this implies, and the test that would check it, is that replaying the same recorded file twice with the same configuration yields byte-identical fill records. We argue the property from the design; we do not report the test here.

Determinism addresses a common weakness of trading research: the research implementation and the production implementation are often different programs, and differences between them are discovered in production.

20 Replay, paper and live equivalence

Two choices are independent: where market data come from (recorded file or live multicast), and where orders go (reconstructed book or exchange). Everything downstream of the message buffer—the decoder, feed handler, order book, strategy and SOM—is the same code whichever source feeds

it. The branch between simulated and live execution is a single condition in the SOM’s new-order handler; both paths apply the same pre-trade reject checks, with one additional rule in live mode for cancels of orders not yet acknowledged by the exchange. The combination “live data, simulated execution” is paper trading: orders enter the live reconstructed book and are filled by the live order flow under the same rule as in replay.

The live order-entry path implements CME’s iLink 3 protocol: binary message encoding, HMAC request signing, sequence management, and failover between primary and secondary gateways. The project documentation states that both the market-data handler and the order-entry session have passed CME’s automated certification. Strategies can be written as in-process actors in C++ or, through an in-process foreign-function interface, in Rust.

Two qualifications keep the claim precise. Switching between simulated and live execution is a single flag in the SOM, but in the distributed build that flag is set in the program source rather than in a configuration file, and the distributed configuration runs in simulation mode; the equivalence demonstrated day to day is therefore between replay and paper trading, with the live path sharing all code up to the SOM branch. And equivalence of code is not equivalence of outcomes: replay assumes no market impact (Section 17) and models latency with configured delays, so a strategy that is profitable in replay can still behave differently live.

21 A model-free execution benchmark

SHADOW-PPOV is the passive execution algorithm used in the case study (Maciejewski, 2026d). It makes no prediction about book evolution. When a third-party limit order is added within a configured distance of the touch, the algorithm may place its own order at the same price, records that participant’s exchange order identifier, and cancels when that order leaves the book. The placement decision is randomised at a configured rate; cancellation can be delayed by a grace period measured in events or time. The design comment for the grace period states the trade-off of Section 13 directly: cancel too quickly and fills are given up; cancel too slowly and the order bears the adverse selection.

Its role in this survey is as a null model for passive placement. It defines the bottom rung of a hierarchy:

model-free placement $\rightarrow$ predictive placement $\rightarrow$ full predictive execution policy.

A predictive model that claims value for passive execution should beat the model-free benchmark *in the same replay, under the same fill rule*, on slippage and on post-fill mark-outs. Any improvement measured against a naive backtest instead is confounded by the fill-inference bias of Section 15.

22 Measurement

Latency claims need their endpoints. In the published measurement of Kaspar’s market-data handler, the start point is a software timestamp at the socket read and the end point is the order book being published; time spent in the network card and kernel before the read is outside the measurement, which is therefore socket-to-book, not wire-to-book. Measured on live CME multicast, the median is $8.0\ \mu\text{s}$, with book-message medians of 7.1 , 7.3 and $7.1\ \mu\text{s}$ for ES, NQ and ZN and 99th percentiles of 18.5 , 13.6 and $57.0\ \mu\text{s}$ (Maciejewski, 2026c). These numbers are quoted as an example of stating endpoints, not as a result of this paper.

The simulator’s slippage instrumentation measures execution cost for each fill against four references: the mid at decision time, the interval volume-weighted average price, same-side passive

fills by other participants, and the touch. Comparing against the interval VWAP separates market drift from adverse selection. Post-fill mark-outs at fixed times and after a fixed number of book events provide the mark-out R_h^{fill} of (32) (the event-count version is its analogue on an event clock) and the adverse-selection quantity of Section 13.

The order-flow signals of Part II—OFI, queue imbalance, micro-price, fill-probability models—are not part of the system described here. They are the candidate inputs that Part V proposes to search over, using this replay as the evaluator.

Part V

LLM-Assisted Model Discovery

About this part. Language models can now write code, propose features and suggest model structures. Part V asks how that ability fits into the research process of the previous parts without corrupting it. It first separates three different uses of generative models in LOB research (Section 23); then describes LAMD, a loop in which a language model proposes candidate models and a deterministic replay judges them (Section 24); explains why a proposal from a language model is not evidence (Section 25); distinguishes searching a fixed space from inventing explanations (Section 26); and shows how the number of candidates tried raises the bar any winner must clear (Section 27).

23 Three uses of generative models, kept apart

Generative and language-model technology now enters LOB research in at least three distinct ways, and they should not be confused.

LOB generation produces market trajectories; the generated messages can be processed by a book simulator to produce synthetic books. Its value lies in simulation, stress testing and data augmentation, and its difficulty lies in preserving the joint structure of orders, queues, executions and prices rather than only their marginal statistics. It is a legitimate research area but not part of the methodology proposed here: consistent with the first principle of Section 1, candidate models are evaluated only on observed market data.

A fourth use is now large enough to have its own surveys: the language model as a *trading agent*. Ding et al. (2024) review LLM agents for financial trading—their architectures, data inputs, backtest performance and open challenges—and Dong et al. (2025) survey LLM agents across five financial domains (data analysis, investment research, trading, investment management and risk management), cataloguing over 30 financial benchmarks and 20 representative models and the obstacles to deployment. Hua et al. (2026) review *agentic* quantitative trading across five stages—factor mining, signal discovery, portfolio construction, order execution and risk management—and find that current systems concentrate on signal discovery, that integration with execution and risk control is still uncommon, and that strong model or forecasting capability “does not reliably translate into trading performance under live market conditions”. That literature asks how a language model can act in markets. This part asks a narrower question: how a language model can take part in quantitative hypothesis generation without becoming either the source of the market data or the source of the evidence that validates a hypothesis.

LOB modelling is the subject of Part II. A pretrained encoder that can also predict the next message, such as LOBERT, is used there as a model of the market, not as a source of data.

Model generation is different in kind. The language model does not produce market data or market forecasts. It produces *hypotheses about how to trade*, which are then evaluated against the recorded market.

24 The lamd loop

LLM-Assisted Model Discovery (LAMD)¹ is a research loop in which a language model proposes candidate models and every other step is external to it and reproducible: given the same data, configuration and random seeds, it gives the same result. It belongs to a broader family of methods in which a language model is paired with an automated evaluator to search a space of programs (Romera-Paredes et al., 2024). The closest published application in finance is Kou et al. (2025), in which prompted language models generate executable alpha-factor candidates, agents filter them by market state and predictive quality, and the surviving factors are combined with dynamically optimised weights; the authors report a 53.17% cumulative return on the SSE 50 from January 2023 to January 2024, ahead of their benchmarks. The difference from LAMD lies less in the generation step than in what surrounds it: LAMD makes the number of candidates tried part of the evidence (Section 27) and, as proposed here for order-book strategies, evaluates each candidate’s execution in queue-exact replay on recorded order-level data; the existing implementation (Mayeski, 2026c) works on daily data. The components are:

1. **Vocabulary.** A human-specified set of primitives the proposals may use: OFI at several windows, queue imbalance, trade flow, Hawkes intensities, returns and volatility, queue position, regime indicators, cross-asset inputs, and execution actions (place, keep, cancel, cross).
2. **Proposal.** The language model proposes or mutates a candidate: a feature combination, a transformation, a decision rule or a model structure, expressed in the vocabulary.
3. **Compilation.** A deterministic compiler turns the proposal into an executable model with explicit free parameters. Proposals that do not compile are rejected without evaluation.
4. **Fitting.** Parameters are fitted on a training window only.
5. **Replay.** The fitted model drives an execution policy in a queue-exact replay of recorded data (Part IV), against the model-free benchmark of Section 21.
6. **Validation.** Walk-forward out-of-sample evaluation with purging and embargo (Section 30).
7. **Certification.** The out-of-sample result is compared against a null that accounts for every candidate evaluated so far (Section 27).
8. **Archive.** Every candidate—prompt, compiled model, fit, split, result—is recorded, whether it passed or failed, and the archive seeds the next round of proposals. The running count of candidates evaluated is N_{trials} .

Example 24.1 (One hypothetical iteration, three candidates). In the table, $\text{OFI}_t^{(5)}$ is $\text{OFI}_t^{(w)}$ with $w = 5$ s; $\lambda_{\text{sell}}(t)$ is the component $\lambda_i(t)$ of a fitted Hawkes model for which i is the sell-market-order type, and $\bar{\lambda}_{\text{sell}}$ the corresponding component of $\bar{\lambda}$; ξ and c are thresholds fitted on the training window.

¹An earlier description of the same loop used the name LLM-Assisted Trading Strategy Search (LATSS) (Mayeski, 2026c).

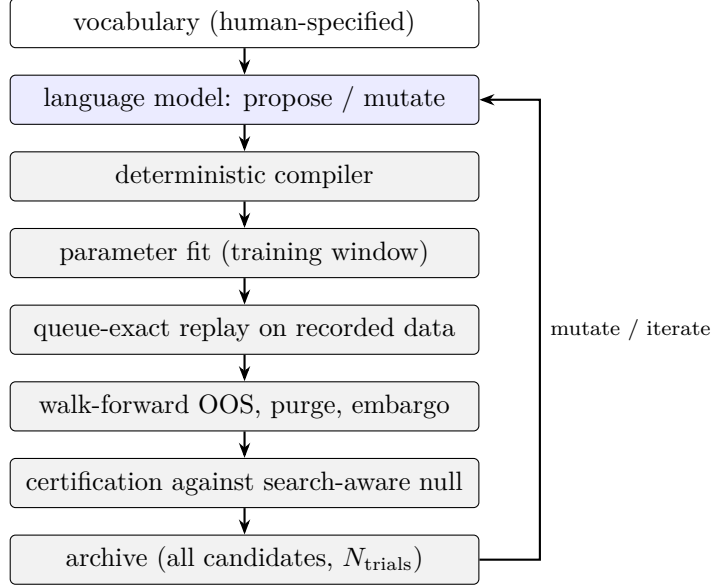


Figure 4: The LAMD loop. Only the shaded blue step involves the language model; every step that touches data or produces evidence is a fixed, reproducible procedure.

id	proposal (paraphrased)	compiled form	OOS net, ticks/fill	N_{trials}
c1	keep a passive bid only while short-horizon OFI agrees	cancel if $\text{OFI}_t^{(5)} < \xi$	+0.04	1
c2	interact OFI with queue imbalance	logit on $\text{OFI}_t^{(5)} \times \text{QI}_t$	+0.01	2
c3	cancel faster when sell-order intensity spikes	cancel if $\lambda_{\text{sell}}(t) > c \bar{\lambda}_{\text{sell}}$	-0.02	3

Each result is measured relative to the model-free benchmark in the same replay. Candidate c1 is not “discovered” when it tops this table; it is discovered only if it clears a null adjusted to the total number of candidates the loop has evaluated, which after a few days of automated search may be in the tens of thousands.

24.1 A worked implementation

The loop has been implemented for daily-frequency strategies in the LATSS system described in [Mayeski \(2026c\)](#), and the details show which choices matter.

- **The language model writes structure, never sees prices.** It composes transformations from a fixed set of fifteen tools (moving averages, rate of change, z-scores, realised volatility, breakouts, percentile ranks, crossovers and similar). Prices reach the code only when the engine runs it. Everything the system can discover is some arrangement of those tools.
- **The model declares parameters; an optimiser sets them.** Every window and threshold is a declared “knob” with a range; differential evolution chooses its value on training data.
- **Validation is fold-based.** Data are split by quarter; candidates are fitted on some quarters and scored on the held-out rest, and the median is kept. The author notes two weaknesses: long look-back windows leak a little information across fold boundaries, and random quarters resample a single regime, which missed a later regime change.
- **The skill bar is empirical.** Each candidate’s positions are circularly shifted in time, which

keeps their exposure and turnover but scrambles their timing; the best score achieved by these skill-free shifted versions is the bar a candidate must clear.

The author’s own assessment is the right one for this survey: with a generic vocabulary the results are generic, and the human decisions—what to look for, which tools to supply, how hard to test, what to trust—remain the binding constraints. For LOB models the vocabulary would be the primitives of Part II, and the evaluator the queue-exact replay of Part IV.

25 Hypothesis generation is not validation

A candidate proposed by a language model has no stronger evidentiary status than one proposed by a person. Its plausibility, its fluency, and the quality of the explanation that accompanies it are not evidence that it captures market structure. The scientific content of LAMD is in the deterministic steps around the proposal, not in the proposal.

The same caution applies to language models used directly on market data. [Chen and Glasserman \(2026\)](#) train a language model on synthetic LOB event sequences and find that it generates valid sequences almost perfectly, yet its implicit world model fails to learn the state of the book, which leads to biased estimates and spurious predictability when it is used to forecast events. Syntactic validity is not evidence of a learned mechanism, and the same holds for plausible-looking trading rules. Only external evaluation on held-out observed data can establish that a candidate works.

26 Search versus abduction

A distinction from the philosophy of science clarifies what such a system can and cannot do. *Search* explores a given space of hypotheses: “here is a space of possible solutions; try things”. *Abduction* creates a hypothesis that was not in the space: “something about reality does not make sense; what new concept would make sense of it?” ([Mayeski, 2026a](#)). Language models are effective at search and recombination: program-search systems built on them have found new mathematical constructions ([Romera-Paredes et al., 2024](#)), and LAMD finds new arrangements of supplied tools. [Zahavy \(2026\)](#) argues that current language models handle induction and, increasingly, deduction, but lack a mechanism for the abductive step from experience to new principles.

For order-book research the implication is concrete. A LAMD-style system can search exhaustively over combinations of known primitives—OFI at several windows, queue imbalance, Hawkes intensities, queue position, state modulators, targets and horizons—far faster than a person. A system restricted to such a vocabulary cannot discover a primitive outside it: a new state variable, an unrecognised interaction between hidden and visible liquidity, or a new relation between depletion and adverse selection. A less constrained system could, in principle, search over representations as well as over combinations of supplied primitives; whether current language models can do so reliably is an empirical question. Whether that limitation is permanent is an open question; that it holds for present systems is the working assumption of this survey ([Mayeski, 2026b](#)). Automated search is an amplifier of human-specified structure, and its outputs must pass the same certification as any human proposal.

27 Search complexity and multiple testing

The more candidates a search evaluates, the larger the best in-sample result it will find by chance. For N_{trials} independent candidates with no skill, the expected maximum of their standardised Sharpe-

ratio estimates grows roughly like $\sqrt{2 \ln N_{\text{trials}}}$. (The Sharpe ratio is average return divided by the standard deviation of return—profit per unit of risk.) In words: toss a thousand fair coins a hundred times each, and the luckiest coin will look like a skilled one; the more coins, the luckier the best one looks, though the growth is slow. The deflated Sharpe ratio (Bailey and López de Prado, 2014) makes the correction precise and accounts for non-normal returns (Section 31). Related tools include the reality check for data snooping (White, 2000), the probability of backtest overfitting (Bailey et al., 2017), and raised significance thresholds for factor discovery (Harvey et al., 2016).

Example 27.1 (The same Sharpe, different evidence). Take the standard error of an annualised Sharpe estimate as approximately $1/\sqrt{T_{\text{yr}}}$ for T_{yr} years of data, so the null maximum is about $\sqrt{2 \ln N_{\text{trials}}}/\sqrt{T_{\text{yr}}}$:

N_{trials}	null maximum, 1 year	null maximum, 4 years
1	0	0
100	≈ 3.0	≈ 1.5
50 000	≈ 4.65	≈ 2.3

A Sharpe ratio of 2 from a single pre-specified test on four years is evidence. The same Sharpe ratio as the best of 50 000 automatically generated variants on four years is below what noise alone is expected to produce.

Automated proposal therefore changes the evaluation problem from “performance and model complexity” to

$$\boxed{\text{performance} + \text{model complexity} + \text{search complexity}} \quad (40)$$

In words: to judge a discovered model you need three things, not one: how well it did, how complicated it is, and how many other candidates were tried before it was found.

The archive’s trial count is as much a part of the result as the out-of-sample Sharpe ratio. Faster replay makes this worse, not better: an order-of-magnitude increase in simulation speed is an order-of-magnitude increase in the number of hypotheses that can be tested, and the skill threshold must rise with it.

The central research question for automated model discovery is thus not whether an AI system can generate a profitable-looking model, but whether an automated search can discover economically significant structure while controlling the false-discovery rate that the search itself induces.

Part VI

Evaluation

About this part. Every earlier part ends with the same question: *is the model any good?* Part VI answers it. Evaluation in this field fails in predictable ways—a metric that rewards the wrong thing, a test set that leaks information from the training set, or a winner chosen from thousands of tries that only looks good by chance—and each section addresses one of them. In order:

- which *statistical* scores measure forecast quality honestly, and why accuracy and F1 are not enough (Section 28);
- how to measure *economic* value—profit after costs, fills and adverse selection—which is what a forecast is ultimately for (Section 29);
- how to split time-ordered data so that the test really is unseen (Section 30);

- why more complex models need more evidence, and how to correct for having tried many models (Section 31);
- how to choose between models of different complexity, which turns Occam’s razor (Principle 3) into a procedure (Section 32);
- and what a benchmark result must report for others to compare and reproduce it (Section 33).

The thread through all six is the distinction of Part I: a forecast is judged first as a forecast, then as a trading input, and the second judgement is the one that matters.

28 Statistical metrics

The right statistical metric depends on the target (Table 15). These metrics measure predictive quality; none of them, by itself, measures trading value.

Proper scoring rules. A scoring rule is *proper* if a forecaster minimises the expected score by reporting their true beliefs (Gneiting and Raftery, 2007). Log loss, the Brier score and the CRPS are proper; accuracy and F1 are not, because they reward only the side of a threshold a forecast falls on. For any model whose output will be used as a probability—for sizing, for the confidence gating of Section 1.4, or for the fill-probability term of execution value—a proper score should be the primary statistical metric, and calibration should be checked directly (events forecast at 70% should happen about 70% of the time). For fill-time models, censoring by cancellations makes some familiar survival metrics improper; the right-censored log-likelihood is the safe choice (Arroyo et al., 2024).

29 Economic metrics

Economic evaluation starts from the aggressive and passive returns of Section 11 and the decomposition $\mathbb{E}[\Pi^{\text{pass}}] = \mathbb{P}(\text{Fill}) \mathbb{E}[\Pi^{\text{pass}} \mid \text{Fill}]$, which separates execution probability from conditional execution quality. For passive strategies both factors should be reported, broken down by queue position where the data allow.

Table 16 orders evaluation metrics by how close they come to trading value. A study should say which levels it reaches; most LOB papers stop at the first or second.

Cost-aware directional metrics, which ask whether a predicted trade would have produced a favourable executable outcome rather than whether the mid moved in the predicted direction, are an improvement over raw accuracy. They are not a complete economic metric: they omit market impact, capacity, inventory, repeated trading, the distribution of latency, and queue position in detail. In Example 13.1, a metric that ignored $\mathbb{P}(\text{Fill})$ would misjudge the two positions. Such metrics should be reported alongside realised trading metrics, not instead of them.

30 Temporal validation

Cross-validation with randomly assigned folds is inappropriate for most LOB problems because observations are dependent in time. A chronological split preserves the direction of information flow, and walk-forward evaluation repeats it. Where features and labels overlap in time, training samples whose information overlaps the test period must be purged and an embargo imposed after each test window (López de Prado, 2018). The embargo should follow from the maximum look-back and the forecast horizon, not be chosen arbitrarily.

Example 30.1 (Sizing an embargo). Features use a 60-second look-back and labels a 10-second horizon. The embargo at each train/test boundary is at least 70 seconds. A walk-forward schedule might train on 20 sessions, validate on 5 and test on 5, then roll forward by 5.

When several models are compared on the same data, a *model confidence set* (Hansen et al., 2011) identifies the subset that cannot be statistically distinguished from the best, at a stated confidence level; it is the honest alternative to declaring the model with the highest score the winner. Beyond a single test period, models should be stress-tested across volatility regimes, tick and fee changes, and assets of different tick size, since predictability differs systematically between large- and small-tick instruments (Briola et al., 2025a; Gould and Bonart, 2016).

LOB research has large implicit search spaces: feature definitions, look-backs, horizons, normalisations, thresholds, architectures, hyper-parameters, assets, sessions and execution assumptions. When thousands of configurations are tried, the best one looks good by chance. Model complexity and research-process complexity must be considered together (Section 27).

31 Capacity, generalisation and multiple testing

A model that fits the training data well can still fail on new data, and the more flexible the model, or the more models were tried, the more likely that is. This section gives the theory behind that statement in three steps: why simpler model families generalise more reliably, how to correct a performance measure for the number of models tried, and why simple models are also easier to test honestly.

31.1 Why simpler models often generalise better

Statistical learning theory makes the intuition behind Occam’s razor precise. For a classifier M chosen from a family of models with *VC dimension* VC —roughly, the largest number of points the family can label in every possible way—and trained on N_s independent examples, with probability at least $1 - \varsigma$ the error rate on new data satisfies (Vapnik, 1998)

$$\text{err}(M) \leq \widehat{\text{err}}_{N_s}(M) + \sqrt{\frac{VC (\ln(2N_s/VC) + 1) - \ln(\varsigma/4)}{N_s}}. \quad (41)$$

In words: error on new data is at most the training error plus a penalty that grows with the richness of the model family and shrinks with the amount of data. A richer family can fit more patterns, including patterns that are only noise, and needs more data before its training error can be trusted.

Rademacher complexity (Bartlett and Mendelson, 2002), which measures how well a model family can fit random noise on the data actually observed, gives bounds of the same form that are never much worse than the VC bounds and can be tighter.

Two features of LOB data make the penalty bite. The signal is weak relative to the noise, so a family that can fit noise will. And observations are strongly dependent in time, so the *effective* number of independent examples is far smaller than the number of events: a day of ES data has millions of messages but far fewer independent price moves.

Two caveats keep the argument honest. Modern deep networks can fit even random labels perfectly and still generalise on real labels (Zhang et al., 2017), and test error can fall again as capacity grows past the point where the training data are fitted exactly (“double descent”) (Belkin et al., 2019). Classical capacity bounds therefore do not decide by themselves whether a deep model will generalise on LOB data. What decides it is out-of-sample evidence—which is why the benchmark studies of Section 8.5 carry more weight than any bound.

Practical penalties. Information criteria trade fit against parameter count explicitly: the Akaike criterion adds twice the number of parameters to minus twice the log-likelihood (Akaike, 1974), and the Bayesian criterion adds the number of parameters times the logarithm of the sample size (Schwarz, 1978), penalising complexity more heavily in large samples. The lasso adds a penalty on the sum of absolute coefficients, which drives unhelpful coefficients exactly to zero and so selects features automatically (Tibshirani, 1996). All three are applied to a single fit; none accounts for the search that produced the model family in the first place, which is the subject of the next subsection.

31.2 Multiple testing and the deflated Sharpe ratio

If N_{trials} strategies with no skill are tested, and their estimated Sharpe ratios have standard deviation σ_{SR} across trials, the expected best estimate is approximately (Bailey and López de Prado, 2014)

$$\mathbb{E}[\max \widehat{\text{SR}}] \approx \sigma_{\text{SR}} \left[(1 - \gamma_E) \Phi^{-1} \left(1 - \frac{1}{N_{\text{trials}}} \right) + \gamma_E \Phi^{-1} \left(1 - \frac{1}{N_{\text{trials}} e} \right) \right], \quad (42)$$

where Φ^{-1} is the inverse of the standard normal distribution function, $\gamma_E \approx 0.5772$ is the Euler–Mascheroni constant and e is Euler’s number.

In words: this is the Sharpe ratio that the luckiest of N_{trials} worthless strategies is expected to show. It grows with the number of trials, roughly like $\sqrt{2 \ln N_{\text{trials}}}$; the example of Section 27 used that rough form.

Write this expected maximum as SR_0 . The *deflated Sharpe ratio* is the probability that the selected strategy’s true Sharpe ratio exceeds SR_0 , given its estimate $\widehat{\text{SR}}$ from N_{obs} returns:

$$\text{DSR} = \Phi \left(\frac{(\widehat{\text{SR}} - \text{SR}_0) \sqrt{N_{\text{obs}} - 1}}{\sqrt{1 - \widehat{\text{skew}} \widehat{\text{SR}} + \frac{\widehat{\text{kurt}} - 1}{4} \widehat{\text{SR}}^2}} \right), \quad (43)$$

where $\widehat{\text{skew}}$ and $\widehat{\text{kurt}}$ are the sample skewness and (raw, not excess) kurtosis of the returns and the Sharpe ratios are per period, not annualised.

In words: the numerator asks how far the observed Sharpe ratio is above what luck alone would produce after this many trials; the denominator widens the uncertainty when returns are skewed or fat-tailed. A DSR near 1 means the result is unlikely to be luck; near 0.5 or below, it is indistinguishable from the best of a batch of noise.

The same authors turn this into a budget. For a given length of history there is a maximum number of configurations that can be tried before a strategy with no skill is expected to show an impressive in-sample Sharpe ratio; with five years of data, “no more than forty-five independent model configurations should be tried” if one wants to avoid an expected in-sample Sharpe ratio of 1 from a strategy whose out-of-sample expectation is zero (Bailey et al., 2014). The probability of backtest overfitting (Bailey et al., 2017), White’s reality check (White, 2000), raised significance thresholds (Harvey et al., 2016) and false-discovery-rate control (Benjamini and Hochberg, 1995) address the same problem from different directions.

31.3 Backtestability

These tools favour simple models for a reason beyond generalisation: a simple model family is *backtestable*. When the family is small, the number of effective trials can be counted or bounded, the null maximum can be computed, and the path from signal to decision can be audited. As the family grows—deep networks with many hyper-parameters, richly parameterised point processes, or

automated search over model structures—the effective number of trials becomes hard to state, and with it the bar a result must clear. Parsimony is therefore a property of the research process as well as of the model.

32 Occam’s razor as a model-selection principle

This section makes Principle 3 operational. We do not propose a single scalar penalty for complexity. With $V(M)$ as in Section 1.2, let

$$\mathcal{C}(M) = (\mathcal{C}_{\text{param}}(M), \mathcal{C}_{\text{lat}}(M), \mathcal{C}_{\text{fit}}(M), \mathcal{C}_{\text{mon}}(M), N_{\text{trials}}(M)) \quad (44)$$

collect the dimensions of complexity: the number of fitted parameters, the inference latency, the effort to fit and refit its parameters, the effort to monitor in production, and the number of candidates $N_{\text{trials}}(M)$ searched to find the model (Section 24).

In words: complexity is a list of separate costs, not one number: how many knobs the model has, how slow it is, how much work it is to tune and to watch, and how hard someone searched to find it.

A complex model M_1 should be preferred to a baseline M_0 only if $\Delta V = V(M_1) - V(M_0)$ is statistically credible and economically meaningful relative to $\Delta \mathcal{C} = \mathcal{C}(M_1) - \mathcal{C}(M_0)$. Because the components of $\mathcal{C}$ share no common unit, the framework asks for them to be reported separately (Table 17), so the reader can judge whether the added complexity is justified.

An explicit exchange rate. When a decision must be made, the separate dimensions can be combined by stating, in advance, how much economic value each unit of added complexity must buy. Writing $\Delta \mathcal{C}$ as a weighted sum of its components with weights chosen by the user, the rule “adopt M_1 only if $\Delta V > \chi \Delta \mathcal{C}$ ” for a stated $\chi > 0$ makes the trade-off explicit and auditable. The value of the rule lies in stating the weights before the comparison, not in any particular choice of them.

Where to start. Table 18 orders candidate approaches by increasing complexity, with the condition under which each is the natural starting point. It is a guide to where the burden of proof lies, not a ranking of methods: a more complex row is justified when the simpler one above it has been tried, on the same data and fill model, and found wanting.

A parsimony checklist. The contrast between an over-engineered and a parsimonious approach can be stated dimension by dimension (Table 19). The right-hand column is a default, to be departed from with evidence.

33 Benchmarking standard and datasets

A scientifically useful LOB benchmark should report at least the following.

Data

exchange; instrument; date range; number of sessions; L1, L2 or L3; event count; tick size; spread; depth.

Prediction

target; horizon; sampling clock; feature look-back; normalisation.

Model

architecture; parameter count; training procedure; hyper-parameter search; computational budget.

Validation

chronological split; walk-forward procedure; purging; embargo; number of trials.

Execution

latency; fees; spread; queue position; fill model; market-impact assumption.

Results

statistical metric; economic metric; confidence interval; null benchmark; robustness across regimes.

Public datasets and frameworks each have limits (Table 20). No single benchmark represents “the LOB”; the strongest evidence survives across datasets, assets and regimes.

A queue-exact replay can supply fields that a snapshot backtest cannot—queue position at entry, the fill rule, the three latencies, the impact assumption—and in doing so makes the *Execution* block of the checklist concrete rather than nominal.

Part VII

Hardware, Failure Modes, and Outlook

About this part. The last part steps back from individual models. It asks what the computing hardware allows, and why faster hardware cannot rescue a signal that decays too quickly (Section 34); how models fail in live markets—regime changes, manipulation, hidden liquidity and data errors (Section 35); which open problems are most worth working on (Section 36); and what the survey as a whole concludes (Section 37).

34 Hardware acceleration and the infrastructure constraint

Hardware interacts directly with both organising principles. The economic value of a signal depends on the latency and the determinism of the infrastructure that evaluates it, and richer models tend to raise both latency and jitter:

$$\text{model complexity} \uparrow \Rightarrow \text{inference latency} \uparrow \Rightarrow \text{signal value at execution} \downarrow . \quad (45)$$

In words: a bigger model takes longer to evaluate, and by the time it answers, some of what it predicted has already happened.

34.1 Determinism as a risk-control property

General-purpose processors, even with kernel-bypass networking, introduce jitter from caches, branch prediction, interrupts and scheduling. A strategy that is profitable at its average latency can lose money if latency spikes during the volatile episodes when its signal matters most. Deterministic, fixed-cycle execution is therefore a risk-control property, not only a speed optimisation.

34.2 Book-building latency versus inference latency

Two pipelines are often conflated. *Book building*—parsing market data and maintaining the order book—is protocol-bound and highly regular. *Inference*—evaluating a model on the book—depends on the model. Field-programmable gate arrays have long been used for the first: feed handling and book maintenance in hardware are well documented (Leber et al., 2011; Lockwood et al., 2012). Placing models on the same fabric is more recent. Tool flows for fast neural-network inference on FPGAs (Duarte et al., 2018) make small, quantised networks feasible with very low and fixed latency, but model capacity is bounded by the fabric. Vendor-audited benchmarks of FPGA trigger paths now report minimum trigger latencies on the order of ten nanoseconds (STAC, 2024), and hardware-aware sequence models designed for FPGA execution from the outset, rather than ported after training, have been proposed for cryptocurrency tick data (Popov and Huber, 2026). The two numbers measure different pipelines—a trigger versus a learned model—and should not be compared directly.

34.3 The commercial FPGA landscape

Most production hardware for trading is commercial, and most of its published figures are vendor claims rather than audited measurements. Table 21 lists representative offerings with the figures their makers publish; the one independently audited result is the STAC-T0 benchmark of an Exegy framework on an AMD card (STAC, 2024). Two cautions apply when reading such numbers. They measure different things—a transceiver loopback, a trigger, a book build, a model inference—that must not be compared directly. And the AMD transceiver figure explicitly excludes protocol processing and programmable logic, so it is a lower bound on a component, not a tick-to-trade time.

34.4 Between FPGA and GPU

A class of accelerators aims at the gap between FPGA determinism and GPU flexibility. Light-Trader (Yoo et al., 2023), from KAIST and the chip designer Rebellions, combines an FPGA for market data and orders with dedicated AI processors and a scheduler that manages workload and power; back-tested on CME data, it reports speed-ups of about 14 times over a GPU-based and 7 times over an FPGA-based system for AI inference. Corsair (Srivastava et al., 2025) is an in-memory-computing chiplet design for large-language-model inference; it is not a trading product, but it illustrates the direction of the hardware industry. Network cards with programmable logic carry inference into the network path itself (Napatech, 2025).

34.5 Transformers on FPGAs

Mapping attention to FPGAs is an active research topic outside finance. FAMOUS (Kabir et al., 2024) accelerates dense multi-head attention on a data-centre FPGA and reports about 2.6 times the throughput of a V100 GPU; ELiTeFormer (Agostinelli et al., 2026) co-designs a ternary-weight Transformer with linear attention for an adaptive SoC. Neither is a trading system. For LOB models the implication is that small, quantised attention models can be placed on programmable logic, at latencies of microseconds rather than nanoseconds, while full spatial-temporal Transformers remain GPU workloads. Quantisation—integer rather than floating-point arithmetic—is the enabling technique (Jacob et al., 2018).

34.6 GPUs: throughput for research

Graphics processors excel at data-parallel workloads: training, large-scale simulation, and models whose memory footprint or operator mix exceeds practical FPGA budgets. JAX-LOB (Frey et al., 2023) moves LOB matching onto accelerators for reinforcement-learning training. Monte Carlo simulation of order-book dynamics parallelises well across paths; one vendor demonstration reports speed-ups of 14 to 114 times over a CPU (Bennett, 2025), though for simulated paths rather than a historical matching-engine backtest. Specialised processors also accelerate training: a multi-horizon DeepLOB variant trained in about 15% of the GPU time on intelligence processing units (Zhang and Zohren, 2021). For single-event, latency-critical inference, batch-oriented accelerators are at a disadvantage; for signals whose useful life is measured in milliseconds or longer they are adequate.

Faster research hardware has the effect noted in Section 27: it increases the number of hypotheses that can be tested, and so the certification threshold must scale with it.

34.7 Networks and distance

Latency is not only computation. The operating system’s network stack adds microseconds; *kernel-bypass* libraries deliver packets directly to user space (DPDK Project, n.d.). Published figures for such stacks include half-round-trip times of about 3 μ s on older 10-gigabit hardware (Solarflare Communications and Arista Networks) and about 1 μ s for UDP (Mellanox Technologies, 2017). Physical distance sets a floor no software can remove: light in optical fibre takes about 5 μ s per kilometre (about 4.9 ns per metre) (Corning, 2013; ITU-T, 2003), which is why trading systems are colocated in the exchange’s data centre and why differences in relative speed translate into differences in trading performance (Baron et al., 2019; Budish et al., 2015). At the extreme, application-specific chips fix a design in silicon, trading the reprogrammability of an FPGA for speed and power (Kuon and Rose, 2007).

34.8 Workload placement

The guiding rule is Occam’s razor (Principle 3) again: place a computation on the most constrained, and therefore most deterministic, hardware that still delivers the required economic performance. Moving a model to faster hardware is justified only when the reduction in latency and jitter measurably improves post-cost performance.

34.9 Hardware cannot extend signal lifetime

Simpler models have a structural advantage on deterministic hardware: fewer parameters and simpler data flow map onto fixed-point, branch-free pipelines. Acceleration can rescue part of a complex model’s value by removing engineering latency, but it cannot extend the statistical lifetime of the signal.

Example 34.1 (Second-order latency). A logistic score on five OFI windows and queue imbalance takes a few tens of multiply–add operations. If the signal’s half-life is 5 ms, evaluating it in 50 μ s rather than 500 ns keeps $2^{-0.01} \approx 99.3\%$ of its value rather than essentially all of it. The difference is second-order; the binding constraint is the quality and horizon of the signal itself.

The design sequence that follows is: identify the horizon and value of the signal under idealised execution; choose the simplest model that captures that value; only then choose the hardware tier whose latency and jitter preserve it after costs.

34.10 Open questions in hardware

- Training objectives that penalise expected inference latency and resource use on a target device, so that models trade accuracy against deployability explicitly.
- Which components of spatial-temporal LOB models survive mapping to programmable logic without losing their economic value.
- Hawkes intensities in hardware: exponential kernels update in constant time and suit pipelines, and a power-law kernel can be approximated by a few exponentials, each with its own running sum (Bochud and Challet, 2007); how many are needed at the precision of fixed-point hardware, and how state-dependent kernels map to pipelines, are open.
- How certification thresholds should scale when faster simulation multiplies the number of candidates that can be tested.
- Whether the statistical properties of a strategy developed on one hardware and latency profile survive deployment on another—the research–production gap of Part IV, at the hardware layer.
- Public, auditable benchmarks that cover book building and inference together, rather than isolated kernels or vendor-defined paths.

35 Failure modes

Models are fitted to the past and run in the future. This section lists the ways that gap breaks them in practice: the market changes regime, liquidity disappears in stress, other participants place orders designed to mislead, displayed size hides real size, and the data themselves are wrong. For each it states what goes wrong and, where one exists, how it can be detected or guarded against.

Regime change and state invalidation. A model fitted in one liquidity regime can fail after volatility shocks, tick-size or fee changes, market-structure changes, scheduled announcements or liquidity withdrawal. A signal can be statistically valid for the regime in which it was estimated and economically stale after a transition.

Volatility and liquidity withdrawal. Queue-based signals rely on the book looking roughly the same when a decision is executed as when the signal was computed. In a volatility burst, best-level depth can vanish and the intensities that drive depletion change on the same time scale as the queues themselves, so first-passage estimates become unreliable. A simple defence is to reduce size or disable microstructure signals when short-term realised volatility is unusually high; it should be compared against more elaborate regime models before those are adopted.

Spoofing and layering. Displayed liquidity is not necessarily committed liquidity. Manipulative orders do not falsify OFI—OFI still measures observed changes—but they alter the mapping from displayed flow to future executable prices, and they distort queue-imbalance, depth and depletion estimates.

Documented cases. Manipulation of the order book is not hypothetical. On the Korea Exchange, Lee et al. (2013) document traders placing large orders away from the market to create imbalance and withdrawing them. A trader in the E-mini S&P 500 was charged with layering over five years, conduct the CFTC alleged contributed to the conditions of the May 2010 flash crash (Commodity

[Futures Trading Commission, 2015](#)). In 2020 JPMorgan paid \$920 million over spoofing in precious-metals and US Treasury futures over at least eight years ([Commodity Futures Trading Commission, 2020](#)). In a spoofing episode in Ultra Treasury bond futures, [Debie et al. \(2023\)](#) measure ask-side liquidity costs falling from about 10.4 to about 8 basis points when the spoof order was placed and recovering after it was cancelled—a displayed improvement in liquidity that was not real. [Cartea et al. \(2020\)](#) model spoofing as an optimal-control problem in which the spoofer trades the gain from skewing the order imbalance against the expected regulatory penalty. Quote stuffing—bursts of order messages—affected over 74% of US-listed securities at least once in 2010, with lower liquidity and higher short-term volatility during episodes ([Egginton et al., 2016](#)).

Hidden liquidity. Iceberg orders make executable liquidity exceed displayed liquidity, so a model trained on displayed volume misestimates depletion, impact and fill probability. In E-mini S&P 500 futures, one detection study estimates that about 9% of book volume is iceberg orders ([Christensen and Woodmansey, 2013](#)); when participants detect an iceberg they trade against it actively, and icebergs create an adverse-selection cost for other limit orders ([Frey and Sandås, 2017](#)). This is one reason message-level and trade data are valuable even for book-based models.

Mitigations include down-weighting orders that are cancelled within a short time, features built from cancellation rates and from the ratio of trades to orders, and detection of iceberg refills. They reduce but do not remove the problem; robustness to each form of contamination belongs in a serious evaluation.

Feed and timestamp errors. At high frequency, data quality is part of the model: timestamp synchronisation, packet loss, sequence gaps, book reconstruction, exchange-specific event semantics, stale snapshots, duplicates, and clock differences between market-data and execution systems.

Example 35.1 (A sequence gap). On a feed with sequence numbers, a missed packet leaves the reconstructed book wrong until the handler recovers from a snapshot. A model trained on rows from that interval learns reconstruction error, not market behaviour. No number of parameters compensates for it; the gap has to be detected and the interval excluded or repaired.

36 Research agenda

The survey has identified several places where the literature stops short of what a trader needs. This section collects them as open research questions; Part VI gives the standard against which answers to them should be evaluated.

Execution-aware objectives. Most prediction models stop at an estimate of $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$. The more useful objective is an estimate of $\mathbb{E}[\Pi \mid a_t, X_t]$, the expected profit of each available action, combining prediction, fill probability, queue position and adverse selection. Calibrated confidence ([Edward Manoharan et al., 2026](#)) is a step toward it: it tells an execution policy when to act.

Queue-aware benchmarks. Queue position should be a first-class state variable. Benchmarks should report performance conditional on queue rank, residual volume, spread and expected depletion time, and should measure the size of the queue-rank effect on fill probability and on conditional mark-outs across assets and regimes, rather than take a single model estimate as given.

Message-level foundation models for execution. Pretrained message-level encoders have so far been evaluated mainly on mid-price and next-message prediction (Linna et al., 2025). Fine-tuning them for fill probability and conditional mark-outs, evaluated in queue-exact replay, is an open direction.

Cross-asset information after costs. When does cross-asset information remain valuable after costs, latency, multiple testing and parameter instability?

Robustness to structural change. Models should be tested across volatility regimes, tick and fee changes, market-structure changes and extreme events.

Automated search as a statistical process. Automated and LLM-assisted search should be evaluated as a search: with complete trial accounting, search-aware nulls, and a reported false-discovery rate. A public, LOB-specific skill-bar benchmark—data with null features and with signals of known strength injected—would let competing search systems be compared on how often they find what is not there and how often they miss what is.

End-to-end signal and execution. Most models output a forecast and leave execution to a separate rule. Models whose output is the decision itself—quote, keep, cancel, cross—and whose training objective is execution value would close the gap that this survey describes. Reinforcement learning in LOB simulators (Lalor and Swishchuk, 2025; Frey et al., 2023) is one route, ideally validated in queue-exact replay; its results must be judged against model-free benchmarks in the same simulator.

Causal rather than predictive questions. The questions that matter for execution are causal: what happens to the queue and the price *if* an order is placed here? Replay with a no-impact assumption answers this only for small orders. Models of the market’s response to one’s own orders, and ways to validate them without live experiments, remain open.

State-space and long-context models for event streams. State-space sequence models are well suited to long, irregular event streams, yet apart from one hardware-oriented design (Popov and Huber, 2026) we found no established LOB forecaster built on them. Testing them against the baselines of Table 18—not against each other—is a clear next step.

Near-criticality and learned models. Estimated branching ratios of order flow are high, from about 0.7–0.8 to about one depending on method. How deep sequence models behave on near-critical, long-memory inputs, and whether Hawkes structure built into them (Section 9) improves robustness, is untested.

Contamination-robust features. Features that remain informative in the presence of spoofing and hidden liquidity—based on order lifetimes, cancellation behaviour and refill patterns—and evaluation protocols that test robustness explicitly.

Generative evaluation. Generators should be evaluated by market invariants and downstream utility rather than by visual or marginal similarity alone (Nagy et al., 2025).

A parsimony audit. For each widely cited deep LOB model, re-estimate its incremental economic value and deflated performance against the best low-capacity baseline on the same data and fill model. Such a meta-study would directly test the central claim of this survey.

Correlated signals and systemic risk. If many participants deploy similar order-book signals, their trades become correlated, and so do their withdrawals of liquidity under stress. The market-wide consequences of widely shared LOB alphas are largely unstudied.

37 Conclusion

The literature on limit-order-book modelling has moved from low-dimensional queue and order-flow models to expressive sequence architectures and pretrained message-level encoders. Statistical prediction has improved. Statistical prediction and executable trading remain different problems. The translation between them is

LOB state $\rightarrow$ representation $\rightarrow$ prediction $\rightarrow$ decision $\rightarrow$ queue $\rightarrow$ execution $\rightarrow$ P&L

and each link can destroy information or add uncertainty.

Order-flow imbalance, queue imbalance, Hawkes intensities and first-passage models give interpretable low-dimensional descriptions of market dynamics. Deep and pretrained models give representational capacity and, increasingly, calibrated confidence. Hybrid models offer a route to combine them. The conclusion is not that simple models are superior. It is that complexity is an empirical hypothesis: a more complex model is justified when it delivers incremental out-of-sample economic value that survives realistic execution, temporal validation, and the multiple-testing burden of its own selection.

Two tools make that standard testable. A deterministic, queue-exact replay turns the execution questions into measurements on observed data. A disciplined model-discovery loop, in which a language model proposes and deterministic machinery disposes, widens the search without weakening the evidence—provided the trial count is part of the result. Automation raises the rate at which hypotheses are generated. It does not remove the statistical problem of deciding whether a discovered pattern is real.

For a reader who must choose a model, the survey’s guidance reduces to an order of operations. Decide first what the output is for—aggressive trading, passive execution, market making or simulation—and therefore which of the targets of Section 8 matters. Establish the strongest simple baseline on the best available representation: order-flow imbalance, queue imbalance, the micro-price, the queue race, and for passive orders the volume ahead. Evaluate it with proper scores and economic metrics in a queue-exact replay, counting every configuration tried. Only then add capacity—a small network, a sequence model, a hybrid—and keep it only if the increment survives the same test (Table 18).

A useful future literature should therefore report not only whether a model predicts the market, but what information it extracts, what decision it changes, how that decision is executed, and whether the incremental value survives a properly constructed null.

Table 6: Classical order-book signals at a glance. Horizons are indicative of how the signals are used in the cited literature, not guarantees.

Signal	What it measures	Typical horizon	How it is fitted	Known failure modes
OFI, best level (Cont et al., 2014)	net buying pressure at the touch	seconds to minutes (contemporaneous fit)	linear regression; slope re-estimated with depth	weak in thin, noisy names; distorted by spoofing
Multi-level / integrated OFI (Xu et al., 2018; Cont et al., 2023)	pressure across the first levels	same	regression; principal components	more inputs to monitor; depth beyond the touch is less committed
Queue imbalance (Gould and Bonart, 2016)	skew of the two best queues	next mid change	logistic regression	weak for small-tick names; misled by hidden size
Weighted mid	price leaning toward the thin side	next mid change	none	noisy; not a martingale
Micro-price (Stoikov, 2018)	expected future mid given imbalance and spread	a few mid changes	Markov chain on imbalance and spread buckets	assumes Markov dynamics; needs symmetrised data
Book slope, curvature	how liquidity is spread across levels	longer	descriptive	sensitive to transient deep orders
Resilience (Large, 2007)	speed of refill after trades	seconds	Hawkes impulse response	varies strongly across regimes

Table 7: Estimated Cont–Stoikov–Talreja parameters for Sky Perfect Communications, Tokyo Stock Exchange, 2006 (Cont et al., 2010). Rates are per minute, in units of the average limit-order size.

Distance from opposite quote $\bar{\ell}$	1	2	3	4	5
Limit-order rate $\hat{r}_L(\bar{\ell})$	1.85	1.51	1.09	0.88	0.77
Per-order cancellation rate $\hat{r}_c(\bar{\ell})$	0.71	0.81	0.68	0.56	0.47
Market-order rate $\hat{r}_M = 0.94$; power-law fit $\hat{r}_0 = 1.92$, $\hat{\Xi} = 0.52$					

Table 8: Probability that the next mid move is up, one-tick spread, Sky Perfect Communications: empirical frequency / Cont–Stoikov–Talreja model (Laplace-transform method) (Cont et al., 2010). Queue sizes in units of the average limit order.

Orders at the best bid, $v_{1,t}^b$	Orders at the best ask, $v_{1,t}^a$				
	1	2	3	4	5
1	0.512 / 0.500	0.304 / 0.336	0.263 / 0.259	0.242 / 0.216	0.226 / 0.188
2	0.691 / 0.664	0.502 / 0.500	0.444 / 0.407	0.376 / 0.348	0.359 / 0.307
3	0.757 / 0.741	0.601 / 0.593	0.533 / 0.500	0.472 / 0.437	0.409 / 0.391
4	0.806 / 0.784	0.672 / 0.652	0.580 / 0.563	0.529 / 0.500	0.484 / 0.452
5	0.822 / 0.812	0.731 / 0.693	0.640 / 0.609	0.714 / 0.548	0.606 / 0.500

Table 9: Probability that an order placed at the best bid is executed before the mid moves, one-tick spread, Cont–Stoikov–Talreja model (Laplace-transform method) (Cont et al., 2010).

Orders at the best bid, $v_{1,t}^b$	Orders at the best ask, $v_{1,t}^a$				
	1	2	3	4	5
1	0.497	0.641	0.709	0.749	0.776
2	0.302	0.449	0.535	0.591	0.631
3	0.206	0.336	0.422	0.483	0.528
4	0.152	0.263	0.344	0.404	0.452
5	0.118	0.213	0.287	0.346	0.393

Table 10: Model cards for the Hawkes variants.

Variant	What you calibrate	How	What you do with it	How to picture it
Sum of exponentials	jumps per (pair of types, decay); decays fixed on a geometric grid	convex likelihood once decays are fixed	long memory with constant-time updates, in software or hardware	a power law drawn with a few rulers of different lengths
Nonlinear / inhibitory	signed kernels; the link function	maximum likelihood (poorly convex)	represent events that discourage others	a thermostat as well as an amplifier
Marked (volume classes)	one set of kernels per size class	as for a multivariate model, one type per class	let large trades matter more, with their own time profile	a big stone makes bigger, longer ripples
Markov-modulated	per-regime background and decay; switching rates	EM over hidden regimes	separate quiet and busy regimes	weather that switches between drizzle and storm
Quadratic (Zumbach)	self-excitation kernel; return-feedback kernel	on binned returns (GMM, then likelihood)	volatility forecasts with the right time asymmetry	a trend, not just activity, wakes the market up
Queue-reactive Hawkes	size-dependent background; excitation kernels	maximum likelihood with fixed decays	simulate queues with realistic clustering	the toll booth of Section 6.3, with rush hours

Table 11: Selected deep LOB models. F1 values are as reported by the authors on the data stated and are not comparable across rows (Section 2.4).

Model	Architecture	Data	Reported result	Independent re-test
CNN (Tsantekidis et al., 2017a)	convolutions over time $\times$ levels	FI-2010	F1 55–59%	–
LSTM (Tsantekidis et al., 2017b)	40-unit LSTM	FI-2010	F1 61–66%	–
DeepLOB (Zhang et al., 2019)	CNN + Inception + LSTM, $\sim 60k$ params	LSE, 5 stocks, 2017	F1 61–70%	re-trained on Nasdaq (Briola et al., 2025a): p_T 0.01–0.19
TABL (Tran et al., 2019)	bilinear + temporal attention, $\sim 10k$ params	FI-2010	F1 73–74%	70% on FI-2010, 60% on 2021 data (Prata et al., 2024)
TransLOB (Wallbridge, 2020)	dilated causal CNN + Transformer	FI-2010	F1 81–92%	59% on FI-2010 (Prata et al., 2024)
Axial-LOB (Kisiel and Gorse, 2022)	axial attention, $<10k$ params	FI-2010	F1 76–86%	73% on FI-2010 (Prata et al., 2024)
TLOB (Berti and Kasneci, 2025)	dual spatial/temporal attention	FI-2010, Nasdaq, BTC	F1 to 93% (FI-2010); 50–80% (INTC)	–
LiT (Xiao et al., 2025)	structured patches + Transformer	Binance, 2024	F1 59–66%	–
OF-MATNet (Bandealineaeni et al., 2025)	multi-asset, multi-axis attention on OFI	110 Nasdaq assets	R^2 gains in $>90\%$ of cases	–

Table 12: The comparisons a deep-model study should include. Each row should be reported with out-of-sample log loss and out-of-sample net P&L under the same execution model. The comparison that matters is row 3 against row 1, not row 3 against row 2.

Model	Input	Parameters	Typical inference latency
Logistic	$OFI_t^{(w)}$, $w = 1, \dots, 5s$; QI_t	~ 10	ns
CNN–LSTM (DeepLOB-type)	10-level snapshots	$\sim 10^5$	μs –ms
Transformer	raw snapshots	$\sim 10^6$	ms
Transformer	message stream + OFI	$\sim 10^6$	ms

Table 13: Representative SHADOW-PPOV parameters (one of the configurations in the code repository).

Parameter	Value	Meaning
place_rate_bp	300	place on 3% of qualifying third-party adds
max_dist / max_dist_cancel	10 / 12	ticks from touch to place / to force cancel
delayed_cancel_events	5	grace period after the shadowed order leaves
nlights_per_side	10	number of lights per side; each holds one order at one price
rng_seed	1	seed for the placement decision

Table 14: Three uses of generative models in LOB research.

Use	What is generated	Examples	In this paper
LOB generation	synthetic order and message streams	autoregressive message models (Nagy et al., 2023) and their evaluation benchmarks (Nagy et al., 2025)	adjacent work
LOB modelling	predictions and representations of market state	Transformers, state-space models, pretrained encoders (Linna et al., 2025)	Part II
Model generation	candidate models, signals and hypotheses	LAMD	this part

Table 15: Statistical metrics by prediction target.

Target	Metrics
Direction	log loss, Brier score (Brier, 1950), AUC (Fawcett, 2006), balanced accuracy, calibration error (Guo et al., 2017)
Direction with confidence	risk-coverage curves; F1 by confidence tier; interval coverage against nominal (e.g. the 68% coverage reported by Edward Manoharan et al. (2026))
Return	MSE, MAE, CRPS (Gneiting and Raftery, 2007)
Event time or fill	right-censored log-likelihood (Arroyo et al., 2024); calibration by queue-position bucket; concordance with caution
Intensity	point-process log-likelihood; time-rescaling residual tests

Table 16: Levels of evaluation, from statistical to economic.

Level	Question	Typical metrics
Prediction	Is the direction or value right?	accuracy, F1, AUC, MSE
Probabilistic prediction	Are the stated probabilities right?	log loss, Brier, CRPS, calibration
Execution	Will the order trade?	fill probability, fill-time likelihood
Conditional execution	How good is the trade when it happens?	mark-outs, adverse selection by queue position
Trading	Does it make money after costs?	net P&L, Sharpe ratio, transaction-level overlap TO (Briola et al., 2025a)
Capacity	How much can be traded before impact erases it?	turnover, impact-adjusted P&L
Robustness	Does it survive new periods, assets and regimes?	out-of-sample by regime and asset
Statistical validity	Could the result be luck given the search?	deflated Sharpe ratio, null maximum, backtest-overfitting probability

Table 17: Complexity dimensions of the model classes discussed in this survey, in orders of magnitude. A comparison should add, for each row, the out-of-sample Sharpe ratio, the deflated Sharpe ratio and the number of configurations tried, all under the same fill model.

Model	Parameters	Typical inference latency	Fitting burden
Model-free benchmark (§21)	~ 10 settings	μs	none
OFI baseline	~ 10	ns- μs	low
Shallow network	10^3 - 10^4	μs	medium
Transformer	$\sim 10^6$	ms	high
Hybrid	$\sim 10^6$	ms	high

Table 18: Starting points ordered by increasing complexity.

Situation	Start with	Why
Decision needed within microseconds; large-tick instrument	linear OFI or queue imbalance; queue-race probability (Section 6.2)	few operations, deterministic latency, strongest evidence in large-tick names (Gould and Bonart, 2016; Briola et al., 2025a)
Passive execution; fill probability matters	residual-volume and OFI baseline; then a survival model with queue position as an input (Zhong et al., 2025)	prices fill probability and adverse selection directly
Decision within milliseconds; liquid instrument	small network on order-flow inputs (Kolm et al., 2023; Lucchese et al., 2024)	order-flow inputs carry most of the gain; in re-tests the simple MLP degraded least (Prata et al., 2024)
Several horizons at once	sequence model only if baselines fail deflated and purged tests	complexity must clear the multiple-testing bar
Unusual volatility or liquidity	volatility gating of a simpler model	simple and robust when learned relations break
Several related instruments	lagged cross-asset OFI (Cont et al., 2023)	modest complexity; evidence of forecasting value

Table 19: Over-engineered versus parsimonious defaults.

Dimension	Over-engineered	Parsimonious default
Features	many correlated indicators	a few interpretable drivers (OFI, imbalance, queue position)
Model	deep network chosen first	linear, logistic or small network first
Fitting criterion	in-sample fit	out-of-sample economic value, with information criteria or shrinkage
Search	unrecorded trials	counted trials and a deflated result
Regime behaviour	fragile under shocks	degrades gracefully; gated by volatility
Diagnosis	opaque when it fails	failures traceable to an identifiable input

Table 20: Data sets and frameworks used in the literature surveyed.

Resource	What it provides	Limits
FI-2010 (Ntakaris et al., 2018)	5 Helsinki stocks, 10 days of 2010, 10 levels, normalised and downsampled	short, one market, pre-processed; results do not transfer reliably (Prata et al., 2024)
LOBSTER (Huang and Polak, 2011)	Nasdaq books reconstructed from ITCH messages, any level, any period	commercial; Nasdaq only; one venue of a fragmented market
Exchange MBO feeds (Nasdaq ITCH (Nasdaq, 2015), CME MDP 3.0 (CME Group, n.d.c))	full message-level data, queue positions	require capture or purchase; large; hard to share
Cryptocurrency exchange APIs	free, 24/7, many venues	different fees, participants and hours; often snapshots plus updates
LOBCAST (Prata et al., 2024)	re-implementations of 15 models, common protocol	classification focus
LOBFrame (Briola et al., 2025a)	open framework with transaction-level evaluation	DeepLOB-centred
LOB-Bench (Nagy et al., 2025)	evaluation suite for generative LOB models	generation, not trading

Table 21: Representative commercial hardware for trading and the figures published by the vendors. Except for the STAC result, figures are vendor claims without independent audit.

Offering	What it is	Published figure
AMD Alveo UL3524 / UL3422 (AMD, 2023, 2024)	FPGA cards built for trading, 64 low-latency transceivers	under 3 ns transceiver latency, raw loopback excluding logic and protocol
Exegy (incl. Enyx, acquired 2022) (Exegy, 2022; STAC, 2024)	FPGA feed handling, order entry and trigger frameworks	STAC-T0 audited trigger latency of the order of ten nanoseconds
Algo-Logic (Algo-Logic Systems and AMD)	CME tick-to-trade system on Alveo U50, with feed handler, book and TCP order entry	under 500 ns tick-to-trade
Magmio (Magmio)	C++ strategies compiled to FPGA by high-level synthesis	under 100 ns for triggers; under 350 ns with book building
Xelera Silva (Xelera Technologies)	tree-ensemble and small-network inference on FPGA	about 250 ns (99th percentile) for a gradient-boosted model inline on UL3524; 1.1–1.3 μ s as card offload
Napatech + Xelera (Napatech, 2025)	the same inference on FPGA network cards	1.55 μ s for a 1000-tree model with 100 features

Table 22: A qualitative guide to workload placement. It reflects the general division of labour between platforms, not measured results.

Workload	Preferred form	plat-	Rationale
Market-data parsing, book maintenance	FPGA		regular, protocol-bound, deterministic
Linear OFI, queue imbalance, simple triggers	FPGA or CPU		few operations; easy to pipeline
Small quantised networks or tree ensembles on the critical path	FPGA		feasible when capacity is constrained by design
Pre-trade risk checks, mass cancel	FPGA		must be deterministic and next to order generation
Multi-horizon or cross-sectional deep models, full Transformers	GPU or other accelerator		memory and operator mix exceed FPGA budgets
Large-scale simulation, training, hyper-parameter search	GPU		throughput dominates
Strategy logic that changes often	CPU with kernel bypass		flexibility outweighs nanoseconds

Table 23: How contamination affects order-book signals.

Form	Signature in the data	Effect on signals
Spoofing	large orders away from the touch, cancelled quickly	OFI and imbalance point the wrong way
Layering	several non-genuine orders at successive levels	distorts depth, slope and multi-level OFI
Quote stuffing	bursts of submissions and cancellations	inflates event counts; breaks event-clock assumptions
Icebergs and hidden orders	repeated executions at one price with stable visible size	displayed depth understates liquidity; depletion and fill estimates biased

Table 24: Evidence status of the main claims in this survey.

Claim	Status
Short-horizon mid changes are approximately linear in OFI, with slope inversely related to depth (Cont et al., 2014)	established
Hawkes stationarity when $\rho(\Gamma) < 1$ and $\bar{\lambda} = (I - \Gamma)^{-1}\mu$	established
Input representation can matter as much as the architecture	synthesis
Naive “touch” backtests overstate passive fills relative to queue-exact replay	synthesis; argued from price–time priority
Front-of-queue positions are worth more than back-of-queue positions, with adverse selection increasing in queue position (Moallemi and Yuan, 2017)	established in a model; size across regimes open
Hybrid inputs derived from the same stream add little to a sufficiently large network	open hypothesis
LLM-driven search raises the multiple-testing burden	established in principle

A Symbols and acronyms

A.1 Symbols

Global symbols are defined in Section 1.2; the others are defined where they first appear. The table lists every symbol used in more than one place, and the main local ones, with the section that defines them.

Symbol	Meaning	Defined in
<i>Time, book and prices</i>		
$t, t_n, t-$	time; time of event n ; just before t	§1.2
n	index of book events	§1.2
H, h	forecast or fill horizon; mark-out horizon after a fill	§1.2
δ	tick size	§1.2
L, ℓ	number of levels; level index	§1
$p_{\ell,t}^b, v_{\ell,t}^b, p_{\ell,t}^a, v_{\ell,t}^a$	price and size at bid / ask level ℓ	§1
m_t, s_t	mid-price; spread	§1
$\Delta m_{t,H}$	mid change over the next H	§1.2
u	elapsed time after a reference time	§6
<i>Information, actions and outcomes</i>		
$X_t, Y_{t+H}, \hat{Y}_t$	observable information; target; prediction	§1.2
a_t, d	action; order side (+1 buy, -1 sell)	§1.2
Q_t^{ahead}, q	volume ahead of an order in its queue; a value of it	§1.2
$\tau_F, t_F, p^{\text{fill}}$	time to fill; fill time; fill price	§1.2
Fill, $\mathbb{K}_{\text{Fill}}$	event $\{\tau_F \leq H\}$; its indicator	§1.2
Π	profit per lot	§1.2
$\Pi^{\text{agg}}, \Pi^{\text{pass}}$	aggressive and passive profit	§11
$R_h^{\text{fill}}, R_{\text{rebate}}$	post-fill mark-out; rebate	§11
$C_{\text{spread}}, C_{\text{fees}}, C_{\text{impact}}$	costs of crossing	§11
$\text{AS}(q, h)$	adverse selection given queue position	§13
$V^{\text{sig}}, V^{\text{exec}}$	signal value; execution value	§11.1
$T_{\text{lat}}, T_{\text{move}}$	total latency; time to the next one-tick mid move	§14, §8
θ	move threshold in prediction targets	§8
<i>Signals</i>		
$\Delta v_n^b, \Delta v_n^a, e_n$	change in best sizes from event n ; its OFI score	§4
$w, \text{OFI}_t^{(w)}$	window length; order-flow imbalance over it	§4
β, ε_t	price-impact coefficient; residual	§4
QI_t, ι_t	queue imbalance in $[-1, 1]$; rescaled to $[0, 1]$	§4, §4.4
$m_t^{\text{wmid}}, m_t^{\text{micro}}$	weighted mid; micro-price	§4, §4.4
$\eta_k, g^{(k)}$	times of successive mid changes; expected adjustments	§4.4
$\mathbf{U}, \mathbf{J}^{(1)}, \mathbf{J}^{(2)}, \mathbf{B}, \mathbf{\Delta}$	micro-price transition matrices and jump sizes	§4.4
$\beta_{\kappa\kappa'}, m_t^{(\kappa)}$	cross-impact coefficient; mid of instrument κ	§10
<i>Queues</i>		
τ^a, τ^b	depletion time of the best ask / bid	§6
$r_{\text{in}}, r_{\text{out}}$	rates at which a queue fills and drains	§6
r_L, r_M, r_C	limit-order, market-order and cancellation rates	§6.2
Θ	average product of new best-queue sizes after a price change	§6.2
$\bar{\ell}$	distance in ticks from the opposite best quote	§6.1
$\mathcal{Q}_{\bar{\ell}}^{\text{bid}}, \mathcal{Q}_{\bar{\ell}}^{\text{ask}}$	orders at distance $\bar{\ell}$ from the opposite quote	§6.1
$r_L(\bar{\ell}), r_c(\bar{\ell}), r_0, \Xi$	limit-order rate; per-order cancellation rate; power-law scale and exponent	§6.1
$N_L(\bar{\ell}), T_{\text{obs}}$	count of queue increases; observed trading time	§6.1
ϖ	Laplace-transform variable	§6.1
x	number of orders (or units) in a queue	§6.1, §6.3

Symbol	Meaning	Defined in
$r_L(x), r_C(x), r_M(x)$	queue-reactive rates at queue size x	§6.3
$p_{\text{ref}}, \pi_{\text{move}}, \pi_{\text{reinit}}$	reference price; probability it moves when a best queue empties; probability the book is redrawn	§6.3
<i>Point processes</i>		
$N(t), N_i(t)$	counting process; for event type i	§7, App. C
$\lambda(t), \lambda_i(t), \bar{\lambda}$	conditional intensity; for type i ; stationary mean	§7
μ, μ_i, μ_0	Poisson or background rate; for type i ; per side in the price model	§7
ϕ, ϕ_{ij}, K	kernel; between types; number of event types	§7
$\Gamma, \rho(\Gamma), I$	branching matrix (scalar: branching ratio); spectral radius; identity	§7
α, ω	exponential-kernel jump and decay rate (decay often written β elsewhere)	§7, App. C
$\tilde{\lambda}_i, \vartheta_i$	state-dependent intensity; its coefficients	§7
$N^\pm(t), \lambda^\pm(t), \delta_m, \Gamma_\times$	up/down mid-move counts; their intensities; mid jump; cross-excitation weight	§7
$\sigma^2(w), \Lambda$	variance per unit time at window w ; total move rate	§7
$\lambda^{L,b}, \lambda^{C,b}, \lambda^{M,b};$ $\lambda^{L,a}, \lambda^{C,a}, \lambda^{M,a}$	intensities of limit orders and cancellations at the bid and ask, and of market buys (M, b) and sells (M, a)	§7.10
$\bar{v}$	average event size	§7.10
t_{end}	end of the observation window	§7.9, App. C
ζ	power-law kernel tail exponent	§7, App. C
ψ_k	state of a discrete-time linear system (stability analogy)	§7
$D(w), \mathcal{H}$	Fano factor; Hurst exponent	App. C
$\Delta t_n, \Psi_n$	gap between arrivals; running sum for the likelihood	App. C
<i>Models and evaluation</i>		
$M, M_0, M_1, V(M), \Delta V$	model; baseline; candidate; out-of-sample value; its difference	§1.2
$\mathcal{C}(M), \chi$	complexity vector; exchange rate between value and complexity	§32
N_{trials}	number of candidates evaluated	§24
TO	transaction-level overlap score (called p_T by its authors)	§8.5
VC, N_s, ς	VC dimension; sample size; failure probability of a bound	§31
$\sigma_{\text{SR}}, \text{SR}_0, N_{\text{obs}}, \gamma_E$	spread of Sharpe estimates; null maximum; number of returns; Euler–Mascheroni constant	§31
$\mathbf{z}_n^{\text{qry}}, \mathbf{z}_n^{\text{key}}, \mathbf{z}_n^{\text{val}}, \text{dim}_{\text{emb}}$	attention query, key and value; their dimension	§8
$\text{att}_{nn'}$	attention weight of event n on event n'	§8
$\mathbf{z}^{\text{hist}}(t), \mathbf{o}_i$	history vector of a neural point process; output weights for type i	§7.12
$\text{softplus}(y)$	$\ln(1 + e^y)$, a smooth positive function; y any real number	§7.12
ν_t, S_t	gate; Hawkes-intensity input	§9
$\mathbb{P}, \mathbb{E}, \Phi$	probability; expectation; standard normal distribution function	§1.3, §31
$A, B, Z, \mathbb{K}_A$	generic events and random number in the primer; indicator	§1.3
R^2	coefficient of determination	§4
$e, \ln$	Euler’s number; natural logarithm	–

A.2 Acronyms

AIC	Akaike information criterion	LOB	limit order book
AUC	area under the ROC curve	LSTM	long short-term memory network
BIC	Bayesian information criterion	MBO / MBP	market by order / by price
CNN	convolutional neural network	MCC	Matthews correlation coefficient
CRPS	continuous ranked probability score	MDP	CME Market Data Platform
DSR	deflated Sharpe ratio	MLP	multilayer perceptron
FIFO	first in, first out	OFI	order-flow imbalance
FPGA	field-programmable gate array	OOS	out of sample
GPU	graphics processing unit	P&L	profit and loss
HFT	high-frequency trading	QI	queue imbalance
IOC / FOK	immediate-or-cancel / fill-or-kill	RNN	recurrent neural network
ITCH	Nasdaq market-by-order data protocol	SOM	simulated order manager
LAMD	LLM-assisted model discovery	SSM	state-space model
LLM	large language model	VC	Vapnik–Chervonenkis

B Glossary

Terms are grouped by subject and listed alphabetically within each group. Each entry gives a plain-language definition and, where useful, a reference for further reading. Introductory textbooks for each area are: market microstructure [Harris \(2002\)](#); [O’Hara \(1995\)](#); [Hasbrouck \(2007\)](#); futures [Hull \(2022\)](#); statistical learning [Hastie et al. \(2009\)](#); deep learning [Goodfellow et al. \(2016\)](#); point processes [Daley and Vere-Jones \(2003\)](#).

B.1 Markets and trading

Adverse selection The tendency of a passive order to be filled exactly when the price is about to move against it, because the counterparty often knows more ([Glosten and Milgrom, 1985](#)). Section 13.

Aggressive / passive An aggressive order trades immediately against waiting orders; a passive order waits in the book for someone else to trade with it.

Alpha The return of a strategy or signal beyond an appropriate benchmark, after the relevant risk and cost adjustments; in this survey, usually the part of a short-horizon forecast that is profitable after costs.

Backtest Running a trading strategy on historical data to estimate how it would have done.

Basis point (bp) One hundredth of a percent, 0.01%. A parameter of 300 bp is 3%.

Capacity How much a strategy can trade before its own price impact erases its profit.

Clearing fees Fees paid to the clearing house, the institution that settles trades and guarantees both sides.

Counterparty The trader on the other side of a trade.

Cross-sectional Across many instruments at the same moment, rather than over time for one.

Depth The total size waiting in the book near the best prices; a *deep* book has a lot.

Drift The average direction in which the price moves over a period, independent of one’s own trades.

Edge Expected profit per trade, before or after costs as stated.

Equity / fixed income Stocks / bonds and other interest-rate products.

ETF Exchange-traded fund: a share, traded like a stock, that holds a basket of assets.

Fill The execution of an order, in full or in part.

Fragmentation The same asset trading on several venues, so its liquidity is split among them.

Futures contract A standardised exchange-traded agreement to buy or sell an asset at a set future date and price (Hull, 2022). ES, NQ and ZN are futures on the S&P 500 index, the Nasdaq-100 index and the 10-year US Treasury note.

Iceberg / hidden order An order that shows only part (iceberg) or none (hidden) of its size.

Implementation cost / slippage The difference between the price obtained and a reference price such as the mid at the moment of decision (Harris, 2002).

Index constituents The individual stocks that make up an index such as the S&P 500.

Informed trader A trader who knows something about the next price move that others do not.

Instrument One specific tradable product, for example the March ES contract.

Inventory / position The net quantity held after one’s fills, long (positive) or short (negative); it carries risk until offset.

Large-tick asset An instrument whose tick is large relative to its typical price moves, so the spread is almost always one tick and queues are long.

Lifting the offer / hitting the bid Buying aggressively at the ask / selling aggressively at the bid.

Liquidity How easily one can trade without moving the price. Resting limit orders supply it; aggressive orders consume it.

Mark-out The gain or loss of a fill measured against the mid a fixed time later. Equation (32).

Market impact How much one’s own trading moves the price against oneself (Bouchaud et al., 2018).

Market making Continuously posting both a bid and an ask in order to earn the spread, while managing inventory (Avellaneda and Stoikov, 2008; Cartea et al., 2015). A posted bid or ask is a *quote*.

Mass cancel A single command cancelling all of a trader’s open orders.

Meta-order A large order executed as a sequence of smaller orders over time.

Micro-price An estimate of the “fair” price that adjusts the mid toward the side with the smaller queue (Stoikov, 2018).

Optimal execution Splitting a large order over time to minimise its total cost and risk (Almgren and Chriss, 2001).

P&L Profit and loss.

Paper trading Running a strategy on live data with simulated, not real, orders.

Pre-trade checks Automatic checks that block an order breaking size, price or position limits before it is sent.

Price point The contract’s own price unit. One ES point is four ticks, worth \$50 per contract.

Quote skew Shifting both a market maker’s bid and ask in the direction of expected price movement or to reduce inventory.

Rebate A payment some venues make to passive orders for supplying liquidity, funded by fees on aggressive orders.

Regime A stretch of time with stable market behaviour, such as calm or volatile.

Return The relative change in value over a period.

Session One period during which the exchange is open, usually one trading day.

Sharpe ratio Average return divided by the standard deviation of return: profit per unit of risk ([Sharpe, 1966, 1994](#)).

Spoofing / layering Placing orders one intends to cancel before they trade—one large order (spoofing) or several stacked across prices (layering)—to create a false impression of supply or demand. Illegal in US futures markets ([Commodity Futures Trading Commission, 2013](#)).

Square-root impact law The empirical regularity that the price impact of a meta-order grows roughly with the square root of its size ([Bouchaud et al., 2018](#)).

Stress testing Checking a system or strategy under extreme or rare scenarios.

The touch The current best bid and best ask.

Venue A trading platform or exchange.

Volatility How much the price fluctuates, usually the standard deviation of returns.

VWAP Volume-weighted average price: the average trade price over an interval, each trade weighted by its size ([Harris, 2002](#)).

Zero-intelligence model A model in which orders arrive at random with no strategy; the matching rules alone generate the book’s behaviour ([Smith et al., 2003](#)).

B.2 Exchanges and market data

CME Group The Chicago Mercantile Exchange group, the main US futures exchange operator; its electronic platform is CME Globex.

Event clock / calendar clock Sampling the market after a fixed number of events / a fixed number of seconds.

Feed / feed handler An exchange’s real-time stream of market-data messages / the software that decodes the feed and maintains the book.

iLink 3 CME’s order-entry protocol, used to send, modify and cancel orders ([CME Group, n.d.a](#)).

ITCH Nasdaq’s market-by-order data protocol ([Nasdaq, 2015](#)); Section 1.

L1, L2, L3 Feed detail levels: best bid and ask only; total size at each price level (market by price, MBP); every individual order (market by order, MBO).

LOBSTER A service that reconstructs Nasdaq order books from ITCH data for research ([Huang and Polak, 2011](#)).

Matching engine The exchange computer that holds the book and matches buy and sell orders.

MDP 3.0 CME’s Market Data Platform protocol: binary messages over UDP multicast ([CME Group, n.d.c](#)).

Order acknowledgement The exchange’s confirmation that it has accepted an order.

Snapshot recovery A periodic full-book message that lets a feed handler resynchronise after lost packets.

B.3 Statistics and evaluation

Ablation Removing or adding one component of a model to measure what it contributes.

Akaike / Bayesian information criterion (AIC / BIC) Scores that trade goodness of fit against the number of parameters; BIC penalises parameters more heavily in large samples ([Akaike, 1974](#); [Schwarz, 1978](#)).

AUC Area under the ROC curve: the probability that a randomly chosen positive case receives a higher score than a randomly chosen negative one ([Fawcett, 2006](#)).

Balanced accuracy The average of the recall (true-positive rate) of each class.

Brier score The mean squared difference between predicted probabilities and 0/1 outcomes ([Brier, 1950](#)).

Calibration error A summary of how far predicted probabilities are from observed frequencies ([Guo et al., 2017](#)).

Calibration Agreement between predicted probabilities and observed frequencies: events given 70% happen about 70% of the time ([Guo et al., 2017](#)).

Coefficient of determination (R^2) The fraction of the variance of a target explained by a fitted model.

Concordance (C-index) The fraction of pairs whose predicted order of event times matches the actual order.

Confidence interval A range that contains the true value with a stated probability over repeated experiments; it can be computed by resampling ([Efron and Tibshirani, 1993](#)).

Confounding Mixing of two effects so that the cause of an observed difference cannot be isolated.

Counterfactual What would have happened under an action that was not actually taken.

CRPS Continuous ranked probability score: a distance between a predicted distribution and the realised outcome; lower is better ([Gneiting and Raftery, 2007](#)).

Decile One tenth of a sorted data set.

Deflated Sharpe ratio A Sharpe ratio corrected for the number of strategies tried and for non-normal returns ([Bailey and López de Prado, 2014](#)).

Double descent The observation that test error can fall again as model capacity grows past the point of fitting the training data exactly ([Belkin et al., 2019](#)).

Epps effect The fall in measured correlation between two assets when it is computed over very short intervals.

F1 / macro-F1 F1 is the harmonic mean of precision and recall; macro-F1 averages F1 over the classes with equal weight.

False-discovery rate The expected fraction of claimed discoveries that are false ([Benjamini and Hochberg, 1995](#)).

Granger causality A test of whether past values of one series improve forecasts of another beyond its own past.

In-sample / out-of-sample (OOS) Measured on the data used to build a model / on data it has never seen.

Interval coverage The fraction of outcomes that fall inside predicted intervals, compared with the intended (*nominal*) fraction.

Cross-validation (random folds) Splitting data into several parts, training on all but one and testing on the remaining one, in rotation (Hastie et al., 2009). Inappropriate for time-ordered market data (Section 30).

Lead–lag A relation in which one instrument’s price moves first and a related one follows after a short delay.

Lag A past value of a variable at a fixed offset in time.

Lasso Regression with a penalty on the sum of absolute coefficients, which sets unhelpful coefficients exactly to zero (Tibshirani, 1996).

Log loss The average of $-\ln$ of the probability a classifier gave to the true class (cross-entropy); lower is better (Hastie et al., 2009).

Log-likelihood The logarithm of the probability a model assigns to the observed data; higher means a better fit.

Matthews correlation coefficient (MCC) A classification score between -1 and 1 that remains informative when classes are unbalanced.

Model confidence set The set of models that cannot be statistically distinguished from the best, at a stated confidence level (Hansen et al., 2011).

MSE / MAE Mean squared error / mean absolute error.

Multiple testing Trying many hypotheses inflates the chance that some look good by luck (White, 2000; Harvey et al., 2016). Section 27.

Non-normal returns Returns whose distribution has fatter tails or more skew than a Gaussian.

Non-stationary Having statistical properties that change over time.

Null hypothesis / significance The null is the assumption of no effect. A result is *significant at 5%* if it would occur less than 5% of the time under the null.

Principal component A direction in the data along which variance is largest; the first principal component is the best one-number summary of several correlated variables.

Probability of backtest overfitting The estimated probability that the configuration that looked best in sample performs below the median out of sample (Bailey et al., 2017).

Proper scoring rule A score that a forecaster optimises by reporting true beliefs, such as log loss, the Brier score and the CRPS (Gneiting and Raftery, 2007).

Purging / embargo Removing training samples whose information overlaps the test period / leaving a gap after each test period, both to prevent leakage of future information (López de Prado, 2018).

Rademacher complexity A measure of how well a family of models can fit random noise on the observed data; it bounds the gap between training and test error (Bartlett and Mendelson, 2002).

Reality check A resampling test of whether the best of many strategies truly beats a benchmark (White, 2000).

Risk–coverage curve The error rate plotted against the fraction of predictions kept, as the least confident are discarded (Geifman and El-Yaniv, 2017).

Selective prediction Acting only on predictions whose confidence exceeds a threshold (Geifman and El-Yaniv, 2017).

Signature plot Measured volatility plotted against the sampling interval; higher measured volatility at short intervals reveals bid–ask bounce.

Standard error / annualised The standard deviation of an estimate / scaled to a one-year period.

Survival model A model of the time until an event that correctly handles cases where the event has not yet happened (*censoring*) (Cox, 1972).

Time-rescaling test A goodness-of-fit test for point-process models: rescaling time by the fitted intensity should turn the gaps between events into independent unit-exponential variables (Brown et al., 2002).

VC dimension The largest number of points a family of classifiers can label in every possible way; a measure of capacity (Vapnik, 1998).

Walk-forward evaluation Train on a past window, test on the next, roll forward and repeat (López de Prado, 2018).

B.4 Machine learning

Attention A mechanism that weights each input element by its learned relevance to the current one (Vaswani et al., 2017).

Attentive neural process A model that predicts a distribution for a new input by attending to a *context set* of example input–outcome pairs (Kim et al., 2019).

Autoregressive model A model that generates a sequence one element at a time, each conditioned on those before it.

Axial attention Attention applied separately along each axis of a multi-dimensional input, such as time and price level.

Backbone The main feature-extracting network beneath a task-specific output layer.

BERT A Transformer encoder pretrained by predicting masked-out tokens (Devlin et al., 2019).

CNN / RNN / LSTM Convolutional networks apply learned sliding filters (LeCun et al., 2015); recurrent networks carry a hidden state along a sequence; the LSTM is a recurrent network with gates for long-range memory (Hochreiter and Schmidhuber, 1997).

Context length How many past elements a sequence model sees at once.

Cross-entropy loss The negative log-likelihood of the true class under a classifier’s predicted probabilities; the standard training loss for classification.

Data augmentation Enlarging a training set with modified or synthetic examples.

Differential evolution A derivative-free optimisation method that evolves a population of candidate parameter vectors.

Diffusion model A generative model that learns to reverse a gradual process of adding noise to data.

Embedding A learned vector representation of an input.

Encoder A network that maps an input to a learned vector representation.

Fine-tuning Further training a pretrained model on a specific task.

Foundation model A large model pretrained on broad data and then adapted to many tasks.

Generative adversarial network (GAN) A generator trained against a discriminator that tries to tell generated data from real data.

Generative model A model that produces new samples resembling its training data.

Inception module A network block that applies several convolutions of different widths in parallel and concatenates the results.

Inference Running a trained model to obtain a prediction; *inference latency* is how long that takes.

Logit / logistic regression The log-odds $\ln(\mathbb{P}(A)/(1 - \mathbb{P}(A)))$ of an event A ; logistic regression models it as a linear function of the inputs (Hastie et al., 2009).

Model capacity How complex a function a model can represent; roughly, its number of parameters.

Pretraining Training a model on a generic task before adapting it to a specific one.

Query, key, value The three vectors an attention layer computes for each element: the query is matched against every key to set the weights, and the output is the weighted average of the values.

Quantisation Using low-precision integers instead of floating point for a network’s weights and arithmetic, for speed or to fit hardware (Jacob et al., 2018).

Reinforcement learning Learning a decision policy by trial and error from rewards.

Selective state-space model A state-space model, such as Mamba, whose recurrence parameters depend on the current input (Gu and Dao, 2023).

Softmax A function that turns a vector of scores into probabilities that are positive and sum to one.

Softplus The function $\ln(1 + e^y)$: smooth, always positive, and close to y when y is large; used to keep predicted rates positive.

State-space model A model with a hidden state updated by a (learned) linear recurrence (Kalman, 1960); recent neural versions such as Mamba handle long sequences efficiently (Gu and Dao, 2023).

Token / tokenisation The unit of input a sequence model reads / splitting data into such units.

Transformer A sequence network built from attention layers; the architecture behind large language models (Vaswani et al., 2017).

Tree ensemble Many decision trees combined, as in random forests or gradient boosting (Hastie et al., 2009).

World model A model’s internal representation of how the state of a system evolves.

B.5 Probability and stochastic processes

Birth–death process A counting process that moves up by one at a birth rate and down by one at a death rate that may depend on its current value (Cont et al., 2010).

Branching ratio In a Hawkes process, the expected number of events directly triggered by one event (the spectral radius of the branching matrix in several dimensions); below one, every cascade dies out.

Branching process A model in which each individual independently produces a random number of offspring; a Hawkes process is one (Hawkes and Oakes, 1974).

Cox–Ingersoll–Ross process A continuous-time random process that is always positive and reverts toward a mean; used for interest rates and volatility.

Diffusive limit The continuous random process that a jumpy process resembles when viewed at a large scale of time and size.

Ergodic Having a unique long-run distribution, so that averages over a long history converge to fixed values.

Expectation–maximisation (EM) An iterative fitting method for models with hidden quantities: guess the hidden quantities probabilistically from the current parameters, re-estimate the parameters, and repeat (Veen and Schoenberg, 2008).

Exogenous / endogenous Caused from outside / triggered by earlier events of the same process.

First-passage time The first time a random quantity reaches a given level.

Hawkes process A self-exciting point process in which each event temporarily raises the rate of future events (Hawkes, 1971; Bacry et al., 2015). Section 7.

Heston model A stochastic-volatility model in which variance follows a Cox–Ingersoll–Ross process.

Hurst exponent A measure of long memory: 0.5 for no memory, above 0.5 for persistence.

Intensity The instantaneous expected rate of events.

Kalman filter A recursive estimator of a hidden state that evolves linearly and is observed with noise (Kalman, 1960).

Laplace transform For a waiting time τ , the function $\varpi \mapsto \mathbb{E}[e^{-\varpi\tau}]$; it turns sums of independent waiting times into products, which is why it is used for first-passage problems.

Markov A process whose future depends only on its present state, not on its history.

Martingale A process whose expected future value, given the present, equals its present value.

Point process A random sequence of event times (Daley and Vere-Jones, 2003).

Queue-reactive model A model of the book in which the rate of each event type at a queue depends on the current queue sizes (Huang et al., 2015).

Queueing model A probabilistic model of arrivals to and departures from queues.

Rough volatility Volatility whose paths are rougher than those of Brownian motion, with a Hurst parameter below 0.5.

Stationary Having statistical properties that do not change over time. For a Hawkes process, the cascade of triggered events does not explode.

Stochastic Random.

Thinning Simulating a point process by proposing candidate events at an upper-bound rate and keeping each with probability equal to the true intensity divided by the bound (Ogata, 1981).

Zumbach effect Past volatility over long horizons predicts future short-horizon volatility better than the reverse; a time asymmetry of volatility (Zumbach, 2009).

B.6 Systems and hardware

Actor model A model of concurrency in which independent actors own their state and communicate only by messages (Hewitt et al., 1973).

ASIC Application-specific integrated circuit: a chip designed for one task.

CGRA Coarse-grained reconfigurable array: a programmable chip whose building blocks are larger than an FPGA’s, often used for AI inference.

Chiplet One of several small dies packaged together to act as a single processor.

Colocation Placing trading servers in the exchange’s data centre to minimise network distance.

FPGA Field-programmable gate array: a chip whose logic can be reconfigured to implement a specific circuit; slower and larger than an ASIC for the same function but reprogrammable (Kuon and Rose, 2007). Its configurable resources are called the *fabric*.

High-level synthesis (HLS) Compiling a program written in a language such as C++ into FPGA logic.

Jitter Variation in latency from one event to the next.

Kernel bypass Receiving and sending network packets directly from user space, skipping the operating system’s network stack (DPDK Project, n.d.).

Multicast One-to-many UDP delivery: the sender transmits once and every subscribed host receives the packet.

Operator mix The kinds of computation a model needs, such as matrix multiplication or softmax.

SmartNIC A network card with programmable logic or processors that can compute on packets as they arrive.

STAC An industry body that runs audited benchmarks of trading technology.

Tick-to-trade The time from a market-data packet arriving to the resulting order leaving.

Transceiver The circuit that converts between parallel data and the serial signal on a network link.

Trigger Pre-computed order logic, often in hardware, that sends an order as soon as a market condition is detected.

B.7 Economics

Abduction Forming a new hypothesis that would explain a surprising observation, as opposed to searching among hypotheses already given (Zahavy, 2026).

Agent-based model A simulation of many artificial traders following rules and interacting.

Econometric / reduced-form model A statistical model fitted to economic data that describes relationships without modelling traders’ decisions.

Equilibrium model A theoretical model in which prices result from rational traders’ optimal strategies.

Glosten–Milgrom and Kyle models Classic models in which market makers lose to better-informed traders and set prices or spreads to recover those losses (Glosten and Milgrom, 1985; Kyle, 1985).

Latency arms race Competition among fast traders to be first, and its economic costs (Budish et al., 2015).

Price formation How trading and information are turned into the market price.

C Point-process background

This appendix is a self-contained introduction to the point processes used in Section 7: the Poisson, renewal and Hawkes processes, the conditional intensity, the branching ratio, the Fano factor and the Hurst exponent, and how a Hawkes process is fitted to data. It assumes no prior knowledge of point processes. It is adapted from Appendix A of Maciejewski (2026a), which also supplies the empirical figures quoted below for CME NQ market data. Readers who know what a conditional intensity is can go straight to Section C.7.

Symbols follow the rest of this paper. For a single event type ($K = 1$) the branching matrix Γ of Section 7 is a single number, the branching ratio, written Γ here as well. Arrivals are indexed by n , as book events are elsewhere in the paper, and a counting window has length w , as the order-flow-imbalance window does.

C.1 The Poisson process

The setup. Watch packets arrive from a market-data feed and write down the time of each arrival:

```
09:30:00.213    <- 1st arrival
09:30:00.451    <- 2nd
09:30:00.802    <- 3rd
09:30:01.014    <- 4th
09:30:01.317    <- 5th
...
```

That list of timestamps is the object being modelled. Nothing else—no prices, no sides, only *when*.

The Poisson idea. Suppose that in every very small slice of time there is a small, *independent* chance of an arrival. Independent means that whether an arrival happens now does not depend on when the previous one came, on how many came in the last minute, or on anything else about the past. The *rate* of the process is a single number μ , in arrivals per second. At a rate of 300 per second, on average 300 arrivals occur in each second. Three consequences follow from the independence.

(1) **The gap between consecutive arrivals is exponentially distributed with mean $1/\mu$.** At $\mu = 300$ per second the mean gap is

$$\text{mean gap} = \frac{1}{\mu} = \frac{1}{300/\text{s}} \approx 3.3 \text{ ms.}$$

The gap distribution is *memoryless*: however long one has been waiting, the expected time to the next arrival is still 3.3 ms. The probability that a gap exceeds a length u is

$$\mathbb{P}(\text{gap} > u) = e^{-\mu u},$$

so at this rate $\mathbb{P}(\text{gap} > 3.3 \text{ ms}) \approx 36.8\%$, $\mathbb{P}(\text{gap} > 6.6 \text{ ms}) \approx 13.5\%$ and $\mathbb{P}(\text{gap} > 10 \text{ ms}) \approx 5.0\%$. The probability that a gap is shorter than a tenth of the mean is

$$\mathbb{P}(\text{gap} < \frac{1}{10} \text{ mean}) = 1 - e^{-0.1} \approx 9.52\%,$$

whatever the rate. On CME NQ packet data the share is 73% (Maciejewski, 2026a).

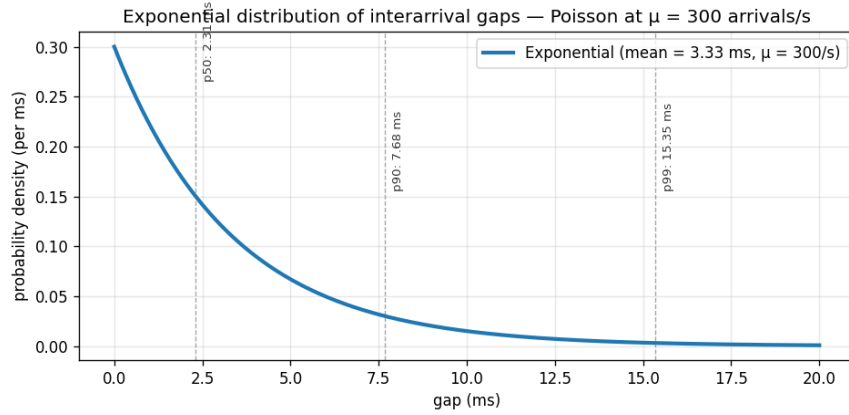


Figure 5: The exponential density of the gaps between arrivals of a Poisson process at 300 arrivals per second. Dashed vertical lines mark the median, the 90th and the 99th percentile. Half the gaps are shorter than the median of 2.3 ms and 10% are longer than 7.7 ms. Real market-data streams have far more mass near zero and a much heavier right tail. Reproduced from Maciejewski (2026a).

(2) The count in any window of length w is Poisson-distributed. Let N_w be the number of arrivals in a window of length w . Then

$$\mathbb{E}[N_w] = \mu w, \quad \text{Var}[N_w] = \mu w,$$

so the ratio of variance to mean is 1 for every w . This ratio is the *Fano factor* (Section C.11); a value other than 1 rejects the Poisson model.

(3) Counts in non-overlapping windows are independent. The number of arrivals in the first second says nothing about the number in the next.

Poisson as the null hypothesis. The Poisson process is the stream with no memory, no clustering and no regularity: arrivals are as uncoordinated as possible. Any departure from it is structure in the arrival stream.

Two ways a real feed departs from Poisson. The first is *bunching*: many gaps are far shorter than the exponential law predicts (73% against 9.52% above). The second is *memory in the rate*: after a burst starts, more arrivals are likely to follow. Modelling either needs a process whose rate can change with its own history—a *conditional-intensity point process*.

C.2 Points, processes and marks

- A **point** is a single instant at which something happens, such as the timestamp of one packet or one order-book event.
- A **process** is a random object that unfolds in time; here, a random list of arrival times $t_1 < t_2 < t_3 < \dots$.
- A **point process** is a random sequence of points on the time axis, specified by *when* events happen and nothing else.

If each event carries additional information (side, price, size, message type), the process is a **marked point process**: the timestamps are the points and the extra information is the *mark*. Four order-book messages in the first second of a session might read

```
09:30:00.001234567   Add,      bid 21432.50, size 3
09:30:00.001234571   Add,      ask 21432.75, size 5
09:30:00.017891234   Cancel, order X, bid 21432.25
09:30:00.238491702   Trade,   ask 21432.75, size 2
```

The point process is the four timestamps; message type, side, price and size are marks. The multivariate Hawkes process of Section 7 is a marked point process whose mark is the event type i .

C.3 The probability space

An arrival stream is modelled as a **stochastic process**, a family of random variables indexed by time, on a probability space $(\Omega, \mathcal{A}, \mathbb{P})$:

- Ω , the *sample space*, is the set of all possible outcomes; here an outcome is one complete realisation of a session's arrivals.
- $\mathcal{A}$, the σ -*algebra*, is the collection of yes-or-no questions about the outcome to which a probability can be assigned, such as “did at least one packet arrive between 09:30:00 and 09:30:01?”. It is closed under negation and countable union.
- $\mathbb{P} : \mathcal{A} \rightarrow [0, 1]$ assigns a probability to each such event, with $\mathbb{P}(\emptyset) = 0$, $\mathbb{P}(\Omega) = 1$, and probabilities of disjoint events adding up.

C.4 History: information accumulated over time

The information available at time t grows as time passes. Formally it is a **filtration**, an increasing family of σ -algebras

$$\{\mathcal{I}_t\}_{t \geq 0}, \quad \mathcal{I}_{t'} \subseteq \mathcal{I}_t \text{ for } t' \leq t.$$

$\mathcal{I}_t$ is everything observable up to and including time t , and $\mathcal{I}_{t-}$ everything observable strictly before t . “Conditional on $\mathcal{I}_{t-}$ ” means “given everything seen strictly before t ”. In the main text this is what conditioning on X_t means: a model may use only what is in the history.

C.5 The counting process

The observable object is the random step function

$$N(t) = \#\{n : t_n \leq t\},$$

the number of arrivals up to time t . It starts at $N(0) = 0$, never decreases, and jumps by one at each arrival. The number of arrivals in the interval $(t, t + \Delta t]$ is $N(t + \Delta t) - N(t)$.

C.6 Conditional intensity

At time t , ask for the probability that at least one arrival occurs in the next short interval of length Δt , given everything seen so far. The **conditional intensity** is that probability divided by Δt , in the limit of a very short interval:

$$\lambda(t \mid \mathcal{I}_{t-}) = \lim_{\Delta t \downarrow 0} \frac{1}{\Delta t} \mathbb{P}(N(t + \Delta t) - N(t) \geq 1 \mid \mathcal{I}_{t-}).$$

It is the instantaneous arrival rate given the history, in arrivals per second; in the rest of the paper it is written simply $\lambda(t)$. At an intensity of 300 per second the probability of an arrival in the next millisecond is about $\lambda(t) \Delta t = 300 \times 0.001 = 0.3$. For a short interval, and for the expected count over a longer one,

$$\mathbb{P}(\text{arrival in } (t, t + \Delta t]) \approx \lambda(t) \Delta t, \quad \mathbb{E}[N(t + w) - N(t)] = \mathbb{E} \int_t^{t+w} \lambda(t') dt'.$$

Specifying λ as a function of the history specifies the point process completely. For a Poisson process $\lambda(t) = \mu$ and the history carries no information. For a Hawkes process the intensity is μ plus a decaying contribution from each past arrival.

C.7 Three canonical models: Poisson, renewal and Hawkes

(a) Poisson. $\lambda(t) = \mu > 0$. Gaps are independent and exponential with mean $1/\mu$; the count in a window of length w is Poisson with mean μw . Every summary statistic takes its reference value: the Fano factor is 1 at every w , the Hurst exponent is 0.5, and the gaps are uncorrelated.

(b) Renewal process. The gaps $\Delta t_n = t_n - t_{n-1}$ are independent draws from a common distribution, not necessarily exponential. The intensity depends only on the time since the last arrival. A renewal process is Poisson only when the gaps are exponential. Heavy-tailed gaps produce many short gaps and a Fano factor well above 1, but the Fano factor levels off as w grows instead of continuing to rise, and the gaps are uncorrelated. Randomly shuffling the order of the gaps leaves a renewal process statistically unchanged; comparing a statistic on a real stream with the same statistic on its shuffled gaps therefore separates what the *order* of the gaps carries from what their *lengths* carry.

(c) Hawkes (self-exciting) process. Each past arrival raises the intensity for a while:

$$\lambda(t) = \mu + \sum_{n: t_n < t} \phi(t - t_n).$$

$\mu \geq 0$ is the **background rate**, the intensity with no prior arrivals. ϕ is the **kernel**: an arrival u seconds ago still contributes $\phi(u)$ to the intensity now. The kernel is the shape of one event's influence on the future. Dropping several stones into a pond is the usual picture: the disturbance at any moment is the sum of the ripples still spreading, each faded by the time since its stone fell. For the self-exciting specification used here the kernel is non-negative, so a past event can only raise future intensity (inhibitory extensions are discussed in Section 7.11), and its total area must be finite, so that the past does not accumulate without bound. This is the one-type case of the multivariate model in Section 7.

Kernel families. The *exponential kernel* $\phi(u) = \alpha e^{-\omega u}$ has two parameters: $\alpha \geq 0$, the jump in intensity immediately after an arrival, and $\omega > 0$, the decay rate, whose inverse $1/\omega$ is the kernel’s decay time. It is Markovian: the current intensity summarises everything needed from the past. Right after an arrival the intensity rises by α ; between arrivals it relaxes toward μ as $\mu + (\lambda(t_n) - \mu) e^{-\omega u}$; so it can be updated in constant time per arrival—a useful property for a low-latency system. The *power-law kernel* $\phi(u) = \alpha (1 + u/u_0)^{-(1+\zeta)}$, with the same jump α , a time scale $u_0 > 0$ and an exponent $\zeta > 0$, decays more slowly; [Hardiman et al. \(2013\)](#) find power-law kernels on E-mini S&P mid-price changes over lags from seconds to days.

C.8 Branching ratio

$$\Gamma = \int_0^\infty \phi(u) du, \quad \Gamma = \alpha/\omega \text{ for the exponential kernel.}$$

Γ is the mean number of arrivals that one arrival triggers *directly*. The Hawkes process is equivalently a **branching process** ([Hawkes and Oakes, 1974](#)): every arrival is either an *immigrant*, generated by the background rate μ , or the *child* of an earlier arrival, and each arrival has on average Γ children. The expected size of the family descended from one immigrant is the geometric series

$$1 + \Gamma + \Gamma^2 + \Gamma^3 + \dots = \frac{1}{1 - \Gamma} \quad (\Gamma < 1),$$

which is the one-type case of the series for $(I - \Gamma)^{-1}$ in Section 7. For $\Gamma < 1$ the process is *subcritical*: families are finite and the long-run rate is $\bar{\lambda} = \mu/(1 - \Gamma)$. As $\Gamma \rightarrow 1$ it becomes *near-critical*: family sizes and durations grow without bound and long-range dependence appears. At $\Gamma = 1$ the process is *critical*: each family still dies out with probability one, but its expected size is infinite, and with a positive background rate there is no stationary version. For $\Gamma > 1$ it is *supercritical*: with positive probability one arrival starts a cascade that never ends, and the intensity grows without bound.

On CME NQ market data, an exponential-kernel fit gives $\Gamma \approx 0.80$ on both packets and matching-engine transactions, with a kernel decay time $1/\omega$ of about $167 \mu\text{s}$: four of every five arrivals are triggered by earlier ones ([Maciejewski, 2026a](#)). On E-mini S&P futures, [Filimonov and Sornette \(2012\)](#) report 0.7–0.8 after 2007, with spikes to 0.9, and [Hardiman et al. \(2013\)](#) report values near 1 for mid-price changes; [Filimonov and Sornette \(2015\)](#) show that a fit with a constant background rate, over a window in which the true background varies, overstates Γ .

The critical boundary in practice. At $\Gamma = 1$ the stationary rate $\mu/(1 - \Gamma)$ is infinite. On a finite window the process still fires finitely often, but the variance of the count grows faster than linearly with the window. A maximum-likelihood fit that lands at $\Gamma = 1$ on real data may signal non-stationarity, a mis-specified kernel or a numerical problem rather than a genuinely critical market; whether markets are critical or merely close to it is itself debated ([Hardiman et al., 2013](#); [Filimonov and Sornette, 2015](#)).

C.9 What Hawkes processes look like

The long-run rate $\bar{\lambda} = \mu/(1 - \Gamma)$ and the branching ratio Γ look different in an event raster. Figure 6 simulates an exponential-kernel Hawkes process over 15 seconds at nine combinations, $\bar{\lambda} \in \{2, 10, 50\}$ events per second and $\Gamma \in \{0.30, 0.60, 0.90\}$, with a kernel decay time of one second. At low rate and low branching ratio the stream is nearly Poisson: arrivals spread evenly, the count grows in a straight line, the intensity is flat. At high rate and low branching ratio it is dense but still nearly Poisson. At low rate and high branching ratio the mean is 2 per second but arrivals come in unmistakable

bursts, with long empty stretches between them and the intensity reaching 20–30 per second inside a burst. At high rate and high branching ratio the intensity swings by an order of magnitude within the window.

C.10 Three signatures that separate Poisson, renewal and Hawkes processes

(i) *An excess of short gaps.* On CME NQ packet data 73% of gaps are shorter than a tenth of the mean gap; any exponential gives 9.52%. A Hawkes process produces such an excess, but so does a renewal process with heavy-tailed gaps, so this signature rejects Poisson without identifying self-excitation.

(ii) *Correlated gaps.* Under self-excitation, short gaps tend to follow short gaps and long gaps follow long ones, so successive gaps are positively correlated. A renewal process has independent gaps. A rate that varies slowly over the observation window also produces positive gap correlation, so this signature has to be checked against that alternative.

(iii) *Burstiness that grows with the time scale.* The Fano factor rises with the window length on a Hawkes process and levels off on a Poisson or renewal process. Clustering is then not a feature of one time scale but looks similar over decades of window lengths. A slowly varying rate can also make the Fano factor rise, which is the effect [Filimonov and Sornette \(2015\)](#) warn about.

C.11 Fano factor

For a window length w , cut the observation period into Υ consecutive windows and let $N_{w,v} = N(vw) - N((v-1)w)$ be the count in window v . The Fano factor, also called the *index of dispersion*, is

$$D(w) = \frac{\text{Var}[N_{w,v}]}{\mathbb{E}[N_{w,v}]},$$

the variance of the window counts divided by their mean. For a Poisson process $D(w) = 1$ at every w . For a renewal process with finite-variance gaps $D(w)$ tends to a constant as w grows. For a clustered process $D(w) > 1$, and under self-excitation or a varying rate it keeps growing with w . The measure is named after U. Fano, who introduced it for fluctuations in counts of ions. On CME NQ packet data $D(w)$ grows from about 52 at $w = 0.1$ s to about a thousand at $w = 60$ s, while the same stream with its gaps shuffled stays flat at about 17 ([Maciejewski, 2026a](#)).

C.12 Hurst exponent from the scaling of the Fano factor

When the Fano factor grows as a power of the window length,

$$D(w) \propto w^{2\mathcal{H}-1},$$

the exponent $\mathcal{H}$ is the *Hurst exponent*. $\mathcal{H} = 0.5$ means no long-range dependence (Poisson, and renewal processes with finite-variance gaps); $0.5 < \mathcal{H} < 1$ means long-range dependence, with clustering that looks similar across time scales; $\mathcal{H}$ near 1 means very long memory. It is estimated by a straight-line fit on a log–log plot,

$$\ln D(w) = (2\mathcal{H} - 1) \ln w + \text{constant},$$

so $\mathcal{H} = (\text{slope} + 1)/2$. On CME NQ packet data $\mathcal{H} \approx 0.74$; shuffling the gaps, which keeps their lengths and destroys their order, takes it to 0.50, so the long-range dependence lives in the order of the gaps ([Maciejewski, 2026a](#)).

C.13 Fitting an exponential-kernel Hawkes process

The input is a sorted list of arrival times $t_1 < \dots < t_{N_{\text{end}}}$ in a window $[0, t_{\text{end}}]$, where $N_{\text{end}} = N(t_{\text{end}})$ is the number of arrivals, and there are three parameters:

- μ , the **background rate**: arrivals per second with no recent history;
- α , the **jump**: the rise in intensity, in arrivals per second, at each arrival;
- ω , the **decay rate**: each jump fades as $e^{-\omega u}$, with decay time $1/\omega$.

Fitting means choosing the parameters under which the observed timestamps are most probable, that is, maximising the log-likelihood (Ozaki, 1979)

$$\ln \text{Lik}(\mu, \alpha, \omega) = \sum_{n=1}^{N_{\text{end}}} \ln \lambda(t_n) - \int_0^{t_{\text{end}}} \lambda(t') dt'. \quad (46)$$

In words: the first term rewards the model for predicting a high rate at the moments arrivals actually happened; the second penalises it for predicting more arrivals in total than occurred. If α is too small the model cannot explain the bursts; if α approaches ω it predicts more clustering than there is; a wrong μ misexplains the quiet periods.

For the exponential kernel both terms can be computed in one pass over the data. With

$$\Psi_n = e^{-\omega(t_n - t_{n-1})} (1 + \Psi_{n-1}), \quad \Psi_1 = 0,$$

the intensity at each arrival is $\lambda(t_n) = \mu + \alpha \Psi_n$, and the integral is

$$\int_0^{t_{\text{end}}} \lambda(t') dt' = \mu t_{\text{end}} + \frac{\alpha}{\omega} \sum_{n=1}^{N_{\text{end}}} (1 - e^{-\omega(t_{\text{end}} - t_n)}).$$

Timestamps are shifted to the start of the window before fitting, so that the exponentials do not underflow. The log-likelihood is then maximised numerically over (μ, α, ω) , for example with the Nelder–Mead method, and the optimum checked with a second optimiser. From the fitted parameters follow the branching ratio $\Gamma = \alpha/\omega$, the kernel decay time $1/\omega$, and the long-run rate $\bar{\lambda} = \mu/(1 - \Gamma)$: the background rate amplified by $1/(1 - \Gamma)$ through self-excitation.

C.14 Symbols used in this appendix

Symbol	Meaning
$\Omega, \mathcal{A}, \mathbb{P}$	sample space, σ -algebra, probability
$\mathcal{I}_t, \mathcal{I}_{t-}$	everything observable up to t inclusive / strictly before t
$t_1, t_2, \dots$	arrival times
$N(t)$	counting process, the number of arrivals up to t
$\lambda(t)$	conditional intensity, arrivals per second
μ	background rate
$\phi(u)$	Hawkes kernel
α, ω	exponential-kernel jump and decay rate; decay time $1/\omega$
Γ	branching ratio, $\int \phi$; α/ω for the exponential kernel
$\bar{\lambda} = \mu/(1 - \Gamma)$	long-run mean rate
$\Delta t_n = t_n - t_{n-1}$	gap between arrivals
w	window length for counts
$N_{w,v}, \Upsilon$	arrivals in window v ; number of windows
$D(w)$	Fano factor, variance over mean of the window counts
$\mathcal{H}$	Hurst exponent, $D(w) \propto w^{2\mathcal{H}-1}$
Ψ_n	running sum used to compute $\lambda(t_n)$ in one pass

Hawkes process across mean intensity $\bar{\lambda}$ and branching ratio n
each cell: event raster (top), cumulative count $N(t)$ (middle), analytic $\lambda(t)$ (bottom); $T = 15$ s, $\beta = 1/s$ (kernel decay time = 1 s)

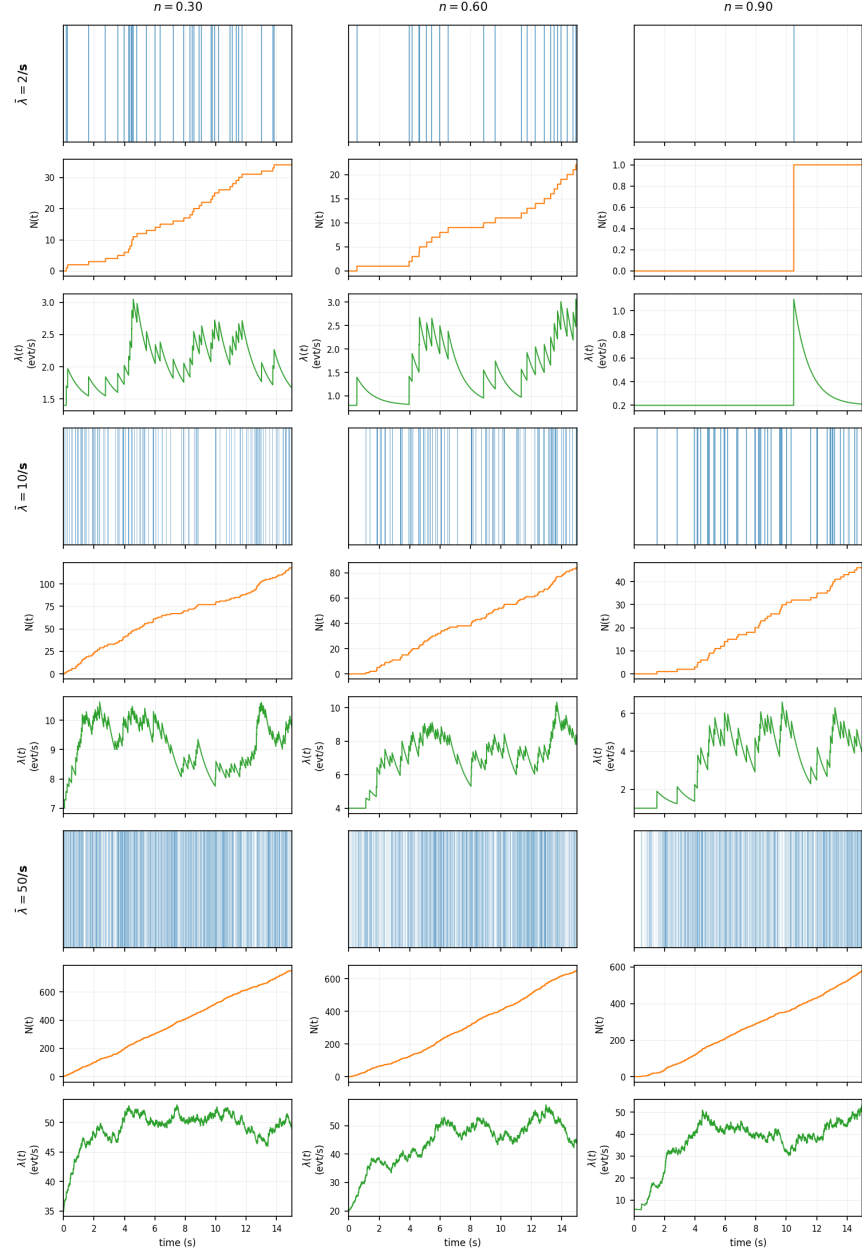


Figure 6: Simulated exponential-kernel Hawkes processes. Each cell stacks the event raster (top), the cumulative count $N(t)$ (middle) and the intensity $\lambda(t)$ (bottom). Rows: long-run rate $\bar{\lambda} \in \{2, 10, 50\}$ events per second. Columns: branching ratio $\Gamma \in \{0.30, 0.60, 0.90\}$. Fifteen seconds, decay rate $\omega = 1/s$. The time scale is chosen for legibility; the kernel fitted on CME data decays about four orders of magnitude faster. Reproduced from [Maciejewski \(2026a\)](#). Labels inside the image use the source’s notation: n for Γ , β for ω , F for D and τ for w .

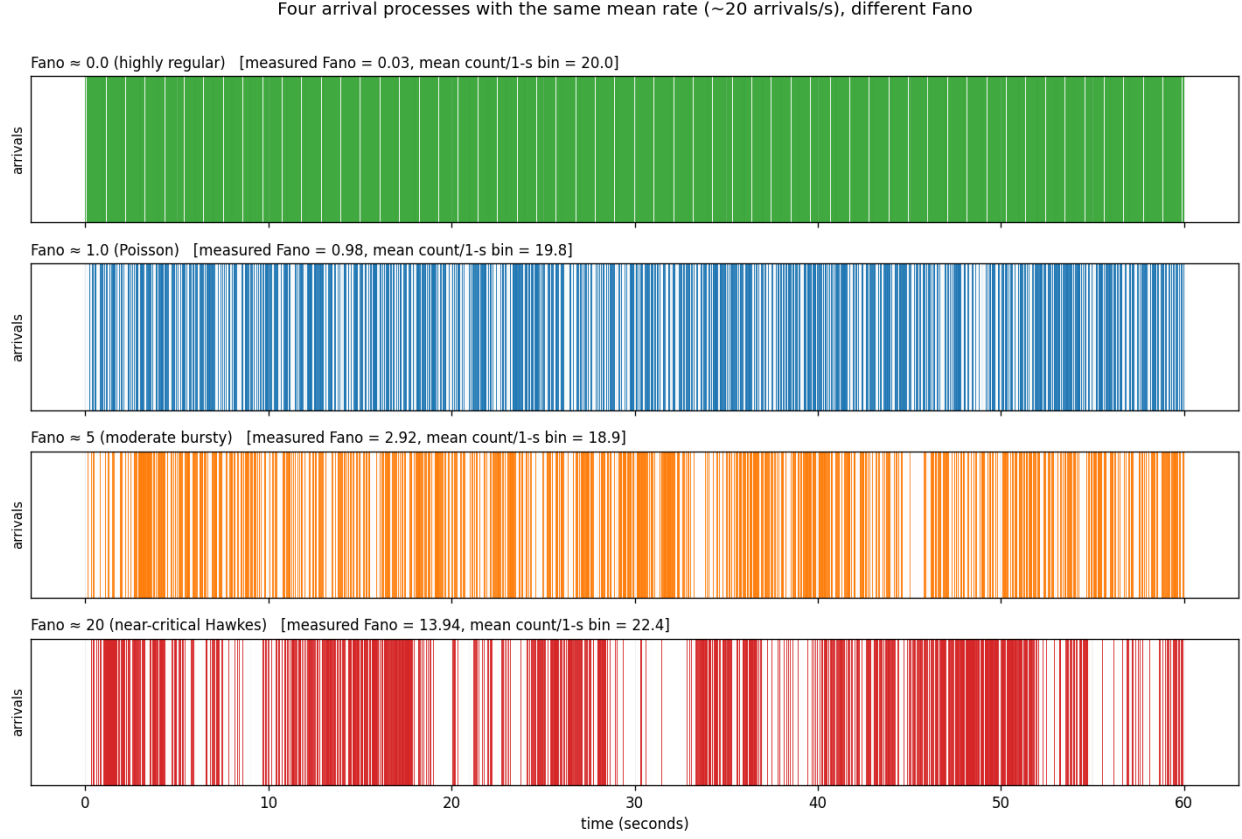


Figure 7: Four simulated processes at the same mean rate of about 20 per second. Top row ($D \approx 0$, highly regular): near-lattice arrivals, nearly equal counts per window. Second row ($D \approx 1$, Poisson): the reference. Third row ($D \approx 3$, moderate Hawkes): visible clumping. Bottom row ($D \approx 14$, near-critical Hawkes): pronounced bursts and long quiet stretches, with window counts from near zero to over 50. Reproduced from Maciejewski (2026a). Labels inside the image use the source’s notation: n for Γ , β for ω , F for D and τ for w .

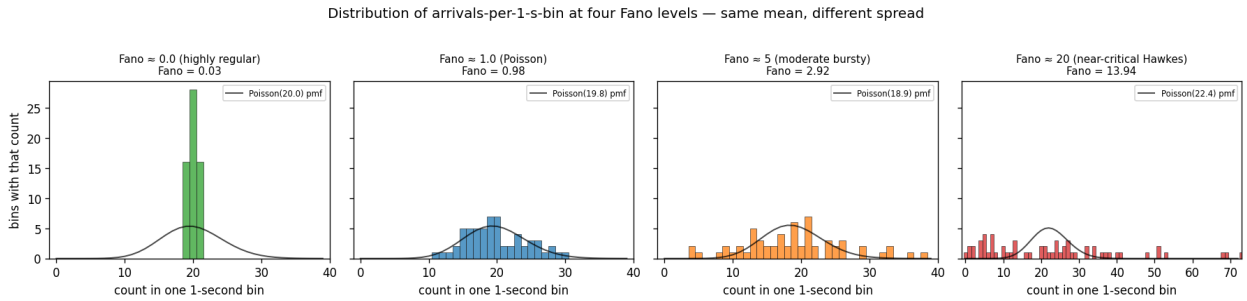


Figure 8: Distribution of arrivals per one-second window (bars) for the four processes of Figure 7, with the Poisson distribution of the same mean overlaid (curve). $D \approx 0$: narrower than Poisson. $D \approx 1$: matches Poisson. $D > 1$: wider than Poisson, with extra mass in both tails. The CME NQ packet stream has $D(1s) = 108$, far wider than any panel shown. Reproduced from Maciejewski (2026a). Labels inside the image use the source’s notation: n for Γ , β for ω , F for D and τ for w .

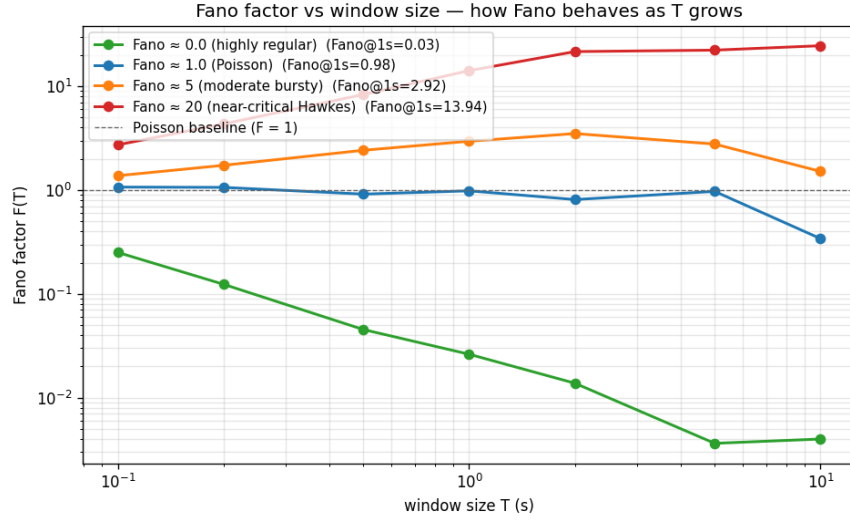


Figure 9: The Fano factor $D(w)$ against the window length w on logarithmic axes for four simulated processes. Regular: near zero. Poisson: flat at 1. Moderate Hawkes: rising. Near-critical Hawkes: a steeper rise, with slope about 0.5, i.e. $\mathcal{H} \approx 0.75$. Reproduced from [Maciejewski \(2026a\)](#). Labels inside the image use the source's notation: n for Γ , β for ω , F for D and τ for w .

References

- Frédéric Abergel, Marouane Anane, Anirban Chakraborti, Aymen Jedidi, and Ioane Muni Toke. *Limit Order Books*. Physics of Society: Econophysics and Sociophysics. Cambridge University Press, 2016. ISBN 9781107163980. doi: 10.1017/CBO9781316683040.
- Massil Achab, Emmanuel Bacry, Stéphane Gaïffas, Iacopo Mastromatteo, and Jean-François Muzy. Uncovering causality from multivariate Hawkes integrated cumulants. *Journal of Machine Learning Research*, 18(192):1–28, 2018. URL <http://jmlr.org/papers/v18/17-284.html>.
- Victor Agostinelli, Nicolas Bohm Agostini, and Antonino Tumeo. ELiTeFormer: An efficient transformer for FPGAs, 2026. URL <https://arxiv.org/abs/2607.03652>.
- Hirotugu Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974. doi: 10.1109/TAC.1974.1100705.
- Algo-Logic Systems and AMD. AMD Alveo U50 with HLS: FPGA accelerated CME tick-to-trade (solution brief). Solution brief. URL <https://www.amd.com/content/dam/amd/en/documents/products/accelerators/alveo/u50/algo-logic-t2t-product-brief.pdf>.
- Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *The Journal of Risk*, 3(2):5–39, 2001. doi: 10.21314/JOR.2001.041.
- AMD. AMD Alveo UL3524 accelerator card: FPGA accelerator for ultra-low latency trading (product brief). Product brief, 2023. URL https://www.xilinx.com/content/dam/xilinx/publications/product-briefs/2233051_Product_Brief_UL3524_Alveo_Accelerator_Card.pdf. Product page: <https://www.amd.com/en/products/accelerators/alveo/ul3524.html>; launch press release 2023-09-27.
- AMD. AMD Alveo UL3422 accelerator card (product brief). Product brief, 2024. URL <https://www.amd.com/content/dam/amd/en/documents/products/accelerators/alveo/ul3422/alveo-ul3422-product-brief.pdf>. Launch press release 2024-10-14.
- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=gerNCVqqtR>.
- Álvaro Arroyo, Álvaro Cartea, Fernando Moreno-Pino, and Stefan Zohren. Deep attentive survival analysis in limit order books: Estimating fill probabilities with convolutional-transformers. *Quantitative Finance*, 24(1):35–57, 2024. doi: 10.1080/14697688.2023.2286351.
- Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008. doi: 10.1080/14697680701381228.
- Emmanuel Bacry and Jean-François Muzy. Hawkes model for price and trades high-frequency dynamics. *Quantitative Finance*, 14(7):1147–1166, 2014. doi: 10.1080/14697688.2014.897000.
- Emmanuel Bacry and Jean-François Muzy. First- and second-order statistics characterization of Hawkes processes and non-parametric estimation. *IEEE Transactions on Information Theory*, 62(4):2184–2202, 2016. doi: 10.1109/TIT.2016.2533397.

- Emmanuel Bacry, Sylvain Delattre, Marc Hoffmann, and Jean-François Muzy. Modelling microstructure noise with mutually exciting point processes. *Quantitative Finance*, 13(1):65–77, 2013. doi: 10.1080/14697688.2011.647054.
- Emmanuel Bacry, Iacopo Mastromatteo, and Jean-François Muzy. Hawkes processes in finance. *Market Microstructure and Liquidity*, 1(1):1550005, 2015. doi: 10.1142/S2382626615500057.
- Emmanuel Bacry, Thibault Jaisson, and Jean-François Muzy. Estimation of slowly decreasing Hawkes kernels: Application to high-frequency order book dynamics. *Quantitative Finance*, 16(8):1179–1201, 2016. doi: 10.1080/14697688.2015.1123287.
- Emmanuel Bacry, Martin Bompaire, Philip Deegan, Stéphane Gaïffas, and Søren V. Poulsen. tick: a Python library for statistical learning, with an emphasis on Hawkes processes and time-dependent models. *Journal of Machine Learning Research*, 18(214):1–5, 2018. URL <http://jmlr.org/papers/v18/17-381.html>.
- David H. Bailey and Marcos López de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014. doi: 10.3905/jpm.2014.40.5.094.
- David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. Pseudomathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5):458–471, 2014. doi: 10.1090/noti1105.
- David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. The probability of backtest overfitting. *The Journal of Computational Finance*, 20(4):39–69, 2017. doi: 10.21314/JCF.2016.322.
- Hamidreza Banderalinaeini, Mohammad Sharifkhani, and Erfan Salavati. Attention-based multi-asset order flow networks for enhanced mid-price prediction. In *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF ’25)*, pages 525–533, 2025. doi: 10.1145/3768292.3770430.
- Matthew Baron, Jonathan Brogaard, Björn Hagströmer, and Andrei Kirilenko. Risk and return in high-frequency trading. *Journal of Financial and Quantitative Analysis*, 54(3):993–1024, 2019. doi: 10.1017/S0022109018001096.
- Peter L. Bartlett and Shahar Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3:463–482, 2002. URL <https://jmlr.org/papers/volume3/bartlett02a/bartlett02a.pdf>.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019. doi: 10.1073/pnas.1903070116.
- Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.
- Mark J. Bennett. GPU-accelerate algorithmic trading simulations by over 100x with Numba. NVIDIA Technical Blog, March 2025. URL <https://developer.nvidia.com/blog/gpu-accelerate-algorithmic-trading-simulations-by-over-100x-with-numba/>.

- Leonardo Berti and Gjergji Kasneci. TLOB: A novel transformer model with dual attention for price trend prediction with limit order book data, 2025.
- Leonardo Berti, Bardh Prenkaj, and Paola Velardi. TRADES: Generating realistic market simulations with diffusion models. In *ECAI 2025*, Frontiers in Artificial Intelligence and Applications, 2025. doi: 10.3233/FAIA251249.
- Pierre Blanc, Jonathan Donier, and Jean-Philippe Bouchaud. Quadratic Hawkes processes for financial prices. *Quantitative Finance*, 17(2):171–188, 2017. doi: 10.1080/14697688.2016.1193215.
- Thierry Bochud and Damien Challet. Optimal approximations of power laws with exponentials: Application to volatility models with long memory. *Quantitative Finance*, 7(6):585–589, 2007. doi: 10.1080/14697680701278291.
- Hamza Bodor and Laurent Carlier. Deep learning meets queue-reactive: A framework for realistic limit order book simulation, 2025.
- Jean-Philippe Bouchaud, Marc Mézard, and Marc Potters. Statistical properties of stock order books: Empirical results and models. *Quantitative Finance*, 2(4):251–256, 2002. doi: 10.1088/1469-7688/2/4/301.
- Jean-Philippe Bouchaud, Yuval Gefen, Marc Potters, and Matthieu Wyart. Fluctuations and response in financial markets: The subtle nature of ‘random’ price changes. *Quantitative Finance*, 4(2):176–190, 2004. doi: 10.1080/14697680400000022.
- Jean-Philippe Bouchaud, J. Dooyne Farmer, and Fabrizio Lillo. How markets slowly digest changes in supply and demand. In *Handbook of Financial Markets: Dynamics and Evolution*, pages 57–160. Elsevier, 2009. ISBN 9780123742582. doi: 10.1016/B978-012374258-2.50006-3.
- Jean-Philippe Bouchaud, Julius Bonart, Jonathan Donier, and Martin Gould. *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press, 2018. doi: 10.1017/9781316659335.
- Michael W. Brandt and Kenneth A. Kavajecz. Price discovery in the U.S. treasury market: The impact of orderflow and liquidity on the yield curve. *The Journal of Finance*, 59(6):2623–2654, 2004. doi: 10.1111/j.1540-6261.2004.00711.x.
- Pierre Brémaud and Laurent Massoulié. Stability of nonlinear Hawkes processes. *The Annals of Probability*, 24(3):1563–1588, 1996. doi: 10.1214/aop/1065725193.
- Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.
- Antonio Briola, Jeremy Turiel, and Tomaso Aste. Deep learning modeling of the limit order book: A comparative perspective, 2020.
- Antonio Briola, Silvia Bartolucci, and Tomaso Aste. Deep limit order book forecasting: A microstructural guide. *Quantitative Finance*, 25(7):1101–1131, 2025a. doi: 10.1080/14697688.2025.2522911.
- Antonio Briola, Silvia Bartolucci, and Tomaso Aste. HLOB–information persistence and structure in limit order books. *Expert Systems with Applications*, 266:126078, 2025b. doi: 10.1016/j.eswa.2024.126078.

- Emery N. Brown, Riccardo Barbieri, Valérie Ventura, Robert E. Kass, and Loren M. Frank. The time-rescaling theorem and its application to neural spike train data analysis. *Neural Computation*, 14(2):325–346, 2002. doi: 10.1162/08997660252741149.
- Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015. doi: 10.1093/qje/qjv027.
- David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS ’20)*, pages 11–22, 2020. doi: 10.1145/3384441.3395986.
- Charles Cao, Oliver Hansch, and Xiaoxin Wang. The information content of an open limit-order book. *Journal of Futures Markets*, 29(1):16–41, 2009. doi: 10.1002/fut.20334.
- Álvaro Cartea, Sebastian Jaimungal, and José Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015. ISBN 978-1-107-09114-6.
- Álvaro Cartea, Sebastian Jaimungal, and Yixuan Wang. Spoofing and price manipulation in order-driven markets. *Applied Mathematical Finance*, 27(1–2):67–98, 2020. doi: 10.1080/1350486X.2020.1726783.
- Junxiao Chen and Paul Glasserman. Do LLMs understand limit order book dynamics?, 2026.
- Hugh L. Christensen and Robert Woodmansey. Prediction of hidden liquidity in the limit order book of GLOBEX futures. *The Journal of Trading*, 8(3):68–95, 2013. doi: 10.3905/jot.2013.8.3.068.
- CME Group. iLink binary order entry. CME Group Client Systems Wiki, n.d.a. URL <https://cmegroupclientsite.atlassian.net/wiki/display/EPICSANDBOX/iLink+Binary+Order+Entry>.
- CME Group. CME Globex matching algorithm steps and supported matching algorithms. CME Group Client Systems Wiki, n.d.b. URL <https://cmegroupclientsite.atlassian.net/wiki/spaces/EPICSANDBOX/pages/457218521>.
- CME Group. CME MDP 3.0 market data. CME Group Client Systems Wiki, n.d.c. URL <https://cmegroupclientsite.atlassian.net/wiki/spaces/EPICSANDBOX/pages/457219613/CME+MDP+3.0+Market+Data>.
- Andrea Coletta, Matteo Prata, Michele Conti, Emanuele Mercanti, Novella Bartolini, Aymeric Moulin, Svitlana Vyetrenko, and Tucker Balch. Towards realistic market simulations: A generative adversarial networks approach. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF ’21)*, pages 1–9, 2021. doi: 10.1145/3490354.3494411.
- Andrea Coletta, Aymeric Moulin, Svitlana Vyetrenko, and Tucker Balch. Learning to simulate realistic limit order book markets from data as a world agent. In *Proceedings of the Third ACM International Conference on AI in Finance (ICAIF ’22)*, pages 428–436, 2022. doi: 10.1145/3533271.3561753.
- Commodity Futures Trading Commission. Antidisruptive practices authority. *Federal Register*, 78(102):31890–31897, May 2013. URL <https://www.federalregister.gov/documents/2013/05/28/2013-12365/antidisruptive-practices-authority>. Interpretive guidance and policy statement; 78 FR 31890 (FR Doc. 2013-12365).

- Commodity Futures Trading Commission. CFTC charges U.K. resident Navinder Singh Sarao and his company Nav Sarao Futures Limited PLC with price manipulation and spoofing. CFTC Press Release No. 7156-15, April 2015. URL <https://www.cftc.gov/PressRoom/PressReleases/7156-15>.
- Commodity Futures Trading Commission. CFTC orders JPMorgan to pay record \$920 million for spoofing and manipulation. CFTC Press Release No. 8260-20, September 2020. URL <https://www.cftc.gov/PressRoom/PressReleases/8260-20>.
- Rama Cont and Adrien de Larrard. Order book dynamics in liquid markets: Limit theorems and diffusion approximations, 2012.
- Rama Cont and Adrien de Larrard. Price dynamics in a Markovian limit order market. *SIAM Journal on Financial Mathematics*, 4(1):1–25, 2013. doi: 10.1137/110856605.
- Rama Cont, Sasha Stoikov, and Rishi Talreja. A stochastic model for order book dynamics. *Operations Research*, 58(3):549–563, 2010. doi: 10.1287/opre.1090.0780.
- Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014. doi: 10.1093/jjfinec/nbt003.
- Rama Cont, Mihai Cucuringu, and Chao Zhang. Cross-impact of order flow imbalance in equity markets. *Quantitative Finance*, 23(10):1373–1393, 2023. doi: 10.1080/14697688.2023.2236159.
- Corning. Corning SMF-28 Ultra optical fiber product information (PI1424). Product information sheet, 2013. URL <https://www.thelightconnection.com/pdf/CorningUltra.pdf>.
- David R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):187–202, 1972. doi: 10.1111/j.2517-6161.1972.tb00899.x.
- José Da Fonseca and Riadh Zaatour. Hawkes process: Fast calibration, application to trade clustering, and diffusive limit. *Journal of Futures Markets*, 34(6):548–579, 2014. doi: 10.1002/fut.21644.
- José Da Fonseca and Riadh Zaatour. Correlation and lead-lag relationships in a Hawkes microstructure model. *Journal of Futures Markets*, 37(3):260–285, 2017. doi: 10.1002/fut.21800.
- D. J. Daley and D. Vere-Jones. *An Introduction to the Theory of Point Processes, Volume I: Elementary Theory and Methods*. Probability and Its Applications. Springer, New York, 2nd edition, 2003. ISBN 0387955410. doi: 10.1007/b97277.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10148–10167. PMLR, 2024.
- Philippe Debie, Cornelis Gardebroek, Stephan Hageboeck, Paul van Leeuwen, Lorenzo Moneta, Axel Naumann, Joost M. E. Pennings, Andres A. Trujillo-Barrera, and Marjolein E. Verhulst. Unravelling the JPMorgan spoofing case using particle physics visualization methods. *European Financial Management*, 29(1):288–326, 2023. doi: 10.1111/eufm.12353.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.

- Han Ding, Yinheng Li, Junhao Wang, Hang Chen, Doudou Guo, and Yunbai Zhang. Large language model agent in financial trading: A survey, 2024. Journal reference on arXiv: International Conference on Computers in Management and Business 2026.
- Matthew Dixon. Sequence classification of the limit order book using recurrent neural networks. *Journal of Computational Science*, 24:277–286, 2018. doi: 10.1016/j.jocs.2017.08.018.
- Dobrislav Dobrev and Ernst Schaumburg. High-frequency cross-market trading: Model free measurement and applications. Working paper (version of october 15, 2017), Federal Reserve Board of Governors, 2017. URL <https://www.aeaweb.org/conference/2018/preliminary/paper/QdkHkBSZ>.
- Yifei Dong, Fengyi Wu, Kunlin Zhang, Yilong Dai, Sanjian Zhang, Wanghao Ye, Sihan Chen, and Zhi-Qi Cheng. Large language model agents in finance: A survey bridging research, practice, and real-world deployment. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17889–17907. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.972.
- DPDK Project. DPDK: Data plane development kit. Project website and documentation, n.d. URL <https://www.dpdk.org/>. Documentation: <https://doc.dpdk.org/guides/>.
- Nan Du, Hanjun Dai, Rakshit Trivedi, Utkarsh Upadhyay, Manuel Gomez-Rodriguez, and Le Song. Recurrent marked temporal point processes: Embedding event history to vector. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1555–1564, 2016. doi: 10.1145/2939672.2939875.
- J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, and Z. Wu. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation*, 13(07):P07027, 2018. doi: 10.1088/1748-0221/13/07/P07027.
- Derrick Gilchrist Edward Manoharan, Eljas Linna, Kestutis Baltakys, Hao Dong, and Juho Kanninen. UQ-LOB: Uncertainty-aware limit order book mid-price forecasting, 2026.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman & Hall, New York, 1993. ISBN 0412042312. doi: 10.1007/978-1-4899-4541-9.
- Jared F. Egginton, Bonnie F. Van Ness, and Robert A. Van Ness. Quote stuffing. *Financial Management*, 45(3):583–608, 2016. doi: 10.1111/fima.12126.
- Zoltán Eisler, Jean-Philippe Bouchaud, and Julien Kockelkoren. The price impact of order book events: Market orders, limit orders and cancellations. *Quantitative Finance*, 12(9):1395–1419, 2012. doi: 10.1080/14697688.2010.528444.
- Exegy. Exegy acquires Enyx, expands global FPGA leadership. Press release, May 2022. URL <https://www.exegy.com/exegy-acquires-enyx/>.
- J. Doyne Farmer, Paolo Patelli, and Ilija I. Zovko. The predictive power of zero intelligence in financial markets. *Proceedings of the National Academy of Sciences*, 102(6):2254–2259, 2005. doi: 10.1073/pnas.0409157102.
- Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.

- Vladimir Filimonov and Didier Sornette. Quantifying reflexivity in financial markets: Toward a prediction of flash crashes. *Physical Review E*, 85:056108, 2012.
- Vladimir Filimonov and Didier Sornette. Apparent criticality and calibration issues in the Hawkes self-excited point process model: Application to high-frequency financial data. *Quantitative Finance*, 15(8):1293–1314, 2015.
- Sascha Yves Frey, Kang Li, Peer Nagy, Silvia Sapora, Christopher Lu, Stefan Zohren, Jakob Foerster, and Anisoara Calinescu. JAX-LOB: A GPU-accelerated limit order book simulator to unlock large scale reinforcement learning for trading. In *Proceedings of the 4th ACM International Conference on AI in Finance (ICAIF '23)*, pages 583–591, 2023. doi: 10.1145/3604237.3626880.
- Stefan Frey and Patrik Sandås. The impact of iceberg orders in limit order books. *Quarterly Journal of Finance*, 7(3):1750007, 2017. doi: 10.1142/S2010139217500070.
- Anningzhe Gao, Shan Dai, and Yan Hu. Mamba Hawkes process, 2024.
- Robert Garrison, Pankaj Jain, and Mark Paddrik. Cross-asset market order flow, liquidity, and price discovery. OFR Working Paper 19-04, Office of Financial Research, 2019. URL https://www.financialresearch.gov/working-papers/files/OFRwp-19_04_cross-asset-market-order-flow-liquidity-and-price-discovery.pdf.
- Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6):933–949, 2018. doi: 10.1080/14697688.2017.1393551.
- Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.
- Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985. doi: 10.1016/0304-405X(85)90044-3.
- Tilman Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Adaptive Computation and Machine Learning. MIT Press, 2016. ISBN 9780262035613.
- Martin D. Gould and Julius Bonart. Queue imbalance as a one-tick-ahead price predictor in a limit order book. *Market Microstructure and Liquidity*, 2(2):1650006, 2016. doi: 10.1142/S2382626616500064.
- Martin D. Gould, Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison. Limit order books. *Quantitative Finance*, 13(11):1709–1742, 2013. doi: 10.1080/14697688.2013.803148.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.

- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
- Peter R. Hansen, Asger Lunde, and James M. Nason. The model confidence set. *Econometrica*, 79(2):453–497, 2011. doi: 10.3982/ECTA5771.
- Stephen J. Hardiman and Jean-Philippe Bouchaud. Branching-ratio approximation for the self-exciting Hawkes process. *Physical Review E*, 90(6):062807, 2014. doi: 10.1103/PhysRevE.90.062807.
- Stephen J. Hardiman, Nicolas Bercot, and Jean-Philippe Bouchaud. Critical reflexivity in financial markets: A Hawkes process analysis, 2013.
- Larry Harris. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, New York, 2002. ISBN 9780195144703. doi: 10.1093/oso/9780195144703.001.0001.
- Campbell R. Harvey, Yan Liu, and Heqing Zhu. ... and the cross-section of expected returns. *The Review of Financial Studies*, 29(1):5–68, 2016. doi: 10.1093/rfs/hhv059.
- Joel Hasbrouck. Intraday price formation in U.S. equity index markets. *The Journal of Finance*, 58(6):2375–2400, 2003. doi: 10.1046/j.1540-6261.2003.00609.x.
- Joel Hasbrouck. *Empirical Market Microstructure: The Institutions, Economics, and Econometrics of Securities Trading*. Oxford University Press, New York, 2007. ISBN 9780195301649. doi: 10.1093/oso/9780195301649.001.0001.
- Joel Hasbrouck and Gideon Saar. Low-latency trading. *Journal of Financial Markets*, 16(4):646–679, 2013. doi: 10.1016/j.finmar.2013.05.003.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2nd edition, 2009. ISBN 9780387848570. doi: 10.1007/978-0-387-84858-7.
- Alan G. Hawkes. Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, 58(1):83–90, 1971. doi: 10.1093/biomet/58.1.83.
- Alan G. Hawkes and David Oakes. A cluster process representation of a self-exciting process. *Journal of Applied Probability*, 11(3):493–503, 1974.
- Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973. URL <https://www.ijcai.org/Proceedings/73/Papers/027B.pdf>.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Marc Hoffmann, Mathieu Rosenbaum, and Nakahiro Yoshida. Estimation of the lead-lag parameter from non-synchronous data. *Bernoulli*, 19(2):426–461, 2013. doi: 10.3150/11-BEJ407.
- Fengrui Hua, Hengyi Yang, Xinlei Hao, Haohan Zhang, Bokai Cao, Yiyan Qi, Jia Li, and Jian Guo. Agentic quantitative trading: A survey of workflows, systems, and evaluation, 2026.

- Ruihong Huang and Tomas Polak. LOBSTER: Limit order book reconstruction system. SSRN working paper, 2011.
- Weibing Huang, Charles-Albert Lehalle, and Mathieu Rosenbaum. Simulating and analyzing order book data: The queue-reactive model. *Journal of the American Statistical Association*, 110(509): 107–122, 2015. doi: 10.1080/01621459.2014.982278.
- John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 11th edition, 2022. ISBN 9780136939917.
- Nicolas Huth and Frédéric Abergel. High frequency lead/lag relationships – empirical facts. *Journal of Empirical Finance*, 26:41–58, 2014. doi: 10.1016/j.jempfin.2014.01.003.
- ITU-T. Recommendation G.114: One-way transmission time. International Telecommunication Union, May 2003. URL <https://www.itu.int/rec/T-REC-G.114-200305-I>.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713. IEEE, 2018. doi: 10.1109/CVPR.2018.00286.
- Konark Jain, Nick Firoozye, Jonathan Kochems, and Philip Treleaven. Limit order book simulations: A review, 2024.
- Thibault Jaisson and Mathieu Rosenbaum. Limit theorems for nearly unstable Hawkes processes. *The Annals of Applied Probability*, 25(2), 2015. doi: 10.1214/14-AAP1005.
- Thibault Jaisson and Mathieu Rosenbaum. Rough fractional diffusions as scaling limits of nearly unstable heavy tailed Hawkes processes. *The Annals of Applied Probability*, 26(5), 2016. doi: 10.1214/15-AAP1164.
- Jiwon Jung and Kiseop Lee. Attention-based reading, highlighting, and forecasting of the limit order book. *Quantitative Finance*, 25(7):1015–1027, 2025. doi: 10.1080/14697688.2025.2522914.
- Ehsan Kabir, Md. Arafat Kabir, Austin R. J. Downey, Jason D. Bakos, David Andrews, and Miaoqing Huang. FAMOUS: Flexible accelerator for the attention mechanism of transformer on UltraScale+ FPGAs. In *2024 International Conference on Field Programmable Technology (ICFPT)*, pages 1–2, 2024. doi: 10.1109/ICFPT64416.2024.11113430.
- R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- Maxime Kawawa-Beaudan, Srijan Sood, Kassiani Papasotiriou, Daniel Borrajo, and Manuela Veloso. TradeFM: A generative foundation model for trade-flow and market microstructure, 2026.
- Hyunjik Kim, Andriy Mnih, Jonathan Schwarz, Marta Garnelo, Ali Eslami, Dan Rosenbaum, Oriol Vinyals, and Yee Whye Teh. Attentive neural processes. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=SkE6PjC9KX>.
- Damian Kisiel and Denise Gorse. Axial-LOB: High-frequency trading with axial attention. In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1327–1333, 2022. doi: 10.1109/SSCI51031.2022.10022284.

- Petter N. Kolm and Nicholas Westray. Deep learning alpha signals from limit order books: Practical insights and lessons learned. Risk.net, Cutting Edge, August 2025. URL <https://www.risk.net/cutting-edge/7961982/deep-learning-alpha-signals-from-limit-order-books-practical-insights-and-lessons-learned>.
- Petter N. Kolm, Jeremy Turiel, and Nicholas Westray. Deep order flow imbalance: Extracting alpha at multiple horizons from the limit order book. *Mathematical Finance*, 33(4):1044–1081, 2023. doi: 10.1111/mafi.12413.
- Zhizhuo Kou, Holam Yu, Junyu Luo, Jingshu Peng, Xujia Li, Chengzhong Liu, Juntao Dai, Lei Chen, Sirui Han, and Yike Guo. Automate strategy finding with LLM in quant investment. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18517–18533. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.1005.
- Ian Kuon and Jonathan Rose. Measuring the gap between FPGAs and ASICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 26(2):203–215, 2007. doi: 10.1109/TCAD.2006.884574.
- Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315, 1985. doi: 10.2307/1913210.
- Luca Lalor and Anatoliy Swishchuk. Event-based limit order book simulation under a neural Hawkes process: Application in market-making. *Applied Mathematical Finance*, 32(2):128–155, 2025. doi: 10.1080/1350486X.2025.2548448.
- Jeremy Large. Measuring the resiliency of an electronic limit order book. *Journal of Financial Markets*, 10(1):1–25, 2007. doi: 10.1016/j.finmar.2006.09.001.
- Gregory Laughlin, Anthony Aguirre, and Joseph Grundfest. Information transmission between financial markets in Chicago and New York. *Financial Review*, 49(2):283–312, 2014. doi: 10.1111/fire.12036.
- Christian Leber, Benjamin Geib, and Heiner Litz. High frequency trading acceleration using FPGAs. In *2011 21st International Conference on Field Programmable Logic and Applications (FPL)*, pages 317–322, 2011. doi: 10.1109/FPL.2011.64.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.
- Eun Jung Lee, Kyong Shik Eom, and Kyung Suh Park. Microstructure-based manipulation: Strategic behavior and performance of spoofing traders. *Journal of Financial Markets*, 16(2):227–252, 2013. doi: 10.1016/j.finmar.2012.05.004.
- Erik Lewis and George Mohler. A nonparametric EM algorithm for multiscale Hawkes processes. Preprint, dated May 9, 2011, 2011. URL http://paleo.sscnet.ucla.edu/Lewis-Molher-EM_Preprint.pdf.
- Fabrizio Lillo and J. Doyne Farmer. The long memory of the efficient market. *Studies in Nonlinear Dynamics & Econometrics*, 8(3), 2004. doi: 10.2202/1558-3708.1226.
- Fabrizio Lillo, Szabolcs Mike, and J. Doyne Farmer. Theory for long memory in supply and demand. *Physical Review E*, 71(6):066122, 2005. doi: 10.1103/PhysRevE.71.066122.

- Eljas Linna, Kestutis Baltakys, Alexandros Iosifidis, and Juho Kanninen. LOBERT: Generative AI foundation model for limit order book messages, 2025.
- John W. Lockwood, Adwait Gupte, Nishit Mehta, Michaela Blott, Tom English, and Kees Vissers. A low-latency library in FPGA hardware for high-frequency trading (HFT). In *2012 IEEE 20th Annual Symposium on High-Performance Interconnects (HOTI)*, pages 9–16, 2012. doi: 10.1109/HOTI.2012.15.
- Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018. ISBN 978-1-119-48208-6.
- Xiaofei Lu and Frédéric Abergel. High-dimensional Hawkes processes for limit order books: Modelling, empirical analysis and numerical calibration. *Quantitative Finance*, 18(2):249–264, 2018. doi: 10.1080/14697688.2017.1403142.
- Lorenzo Lucchese, Mikko S. Pakkanen, and Almut E. D. Veraart. The short-term predictability of returns in order book markets: A deep learning perspective. *International Journal of Forecasting*, 40(4):1587–1621, 2024. doi: 10.1016/j.ijforecast.2024.02.001.
- Vincent Maciejewski. Packets, transactions and queues: Design principles for HFT systems from a measurement study of CME market data, 2026a.
- Vincent Maciejewski. Adapting the actor model of concurrency for high-frequency trading: Synchronous message delivery (fast_send) and a tick-to-book latency study, 2026b.
- Vincent Maciejewski. The queue is inside the packet: Market-data latency measured one message at a time. Technical note, M2 Technologies, 2026c.
- Vincent Maciejewski. Model-free passive execution via order-level shadowing, 2026d.
- Magmio. Magmio: FPGA system for ultra-low latency trading. Vendor website. URL <https://www.magmio.com/product>. Accessed 2026-10-02.
- Vincent Mayeski. Search versus abduction: Why LLMs cannot invent new alpha. Substack, September 2026a. URL <https://vincentmayeski.substack.com/p/search-versus-abduction-why-llms>.
- Vincent Mayeski. Using AI in trading strategy development: the AI quant agent. Substack, September 2026b. URL <https://vincentmayeski.substack.com/p/using-ai-in-trading-strategy-development>.
- Vincent Mayeski. LLM-assisted trading strategy search (LATSS). Substack, 2026c. URL <https://vincentmayeski.substack.com/p/llm-assisted-trading-strategy-search>.
- Hongyuan Mei and Jason Eisner. The neural Hawkes process: A neurally self-modulating multivariate point process. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 6754–6764, 2017.
- Mellanox Technologies. Accelerating electronic trading (solution brief). Solution brief, 2017. URL https://network.nvidia.com/pdf/applications/SB_HighFreq_Trading.pdf.
- Albert J. Menkveld. High frequency trading and the new market makers. *Journal of Financial Markets*, 16(4):712–740, 2013. doi: 10.1016/j.finmar.2013.06.006.

- Ciamac C. Moallemi and Kai Yuan. A model for queue position valuation in a limit order book. Working paper, Columbia Business School (initial version December 2016, revised June 2017), 2017. URL <https://moallemi.com/ciamac/papers/queue-value-2016.pdf>.
- Maxime Morariu-Patrichi and Mikko S. Pakkanen. State-dependent Hawkes processes and their application to limit order book modelling. *Quantitative Finance*, 22(3):563–583, 2022. doi: 10.1080/14697688.2021.1983199.
- Ioane Muni Toke. “Market making” in an order book model and its impact on the spread. In *Econophysics of Order-driven Markets*, New Economic Windows, pages 49–64. Springer, Milano, 2011. doi: 10.1007/978-88-470-1766-5_4. arXiv title: “Market making” behaviour in an order book model and its impact on the bid-ask spread.
- Peer Nagy, Sascha Frey, Silvia Sapor, Kang Li, Anisoara Calinescu, Stefan Zohren, and Jakob Foerster. Generative AI for end-to-end limit order book modelling: A token-level autoregressive generative model of message flow using a deep state space network. In *Proceedings of the 4th ACM International Conference on AI in Finance (ICAIF ’23)*, pages 91–99, 2023. doi: 10.1145/3604237.3626898.
- Peer Nagy, Sascha Frey, Kang Li, Bidipta Sarkar, Svitlana Vyetrenko, Stefan Zohren, Ani Calinescu, and Jakob Foerster. LOB-Bench: Benchmarking generative AI for finance – an application to limit order book data. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 45437–45460, 2025.
- Napatech. Napatech turbocharges Xelera software to deliver ultra-low latency AI inference for financial trading. Press release, May 2025. URL <https://www.napatech.com/media/press-releases/napatech-turbocharges-xelera-software-to-deliver-ultra-low-latency-ai-inference-for-financial-trading/>.
- Nasdaq. Nasdaq TotalView-ITCH 5.0 interface specification, 2015. URL https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCHSpecification_5.0.pdf. Version 5.0, 03/06/2015.
- Adamantios Ntakaris, Martin Magris, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018. doi: 10.1002/for.2543.
- Yosihiko Ogata. On Lewis’ simulation method for point processes. *IEEE Transactions on Information Theory*, 27(1):23–31, 1981. doi: 10.1109/TIT.1981.1056305.
- Yosihiko Ogata. Statistical models for earthquake occurrences and residual analysis for point processes. *Journal of the American Statistical Association*, 83(401):9–27, 1988. doi: 10.1080/01621459.1988.10478560.
- Maureen O’Hara. *Market Microstructure Theory*. Blackwell Publishers, Cambridge, MA, 1995. ISBN 9781557864437.
- Takahiro Omi, Naonori Ueda, and Kazuyuki Aihara. Fully neural network based model for general temporal point processes. In *Advances in Neural Information Processing Systems 32*, 2019.
- T. Ozaki. Maximum likelihood estimation of Hawkes’ self-exciting point processes. *Annals of the Institute of Statistical Mathematics*, 31(1):145–155, 1979.

- Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3760–3765, 2020. doi: 10.1109/TNNLS.2019.2944933.
- Andrei Popov and Martina Huber. Scaling temporal feature extraction for high-frequency crypto trading via hardware-aware state space design. *Computer Science Bulletin*, 9(1):177–187, 2026. doi: 10.71465/csb211.
- Matteo Prata, Giuseppe Masi, Leonardo Berti, Viviana Arrigoni, Andrea Coletta, Irene Cannistraci, Svitlana Vyetrenko, Paola Velardi, and Novella Bartolini. LOB-based deep learning models for stock price trend prediction: A benchmark study. *Artificial Intelligence Review*, 57(5), 2024. doi: 10.1007/s10462-024-10715-4.
- Eghbal Rahimikia, Hao Ni, and Weiguan Wang. Re(visiting) time series foundation models in finance, 2025.
- Marcello Rambaldi, Emmanuel Bacry, and Fabrizio Lillo. The role of volume in order book dynamics: A multivariate Hawkes process analysis. *Quantitative Finance*, 17(7):999–1020, 2017. doi: 10.1080/14697688.2016.1260759.
- Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, Marin Biloš, Sahil Garg, Anderson Schneider, Nicolas Chapados, Alexandre Drouin, Valentina Zantedeschi, Yuriy Nevmyvaka, and Irina Rish. Lag-Llama: Towards foundation models for probabilistic time series forecasting, 2023.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978. doi: 10.1214/aos/1176344136.
- William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1, Part 2):119–138, 1966. doi: 10.1086/294846.
- William F. Sharpe. The Sharpe ratio. *The Journal of Portfolio Management*, 21(1):49–58, 1994. doi: 10.3905/jpm.1994.409501.
- Oleksandr Shchur, Marin Biloš, and Stephan Günnemann. Intensity-free learning of temporal point processes. In *International Conference on Learning Representations (ICLR)*, 2020.
- Oleksandr Shchur, Ali Caner Türkmen, Tim Januschowski, and Stephan Günnemann. Neural temporal point processes: A review. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)*, pages 4585–4593, 2021. doi: 10.24963/ijcai.2021/623.
- Yu Shi, Zongliang Fu, Shuo Chen, Bohan Zhao, Wei Xu, Changshui Zhang, and Jian Li. Kronos: A foundation model for the language of financial markets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 25366–25373, 2026. doi: 10.1609/aaai.v40i30.39730. Vol. 40, no. 30.

- Zijian Shi and John Cartlidge. State dependent parallel neural Hawkes process for limit order book event stream prediction and simulation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*, pages 1607–1615, 2022. doi: 10.1145/3534678.3539462.
- Justin Sirignano and Rama Cont. Universal features of price formation in financial markets: Perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019. doi: 10.1080/14697688.2019.1622295.
- Justin A. Sirignano. Deep learning for limit order books. *Quantitative Finance*, 19(4):549–570, 2019. doi: 10.1080/14697688.2018.1546053.
- Eric Smith, J. Doyne Farmer, László Gillemot, and Supriya Krishnamurthy. Statistical theory of the continuous double auction. *Quantitative Finance*, 3(6):481–514, 2003. doi: 10.1088/1469-7688/3/6/307.
- Solarflare Communications and Arista Networks. 10Gb Ethernet: The foundation for low-latency, real-time financial services applications and other, latency-sensitive applications. Solution brief (Solarflare/Arista joint paper). URL https://www.arista.com/assets/data/pdf/JointPapers/Arista_Solarflare_Low_Latency_10GbE.pdf.
- Satyam Srivastava, Akhil Arunkumar, Nithesh Kurella, Amrit Panda, Gaurav Jain, Purushotham Kamath, Mark Wutzke, Arun Tiruvur, Mayank Mike Gupta, Ilya Soloveychik, Vamsi Darsi, Malav Dalal, Vinayak Patankar, Sasidhar Dudyala, Senthil Duraisamy, Santhosh Ramchandran, Raghav Venkatasubramanian, Yuwei Qin, Xin Wang, Jayaprakash Balachandran, Ali Murat Gok, Piotr Wojciechowski, Saliya Ekanayake, Chris Ng, Ranju Sarma, Shubhankit Rathore, Tristan Trouwen, Siwei Zhuang, Chris Nicol, and Sudeep Bhoja. Corsair: An in-memory computing chiplet architecture for inference-time compute acceleration. *IEEE Micro*, 45(5):30–42, 2025. doi: 10.1109/MM.2025.3593444.
- STAC. STAC report: New STAC-T0 results with an Exegy/AMD FPGA solution. STAC Research news, 2024. URL <https://docs.stacresearch.com/news/AMD240422>.
- Sasha Stoikov. The micro-price: A high-frequency estimator of future prices. *Quantitative Finance*, 18(12):1959–1966, 2018. doi: 10.1080/14697688.2018.1489139.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.
- Mehdi Tomas, Iacopo Mastromatteo, and Michael Benzaquen. How to build a cross-impact model from first principles: Theoretical requirements and empirical results. *Quantitative Finance*, 22(6):1017–1036, 2022. doi: 10.1080/14697688.2021.2020328.
- Dat Thanh Tran, Alexandros Iosifidis, Juho Kannianen, and Moncef Gabbouj. Temporal attention-augmented bilinear network for financial time-series data analysis. *IEEE Transactions on Neural Networks and Learning Systems*, 30(5):1407–1418, 2019. doi: 10.1109/TNNLS.2018.2869225.
- Abhinava Tripathi, Vipul, and Alok Dixit. Limit order books: A systematic review of literature. *Qualitative Research in Financial Markets*, 12(4):505–541, 2020. doi: 10.1108/QRFM-07-2019-0080.

- Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis. Forecasting stock prices from the limit order book using convolutional neural networks. In *2017 IEEE 19th Conference on Business Informatics (CBI)*, pages 7–12, 2017a. doi: 10.1109/CBI.2017.23.
- Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis. Using deep learning to detect price change indications in financial markets. In *2017 25th European Signal Processing Conference (EUSIPCO)*, pages 2511–2515, 2017b. doi: 10.23919/EUSIPCO.2017.8081663.
- Vladimir N. Vapnik. *Statistical Learning Theory*. Adaptive and Learning Systems for Signal Processing, Communications, and Control. Wiley, 1998. ISBN 978-0-471-03003-4.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5998–6008, 2017.
- Alejandro Veen and Frederic P. Schoenberg. Estimation of space–time branching process models in seismology using an EM-type algorithm. *Journal of the American Statistical Association*, 103(482):614–624, 2008. doi: 10.1198/016214508000000148.
- James Wallbridge. Transformers for limit order books, 2020.
- Ting Wang, Mark Bebbington, and David Harte. Markov-modulated Hawkes process with stepwise decay. *Annals of the Institute of Statistical Mathematics*, 64(3):521–544, 2012. doi: 10.1007/s10463-010-0320-7.
- Zhuohan Wang and Carmine Ventre. DiffVolume: Diffusion models for volume generation in limit order books. In *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF ’25)*, pages 587–595, 2025. doi: 10.1145/3768292.3770413.
- Zhuohan Wang and Carmine Ventre. DiffLOB: Diffusion models for counterfactual generation in limit order books. In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-26)*, pages 6564–6572, 2026. doi: 10.24963/ijcai.2026/730.
- Halbert White. A reality check for data snooping. *Econometrica*, 68(5):1097–1126, 2000. doi: 10.1111/1468-0262.00152.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 53140–53164. PMLR, 2024.
- Peng Wu, Marcello Rambaldi, Jean-François Muzy, and Emmanuel Bacry. Queue-reactive Hawkes models for the order flow, 2019.
- Yufei Wu, Mahmoud Mahfouz, Daniele Magazzeni, and Manuela Veloso. Towards robust representation of limit orders books for deep learning models, 2021.
- Xelera Technologies. Xelera Silva: Low-latency inference for gradient-boosted trees and neural networks. Product page. URL <https://www.xelera.io/products/silva>. Accessed 2026-10-02.

- Yue Xiao, Carmine Ventre, Yuhan Wang, Haochen Li, Yuxi Huan, and Buhong Liu. LiT: Limit order book transformer. *Frontiers in Artificial Intelligence*, 8:1616485, 2025. doi: 10.3389/frai.2025.1616485.
- Ke Xu, Martin D. Gould, and Sam D. Howison. Multi-level order-flow imbalance in a limit order book. *Market Microstructure and Liquidity*, 4(3–4):1950011, 2018. doi: 10.1142/S2382626619500114. Issue cover-dated 2018 per Crossref; published online 8 January 2020; arXiv July 2019.
- Yingxiang Yang, Jalal Etesami, Niao He, and Negar Kiyavash. Online learning for multivariate Hawkes processes. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Curran Associates, Inc., 2017. URL <https://papers.nips.cc/paper/2017/hash/92a0e7a415d64ebafcb16a8ca817cde4-Abstract.html>.
- Sungyeob Yoo, Hyunsung Kim, Jinseok Kim, Sunghyun Park, Joo-Young Kim, and Jinwook Oh. LightTrader: A standalone high-frequency trading system with deep learning inference accelerators and proactive scheduler. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1017–1030, 2023. doi: 10.1109/HPCA56546.2023.10070930.
- Tom Zahavy. LLMs can’t jump. Position paper (preprint), Google DeepMind, 2026. URL <https://www.tomzahavy.com/files/llms-cant-jump.pdf>.
- Ilia Zaznov, Julian Kunkel, Alfonso Dufour, and Atta Badii. Predicting stock price changes based on the limit order book: A survey. *Mathematics*, 10(8):1234, 2022. doi: 10.3390/math10081234.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://openreview.net/forum?id=Sy8gdB9xx>.
- Qiang Zhang, Aldo Lipani, Omer Kirnap, and Emine Yilmaz. Self-attentive Hawkes process. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 11183–11193, 2020.
- Ting Zhang, Chi Xie, and Gang-Jin Wang. Do order imbalances predict intraday returns? new evidence from the Chinese stock market. *Asia-Pacific Journal of Accounting & Economics*, pages 1–15, 2025. doi: 10.1080/16081625.2025.2604824. Published online 19 December 2025.
- Yanzhi Zhang, Yu Ma, Yilin Cheng, Jian Li, and Yitong Duan. M3: A state-event generative foundation model for market microstructure dynamics, 2026.
- Zihao Zhang and Stefan Zohren. Multi-horizon forecasting for limit order books: Novel deep learning approaches and hardware acceleration using intelligent processing units, 2021.
- Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019. doi: 10.1109/TSP.2019.2907260.
- Jinfeng Zhong, Emmanuel Bacry, Agathe Guilloux, and Jean-François Muzy. KANFormer for predicting fill probabilities via survival analysis in limit order books, 2025.
- Gilles Zumbach. Time reversal invariance in finance. *Quantitative Finance*, 9(5):505–515, 2009. doi: 10.1080/14697680802616712.

Simiao Zuo, Haoming Jiang, Zichong Li, Tuo Zhao, and Hongyuan Zha. Transformer Hawkes process. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 11692–11702, 2020.